Data Analyst - Convexity Sensitivity AI Agent

Final Project Report (Project 1B)

Duration, Convexity and Key-Rate Sensitivity of a 300-Bond Multi-Currency Portfolio under Monte Carlo, Machine-Learning and Yield-Curve Analytics

Item	Detail
Prepared by	**Sam Solomon**, Data Analyst Intern
Role in scenario	Lead Quantitative Analyst, Pinnacle Fixed Income Partners (fictional AMC)
Commissioned by	ZeTheta Algorithms Private Limited - Project 1B, 15-day AI-accelerated research project
Portfolio studied	300 bonds, 6 currencies, INR 3,200 crore (synthetic, seeded)
Report date	17 October 2026
Deliverable file name	421_SamSolomon_MainReport.docx / .pdf
Companion files	421_SamSolomon_ValidationWorkbook.xlsx; 421_SamSolomon_PythonCodebase.zip

Strictly Private and Confidential - Not for Circulation. All IP arising from this project is the property of ZeTheta Algorithms Pvt. Ltd.

Contents

Document Control, Provenance and Reading Guide....4
Purpose and structure....4
What is real, what is synthetic, what was not run....4
Corrections to figures printed in the brief....4
Structural inconsistencies in the brief, and how they were handled....5
Executive Summary....6
Five findings that change decisions....6
What remains to be done before production use....6
PART A: TRAINING MATERIAL AND LEARNING RESOURCES....7
Section A1: Domain Fundamentals - The Fixed Income Risk Ecosystem....7
Section A2: Python Implementations....11
Section A3: R and Excel/VBA Implementations....23
Section A4: Yield-Curve Modelling....25
Section A5: Power BI Data Model and Advanced DAX....28
Section A6: Comprehensive Worked Examples....30
Section A7: Portfolio-Scale Results (300-Bond Book)....32
Section A8: Data Schemas, Scenario Frameworks and Hedging....41
PART B: GAMIFIED SIMULATION BUSINESS CONCEPT - BOND RISK LAB....45
Section B1: Platform Overview and Architecture....45
Section B2: Scoring System and Game Mechanics....47
Section B3: Scenario Engine and Business Model....49
PART C: CASE STUDIES AND REAL-WORLD APPLICATIONS....52
Section C1: RBI Rate-Hike Cycle 2022-23 - Duration Risk Crystallised....52
Section C2: Silicon Valley Bank (March 2023) - Duration Mismatch and Negative Convexity....53
Section C3: Indian Gilt Rally 2019-20 - Convexity as Return Driver....55
Section C4: Perpetual Bonds - Yield-Dependent Duration and Optionality (Tata Sons-type Perpetual NCDs, AT1)....56
Section C5: Cross-Case Synthesis....58
PART D: 15-DAY PROJECT METHODOLOGY AND DELIVERABLES LOG....59
Section D1: Method and Reading Guide....59
Section D2: Chronological Execution Diary....59
PART E: NO-CODE TOOLS AND PLATFORMS GUIDE....62
Section E1: Python Environment....62
Section E2: R Environment....62

Section E3: Power BI and DAX (no-code route)....63
Section E4: Excel (no-code validation route)....63
Section E5: Data Sources....64
Section E6: AI-Assisted Development and Validation Log....64
PART F: SUBMISSION PROTOCOL AND SELF-ASSESSMENT....65
Section F1: File Naming and Consolidation....65
Section F2: GitHub Repository Preparation and Transfer....65
Section F3: Automatic Disqualifier Check....66
Section F4: Self-Assessment Against the 1,000-Point Rubric....66
References and Source Notes....68
Appendix: Reproducibility and Archive Contents....69

Document Control, Provenance and Reading Guide

Purpose and structure

This report answers the ZeTheta brief for Project 1B. A Chief Investment Officer needs an AI-assisted system that can stress-test a INR 3,200 crore multi-currency bond book under 1,000+ interest-rate scenarios inside 24 hours, quantify the INR effect of a 100 bp parallel shift, a steepening or a butterfly, and show where machine learning adds value beyond classical duration-convexity mathematics. The report follows the six parts of the brief: **Part A** training material and implementations, **Part B** the gamified Bond Risk Lab concept, **Part C** quantitative case studies, **Part D** the 15-day execution log, **Part E** the tools guide, and **Part F** submission protocol with a self-assessment.

What is real, what is synthetic, what was not run

Because this report is meant to be audited, every number is tagged by origin. Readers should apply the following legend throughout.

Table 1. Provenance legend used throughout the report

Tag	Meaning	Examples in this report
Computed	Produced by the Python modules listed in Section A2 and re-executed for this report; cross-checked in Excel where stated.	Bond prices, durations, convexities, KRDs, VaR/ES, ML RMSE, Nelson-Siegel fits, case-study decompositions.
Synthetic	Generated with a fixed random seed because the ZeTheta portfolio CSV and live CCIL/FIMMDA series were not part of the supplied files.	The 300-bond book, FX rates, the illustrative zero curves, the 1,000-day P&L history used for back-testing, the ML training universe.
Sourced	Taken from a primary or reputable public source and checked during preparation.	SVB 10-K figures, RBI repo path, 10-year G-sec yields quoted by Reuters/ICMR, SEBI perpetual-bond circular, Yes Bank AT1 write-off.
Assumed	A stated modelling assumption or the brief's own stylised input.	Fund-category duration/convexity pairs, SaaS pricing and adoption volumes, factor volatilities and correlations.
Not executed	Code is supplied but could not be run in the offline build sandbox.	XGBoost, TensorFlow/Keras and `shap` library paths; all R scripts; DAX measures; Power BI model extension.

Environment note. The sandbox that produced the results had NumPy, SciPy, pandas, scikit-learn and Matplotlib but not xgboost, tensorflow, shap or R. Executed ML results therefore use scikit-learn analogues (marked with an asterisk, e.g. XGBoost, *NeuralNet*) that share the specified hyper-parameters. The genuine XGBoost/Keras/SHAP/R code is included in Sections A2 and E and falls back automatically in the Python modules. Re-run on a machine with those packages before quoting their numbers.

Corrections to figures printed in the brief

While validating against Excel's PRICE, DURATION and MDURATION functions and against primary sources, several illustrative figures in the brief did not reproduce. They are corrected in this report; the table records each change so a reviewer can trace it.

Table 2. Verification log of brief figures

#	Item in brief	Brief value	Verified value	Basis
1	7.26% GOI 2033 @7.35%: price	99.4181	99.4150	Python and Excel PRICE agree to 6 d.p.
2	Macaulay / Modified duration	6.8831 / 6.6389	6.7536 / 6.5142	Excel DURATION / MDURATION agree to 6 d.p.
3	Convexity	62.16	53.68	Excel ladder; 22,945 / (99.415 x 4 x 1.0743)
4	DV01 per 100 face	0.0660	0.0648	ModDur x price / 10,000

#	Item in brief	Brief value	Verified value	Basis
5	Repriced +100 bp price	93.16 (est. 93.1263)	93.1978 (est. 93.2057)	Full repricing at 8.35%
6	GOI 2033 convexity in instrument table	58.20	53.68	Same bond, same yield as Example 1
7	Fed hikes before SVB failure	525 bp, Mar-22 to Jul-23	450 bp to 4.50-4.75% by Mar-23	Reuters, 2-Mar-2023; the 525 bp total post-dates the failure
8	SVB portfolio duration	5.6 y on $91B	HTM 6.2 y on $91.3B; total book 5.6 y incl. swaps	SVB FY2022 Form 10-K
9	India 10Y yield in hike cycle	6.8% to 8.3% (+150 bp)	+48 bp in FY2023 (6.84% to 7.32%)	Reuters, 31-Mar-2023
10	Gilt-fund loss of 16-18% in the hike cycle	Stated as outcome	A 250 bp full pass-through stress, not the observed outcome	10Y moved only c.19% of the repo change
11	Gilt rally 250 bp	250 bp move	10Y fell 158 bp (7.591% to 6.013%)	ICMR case catalogue
12	Python MC engine (brief A2.1)	Uses parallel factor only; appends P&L on repeat calls	3-factor, vectorised, deterministic seed	Section A2.3

Item 12 matters operationally: the brief's engine generated twist and butterfly shocks but never used them, so its VaR could not respond to curve-shape risk, and its `pnl` list silently doubled in length if `compute_pnl()` was called twice.

Structural inconsistencies in the brief, and how they were handled

- **Tenor set.** Section A5.2 requires 11 curve tenors (3M to 30Y including 15Y) while Day 4 lists 10 key-rate buckets (no 15Y). The curve is stored at all 11 tenors; Key Rate Durations use the 10 Day-4 buckets, with the 15Y point interpolated between the 10Y and 20Y keys.
- **Badges.** Part B says 20 achievement badges but lists ten. Ten additional badges are specified in Section B2.4 so the count is met.
- **Rubrics.** Part B defines a six-category, 1,000-point *game* rubric; Part F cites a seven-dimension, 1,000-point *project* rubric without weights. Section F3 states the weights assumed for the self-assessment.
- **Checklist carry-over.** Section F2 of the brief refers to 'MatRisk AI v1.0', Docker, DVC, MLflow and Sphinx. These appear to be template carry-over from another project; Section F handles them explicitly rather than ignoring them.

Executive Summary

The agent built for this project revalues a 300-bond, six-currency book worth **INR 3,200 crore** by vectorised cash-flow repricing, so a 10,000-scenario three-factor Monte Carlo (level, twist, butterfly) with full revaluation of every bond completes in seconds on a single core. That comfortably meets the CIO's '1,000 scenarios within 24 hours' requirement with room for ML-assisted exploration.

Table 3. Headline results

Metric (synthetic book)	Result	Reading
Portfolio modified duration / convexity	5.86 yrs / 64.9	Market-value weighted; multi-currency
Portfolio DV01	INR 1.876 crore per bp	About INR 18.8 million per basis point
+100 bp parallel: duration-only / with convexity / full repricing	-187.6 / -177.2 / -176.8 cr	Convexity recovers INR 10.4 crore (5.5% of the duration loss)
+300 bp parallel: duration-only / full repricing	-562.9 / -480.5 cr	Duration-only overstates the loss by 17.1%
1-day 95% / 99% VaR (3-factor, full revaluation)	INR 19.4 / 27.5 cr	Delta-normal check: 19.4 / 27.5 cr
1-day 95% / 99% Expected Shortfall	INR 24.3 / 31.1 cr	Average loss beyond VaR
Annual-horizon 95% VaR: duration-only vs duration+convexity	INR 316 vs 286 cr	Ignoring convexity overstates VaR by 10.3%
Nelson-Siegel fit (11 INR tenors)	R2 = 0.9972; RMSE 2.2 bp	Svensson R2 = 0.9978
Best ML pricer (test RMSE, price points)	NeuralNet* 0.386; RF 0.513; GBM* 0.857	Target < 0.50 met by the neural net only; all fail outside the training yield range
Verification	14/14 unit tests pass; Excel agrees to 6 d.p.	Section D and Part F evidence

Five findings that change decisions

1. **Convexity is a material correction only for large moves.** At +100 bp it offsets 5.5% of the duration loss; at +300 bp it offsets 16.6%. For daily risk (about 6 bp) it is irrelevant, which is why delta-normal and Monte Carlo daily VaR agree within 1-2%.
2. **Curve-shape risk is concentrated in the 7Y-20Y zone.** Key-rate durations peak at the 10Y bucket and 61% of duration sits in INR assets, so an RBI-driven long-end move matters more than the headline duration suggests.
3. **Treat ML pricers as accelerators, not oracles.** In range, the neural net prices to 0.39 points RMSE, but on yields 3 points above the training range tree models err by roughly 10-11 price points. Analytical duration+convexity is about 9x more accurate than the best ML model for a 9-year bond over +/-200 bp; ML only wins for a 30-year bond at the wings.
4. **Normal-distribution VaR fails back-tests when shocks are fat-tailed.** On a 750-day synthetic history the delta-normal 99% VaR produced 18 breaches against 7.5 expected (Kupiec p = 0.001); historical simulation produced 14 (p = 0.033).
5. **Perpetual and callable instruments need yield-dependent measures.** A callable 10-year bond shows negative effective convexity between about 5% and 10% yield; SEBI's 100-year deemed-maturity rule raises an AT1 bond's modified duration from 6.6 to 11.8 years.

What remains to be done before production use

- Replace the synthetic book and curves with the supplied portfolio CSV and a CCIL/FIMMDA download, then re-run `run_core.py`.
- Execute the XGBoost, Keras, SHAP and R paths; extend the Power BI file from 5 pages to the 8+ pages and 40+ measures specified (Section A5).
- Add spread (credit) duration and FX risk; the engine measures interest-rate risk only and applies one curve shock to all six currencies.

PART A: TRAINING MATERIAL AND LEARNING RESOURCES

Section A1: Domain Fundamentals - The Fixed Income Risk Ecosystem

A1.1 Why duration and convexity are the core of interest-rate risk

A bond is a bundle of dated cash flows. Its price is the sum of those cash flows discounted at the market yield, and because yield is the single dominant risk factor for sovereign and investment-grade credit, the sensitivity of that sum to yield is the first thing any treasury, ALM, pension or insurance risk function measures. For a bond with cash flows CF_i at times t_i (in years) and compounding frequency f:

$$P = \sum_{i=1}^{N} \frac{CF_i}{(1 + y/f)^{f t_i}} \tag{A.1}$$

The price-yield curve is convex: it bends upward away from its tangent. The slope of the tangent is the first-order sensitivity (duration); the bending is the second-order sensitivity (convexity). The two combine in the second-order Taylor approximation that the brief names as the core formula:

$$\frac{\Delta P}{P} \approx -D_{Mod}\,\Delta y + \frac{1}{2} C\,(\Delta y)^2 \tag{A.2}$$

where Delta-y is the yield change in decimal form (100 bp = 0.01). Figure 1 shows the three curves for the 7.26% 9-year bond used throughout this report: the tangent line (duration only) lies below the true price on both sides, and adding the convexity term closes almost all of the gap.

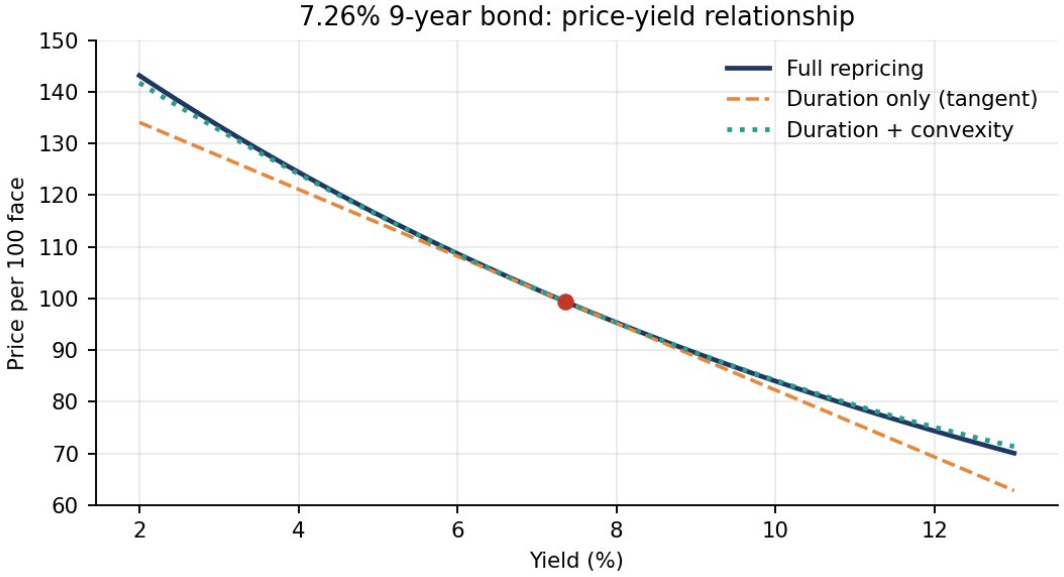


Figure 1. Price-yield relationship of the 7.26% 9-year bond (Computed). The tangent understates price at both low and high yields.

A1.2 Why convexity becomes critical beyond 50 bp

Duration is exact only for infinitesimal moves. The error of the duration-only estimate grows with the square of the yield change, so it is negligible for the 5-10 bp moves of a normal day and material for policy shocks. For the 9-year bond the duration-only error is already 0.02 price points at +/-25 bp, 0.27 at +/-100 bp and 2.2 at +/-300

bp (Figure 2); after adding convexity the residual error is an order of magnitude smaller at every shock size, and what remains is third-order (the change of convexity itself).

Table 4. Second-order estimates for the 7.26% 9-year bond (price per 100 face; Computed)

Shift (bp)	Repriced	Duration effect	Convexity effect	Estimate	Error	Convexity / | duration |
+100	93.198	-6.476	0.267	93.206	0.008	4.1%
-100	106.166	6.476	0.267	106.158	-0.008	4.1%
+200	87.468	-12.952	1.067	87.530	0.062	8.2%
+300	82.184	-19.428	2.402	82.388	0.204	12.4%

The brief states that for 100+ bp moves 'the convexity adjustment can represent 10-30% of the total price change'. The table shows the share depends on duration and shock: for this 6.5-duration bond it is 4.1% at 100 bp and 12.4% at 300 bp; for a 30-year bond (modified duration 13.4, convexity 282) it reaches 10.5% at 100 bp. The 10-30% range is therefore realistic only for long-dated bonds and 200 bp-plus shocks.

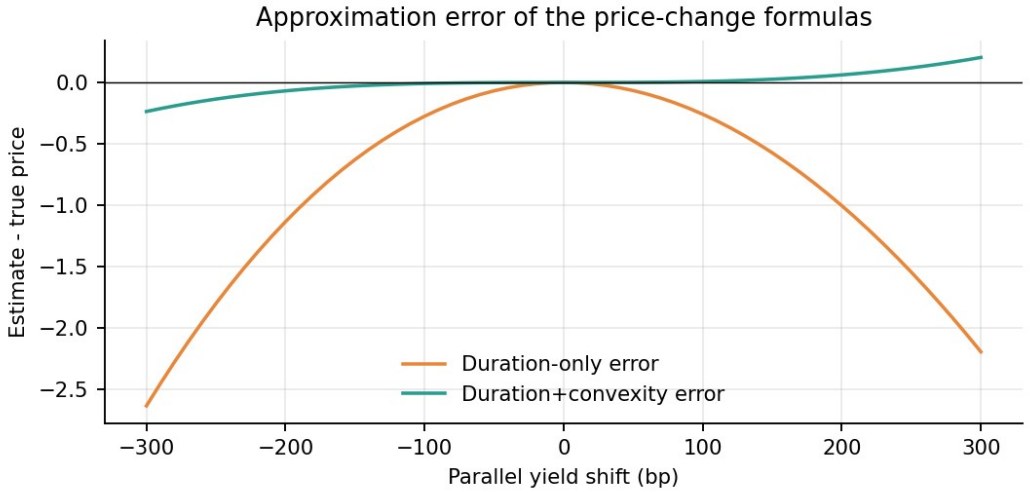


Figure 2. Estimation error of the duration-only and duration+convexity formulas versus full repricing (Computed).

A1.3 The measures: formal definitions and institutional use

Macaulay duration (Macaulay, 1938) is the present-value-weighted average time to receive the bond's cash flows, in years. It is the quantity SEBI and AMFI use to classify debt-fund categories (for example 'Low Duration' funds have Macaulay duration of 6-12 months).

$$D_{Mac} = \frac{1}{P}\sum_{i=1}^{N} t_i\, PV(CF_i) \tag{A.3}$$

Modified duration converts Macaulay duration into a percentage price sensitivity per unit yield change. It is the first-order coefficient in (A.2) and the number most portfolio managers quote.

$$D_{Mod} = \frac{D_{Mac}}{1 + y/f} = -\frac{1}{P}\frac{\partial P}{\partial y} \tag{A.4}$$

Effective duration and effective convexity replace the analytic derivatives with finite differences of a pricing model, which is the only coherent definition when cash flows themselves depend on yields (callable, putable, prepayable or perpetual-with-call instruments). The pricing function must re-run the option model at each bumped yield; bump sizes of 10-25 bp are typical for option-adjusted work because lattice prices are not smooth at 1 bp.

$$D_{eff} = \frac{P_- - P_+}{2\,P_0\,\Delta y}, \qquad C_{eff} = \frac{P_- + P_+ - 2P_0}{P_0\,(\Delta y)^2} \tag{A.5}$$

Key Rate Durations decompose the single scalar into a vector of sensitivities to local moves at chosen tenors. Each bucket is bumped with a triangular ('hat') shock that equals 1 at its key tenor and falls linearly to zero at the neighbouring keys, so the buckets sum to a unit parallel shift. This is what makes non-parallel scenarios computable: a steepening is simply a different weighting of the same vector.

$$KRD_k = -\frac{P(y_k + \delta) - P(y_k - \delta)}{2\,P_0\,\delta}, \qquad \sum_k KRD_k \approx D_{eff} \tag{A.6}$$

DV01 / PVBP is the absolute (currency) price change for one basis point. It is the operational currency of limits, hedging and ALCO reporting, because DV01s are additive across positions and across hedging instruments while durations are not.

$$DV01 = D_{Mod} \times P \times 10^{-4}, \qquad DV01_{port} = \sum_j q_j\,DV01_j \tag{A.7}$$

Convexity is the second derivative of price with respect to yield, scaled by price. For period index k_i = f t_i:

$$C = \frac{1}{P}\frac{\partial^2 P}{\partial y^2} = \frac{1}{P\,f^2(1 + y/f)^2}\sum_i k_i(k_i + 1)\,PV(CF_i), \quad k_i = f\,t_i \tag{A.8}$$

Table 5. Comparison of the sensitivity measures

Measure	Order	Units	Handles non-parallel moves?	Handles optionality?	Typical use
Macaulay duration	-	Years	No	No	Immunisation horizon, SEBI category
Modified duration	1st	% per 100% yield	No (parallel only)	No	Headline risk, parallel shocks
Effective duration	1st	% per 100% yield	No	Yes	Callables, MBS, perpetuals
Key-rate durations	1st	Years per bucket	Yes	If effective	Curve trades, ALCO buckets
DV01 / PVBP	1st	INR per bp	Per bucket if KRD-based	If effective	Limits, hedge ratios
Convexity	2nd	Years squared	Parallel part only	If effective	Large-move correction, barbell vs bullet

A1.4 Properties of convexity and the convexity premium

- **Zero-coupon bonds have the highest convexity for a given maturity** because the whole cash flow sits at the end (approximately T squared).
- **Longer maturity, lower coupon and lower yield all raise convexity**; the effect is roughly quadratic in maturity, so a 30-year bond has more than 4 times the convexity of a 10-year bond (282 for the 30-year bond against about 60 for a 10-year bond in this report's examples).
- **Non-callable bonds always have positive convexity**; the price-yield curve is convex everywhere.
- **Dollar (INR) convexity** = convexity x price, giving the INR second-order term 0.5 x INR-convexity x (Delta y)^2 used in portfolio P&L.

The convexity premium. Two portfolios with the same DV01 but different convexity do not have the same expected return in an efficient market. The higher-convexity portfolio earns more when yields move far in either direction, so investors pay for it through a lower yield. The barbell/bullet example in Section A8.4 quantifies this: a 2Y/30Y barbell matched to the duration of a 10-year bullet has convexity 119.7 versus 62.7, and earns +0.25% (parallel +100 bp) and +0.32% (parallel -100 bp) more than the bullet, at the cost of a yield give-up and exposure to steepening (-1.4% in a bear steepener relative to the bullet). The fair price of the convexity is the yield give-up

for which expected volatility gains exactly offset it: approximately 0.5 x (C_barbell - C_bullet) x sigma_y^2 per year. With 100 bp annual yield volatility that is 0.5 x 57 x 0.0001 = 0.29% per year, which is the breakeven carry give-up.

A1.5 Negative convexity: callable bonds and prepayable assets

When an issuer can redeem a bond at a fixed price, the bond's value is capped near that price as yields fall: the investor has written the issuer a call option. The callable bond's price equals a straight bond minus the call, and the call gains value as yields fall, so the callable's price rises more slowly than the straight bond's - its convexity turns negative and its effective duration *shortens* when yields fall and *extends* when yields rise. Agency mortgage-backed securities behave the same way because borrowers hold a prepayment option (Section C2).

The table and Figure 3 below are Computed from the model in `bond_pricing.callable_price_bachelier`: a 10-year 8% bond callable at 100 from year 3, with the call valued as a European option on the forward bond price under a normal model (100 bp annual yield volatility). It is an educational single-call approximation of an OAS lattice, but it reproduces the textbook behaviour: effective convexity is negative from about 5% to 10% yield, most negative (-66) near the coupon rate, and effective duration rises from 2.9 to 6.2 as yields climb from 5% to 11% (extension risk).

Table 6. Callable versus straight 10-year 8% bond, callable at par after year 3 (Computed)

Yield	Callable price	Straight price	Eff. duration (callable)	Eff. convexity (callable)	Mod. duration (straight)	Convexity (straight)
5%	108.02	123.38	2.90	-11.4	7.21	65.8
6%	104.79	114.88	3.20	-31.8	7.07	63.9
7%	101.23	107.11	3.76	-55.7	6.93	62.0
8%	97.12	100.00	4.57	-66.5	6.80	60.2
9%	92.38	93.50	5.41	-47.4	6.66	58.3
10%	87.22	87.54	6.01	-5.0	6.51	56.5
11%	82.02	82.07	6.24	32.1	6.37	54.7

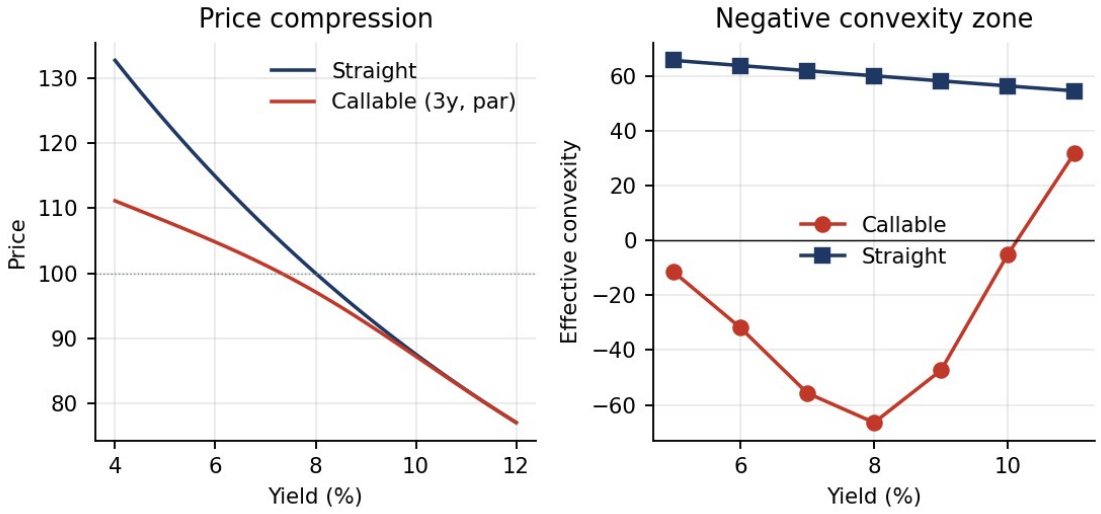


Figure 3. Negative convexity of the callable bond: price compression (left) and effective convexity by yield (right).

A1.6 Convexity of common Indian instruments

The brief tabulates durations and convexities for seven benchmark instruments. The table below recomputes them from cash flows (semi-annual coupons, yields assumed near current benchmark levels). The final column is the second-order gain at a 100 bp move, 0.5 x C x (0.01)^2, in percent of price.

Table 7. Duration and convexity of benchmark instruments (Computed; yields assumed)

Instrument	Coupon	Yield	Years	Price	ModDur	Convexity	Conv. gain at 100 bp
91-day T-Bill	0.00%	6.50%	0.25	98.41	0.242	0.18	+0.001%
364-day T-Bill	0.00%	6.60%	1.00	93.71	0.968	1.41	+0.007%
7.26% GOI 2033	7.26%	7.35%	9.00	99.41	6.514	53.68	+0.268%
7.54% GOI 2036	7.54%	7.45%	12.00	100.71	7.824	81.14	+0.406%
6.10% GOI 2054	6.10%	6.40%	30.00	96.02	13.381	281.55	+1.408%
AAA Corp 3Y 8.0%	8.00%	7.60%	3.00	101.06	2.628	8.57	+0.043%
AA Corp 5Y 9.0%	9.00%	8.70%	5.00	101.20	3.968	19.55	+0.098%

Two corrections to the brief's table follow from this. The 7.26% GOI 2033 convexity is 53.7, not 58.2, at the brief's own 7.35% yield; and the 6.10% GOI 2054 shows modified duration 13.4 (brief: 14.2), which would require a yield near 6.1% rather than the 6.4% assumed here. Neither changes any conclusion, but it is why every number in this report is generated by code rather than copied.

A1.7 Yield-curve dynamics and the three scenario factors

The yield curve is described economically by its shape and statistically by three factors that explain more than 95% of its variation in almost every market: **level** (parallel shift), **slope** or twist (steepening or flattening), and **curvature** or butterfly (belly versus wings). The report models each as a loading profile across tenors, normalised so that one unit of the factor has an unambiguous market meaning:

Table 8. Scenario factor definitions used by the Monte Carlo and stress engines

Factor	Definition of 1 bp of factor	Loading g(t)	Typical trigger
Level	All tenors move 1 bp	g(t) = 1	RBI repo move, global risk-off
Twist	10Y minus 2Y changes by 1 bp (pivot near 5Y)	g(t) = [G(t) - G(5)] / [G(10) - G(2)], G(t) = 1 - exp(-t/4); bounded at the long end	Operation Twist, fiscal supply, term-premium shifts
Butterfly	2 x 5Y minus 2Y minus 10Y changes by 1 bp	Gaussian bump centred on 5Y, normalised so 2h(5) - h(2) - h(10) = 1	Belly cheapening on supply, hawkish near-term surprise

- **Normal (upward sloping):** short rates below long rates; expansion; policy rate below neutral. This is the INR curve in the illustrative snapshot (3M 6.62%, 10Y 7.37%, 30Y 7.77%).
- **Flat:** short equals long; transition and uncertainty about the next move.
- **Inverted:** short above long; tight policy and recession expectations. The US 2s10s inversion of 2022-23 is the textbook case.
- **Humped:** the belly yields more than both ends; mixed expectations or heavy belly supply.

Indian market anchors. The repo rate anchors the short end of the INR curve; the LAF corridor (standing deposit facility to MSF) bounds the overnight rate; the 10-year benchmark G-sec is the reference for long-end pricing; SDLs trade at a state-credit spread over G-secs (35 bp assumed here); and RBI open-market operations - including Operation Twist-style simultaneous purchases of long bonds and sales of short bonds - are the main tool for reshaping the curve without moving the policy rate.

Section A2: Python Implementations

Eight Python modules implement the brief's required codebase (Deliverable 2), plus a test module. They are listed in full below so that the report is self-contained; the same files are delivered in `421_SamSolomon_PythonCodebase.zip`. All modules use relative imports only, no hard-coded file paths, and a fixed random seed for reproducibility.

Table 9. Python codebase map

Module	Responsibility	Key public functions / classes	Run status
bond_pricing.py	Cash-flow engine, price, durations, convexity, DV01, effective measures, perpetual and callable models	bond_price, macaulay_duration, modified_duration, convexity, dv01, effective_duration_convexity, callable_price_bachelier	Executed; 8 tests

Module	Responsibility	Key public functions / classes	Run status
portfolio_analytics.py	Synthetic 300-bond book, whole-book repricer, KRD matrix, contribution analysis, factor shapes	build_portfolio, BookRepricer, portfolio_krd, contribution	Executed; 3 tests
monte_carlo.py	BondRiskMC: Cholesky 3-factor scenarios, three P&L engines, VaR/ES	generate_scenarios, compute_pnl, var, es, parametric_var	Executed; 2 tests
ml_models.py	Dataset factory, RF / boosted trees / NN, train-evaluate with out-of-range stress set	make_dataset, build_models, train_and_evaluate	Executed (sklearn analogues)
shap_explainability.py	SHAP via library if available, otherwise exact Shapley enumeration	exact_shapley, explain, global_importance	Executed (exact path)
sensitivity_analysis.py	ML vs analytical price-change tables over -200..+200 bp	sensitivity_table, add_ensemble, error_summary	Executed
yield_curve.py	Nelson-Siegel, Svensson, bootstrapping, forwards, PCA	fit_nelson_siegel, fit_svensson, bootstrap_zero_curve, forward_rate	Executed; 2 tests
var_backtest.py	Breach counting, Kupiec POF test, Basel traffic light, rolling back-test	kupiec_pof, basel_zone, rolling_var_backtest	Executed; 1 test
ml_xgb_keras.py	Genuine XGBoost / Keras / shap-library path	script	Not executed
test_core.py	14 unittest cases	-	14 / 14 pass

A2.1 Core bond mathematics (bond_pricing.py)

The module generates cash-flow times from maturity backwards, which handles whole and fractional maturities with one code path (a short first period pays a full coupon, i.e. dirty-price convention with no accrual split). Duration and convexity are vectorised sums over the ladder, so a 30-year semi-annual bond costs 60 array elements rather than a Python loop. The brief's original `macaulay_duration` and `convexity` functions re-called `bond_price()` for the denominator and looped in Python; results are identical, and fractional maturities such as the 91-day T-bill (0.25 years) are now supported, which the integer-period loop could not do. (A micro-benchmark shows no speed gain for a single bond - about 18 microseconds versus 11 for the loop on a 9-year bond - so the benefit is generality here and vectorisation across the whole book in `portfolio_analytics.py`.)

Listing 10. bond_pricing.py (complete)

```
"""bond_pricing.py - core fixed-income mathematics for the Convexity Sensitivity AI Agent.

Conventions
-----------
* Yields are decimals (0.0735 = 7.35%), compounded `freq` times per year.
* Prices are per 100 face. Cash-flow times are in years and may be fractional;
  a short first period pays a full coupon (dirty-price style, no accrual split).
* Convexity is d2P/dy2 / P (units: years^2), so dP/P ~ -D*dy + 0.5*C*dy^2.
"""
from __future__ import annotations

import numpy as np


def cash_flows(face: float, coupon_rate: float, years: float, freq: int = 2):
    """Return (times_in_years, cash_flows) for a plain fixed-coupon bond."""
    if years <= 0:
        raise ValueError("years must be positive")
    step = 1.0 / freq
    n = int(np.ceil(years * freq - 1e-9))
    times = years - step * np.arange(n - 1, -1, -1)      # ascending, last == years
    cf = np.full(n, face * coupon_rate / freq)
    cf[-1] += face
    return times, cf


def bond_price(face, coupon_rate, ytm, years, freq=2):
    t, cf = cash_flows(face, coupon_rate, years, freq)
    return float(np.sum(cf / (1.0 + ytm / freq) ** (freq * t)))


def macaulay_duration(face, coupon_rate, ytm, years, freq=2):
    t, cf = cash_flows(face, coupon_rate, years, freq)
    pv = cf / (1.0 + ytm / freq) ** (freq * t)
    return float(np.sum(t * pv) / np.sum(pv))


def modified_duration(face, coupon_rate, ytm, years, freq=2):
    return macaulay_duration(face, coupon_rate, ytm, years, freq) / (1.0 + ytm / freq)
```

```
def convexity(face, coupon_rate, ytm, years, freq=2):
    t, cf = cash_flows(face, coupon_rate, years, freq)
    k = freq * t                                          # periods (may be fractional)
    pv = cf / (1.0 + ytm / freq) ** k
    return float(np.sum(k * (k + 1) * pv) / (np.sum(pv) * freq ** 2 * (1.0 + ytm / freq) ** 2))


def dv01(face, coupon_rate, ytm, years, freq=2):
    """Analytic DV01 per `face` of notional: ModDur x Price / 10,000."""
    return modified_duration(face, coupon_rate, ytm, years, freq) * bond_price(
        face, coupon_rate, ytm, years, freq) / 10_000.0


def dv01_numeric(face, coupon_rate, ytm, years, freq=2, bump=1e-4):
    p_dn = bond_price(face, coupon_rate, ytm - bump / 2, years, freq)
    p_up = bond_price(face, coupon_rate, ytm + bump / 2, years, freq)
    return p_dn - p_up


def effective_duration_convexity(price_fn, y0, dy=1e-4):
    """Finite-difference effective duration/convexity for any pricing function price_fn(y)."""
    p0, p_dn, p_up = price_fn(y0), price_fn(y0 - dy), price_fn(y0 + dy)
    eff_dur = (p_dn - p_up) / (2.0 * p0 * dy)
    eff_conv = (p_dn + p_up - 2.0 * p0) / (p0 * dy ** 2)
    return float(eff_dur), float(eff_conv)


def price_change_estimate(price, mod_dur, conv, dy):
    """Second-order price estimate; returns (duration_effect, convexity_effect, new_price)."""
    d_eff = -mod_dur * dy * price
    c_eff = 0.5 * conv * dy ** 2 * price
    return d_eff, c_eff, price + d_eff + c_eff


# ---- perpetuals (closed forms; annual compounding) ---------------------------------
def perpetual_metrics(y):
    """Annual-pay perpetual at yield y: price=c/y; ModDur=1/y; MacDur=(1+y)/y; Conv=2/y^2."""
    return {"mac_dur": (1 + y) / y, "mod_dur": 1.0 / y, "convexity": 2.0 / y ** 2}


# ---- callable bond via bond-minus-option (smooth, educational) --------------------
def callable_price_bachelier(face, coupon_rate, ytm, years, call_years, strike=100.0,
                             freq=2, yield_vol=0.0100):
    """Callable price = straight price - European call on the forward bond price.

    The call (exercisable at `call_years` at `strike`) is valued with a normal
    (Bachelier) model on the forward bond price; price vol = ModDur(remaining) x
    forward price x yield_vol. Smooth in yield, so finite-difference *effective*
    duration/convexity are stable. Approximation: single call date, flat curve.
    """
    from math import exp, sqrt
    from scipy.stats import norm
    straight = bond_price(face, coupon_rate, ytm, years, freq)
    rem = years - call_years
    p_call_date = bond_price(face, coupon_rate, ytm, rem, freq)       # value of remainder at call date
    md_rem = modified_duration(face, coupon_rate, ytm, rem, freq)
    # forward price of the remaining bond (strip coupons paid before the call date)
    t, cf = cash_flows(face, coupon_rate, years, freq)
    pv_pre = float(np.sum(cf[t <= call_years + 1e-9] / (1 + ytm / freq) ** (freq * t[t <= call_years + 1e-9])))
    df = (1 + ytm / freq) ** (-freq * call_years)
    fwd = (straight - pv_pre) / df
    sigma = md_rem * fwd * yield_vol * sqrt(call_years)
    d = (fwd - strike) / sigma
    call_val = (fwd - strike) * norm.cdf(d) + sigma * norm.pdf(d)
    return float(straight - df * call_val)


if __name__ == "__main__":
    p = bond_price(100, 0.0726, 0.0735, 9)
    print(round(p, 4), round(macaulay_duration(100, 0.0726, 0.0735, 9), 4),
          round(modified_duration(100, 0.0726, 0.0735, 9), 4),
          round(convexity(100, 0.0726, 0.0735, 9), 4),
          round(dv01(100, 0.0726, 0.0735, 9), 5))
```

A2.2 Portfolio engine and Key Rate Durations (portfolio_analytics.py)

Three design choices deserve explanation. First, **whole-book repricing is vectorised**: every cash flow of every bond is stacked into flat arrays, and `np.add.reduceat` collapses discounted flows back to bond prices, so a full revaluation of all 300 bonds under any curve shock takes about 0.07 milliseconds (measured). Second, **KRDs use hat functions on the zero curve** and central differences, so the ten KRDs sum to the effective duration under a parallel shift; the sum differs from the yield-based modified duration by only 0.55% for the book (5.829 versus

5.863), inside the brief's 1% tolerance. Third, **factor shapes are normalised** so that 1 bp of twist means a 1 bp change in the 10Y-2Y spread and 1 bp of butterfly means a 1 bp change in 2 x 5Y - 2Y - 10Y; without this, factor volatilities are not interpretable.

Listing 11. portfolio_analytics.py (complete)

```
"""portfolio_analytics.py - synthetic 300-bond multi-currency portfolio, bond-level and
portfolio-level duration / convexity / DV01, Key Rate Durations and contribution analysis.

All market inputs below are ILLUSTRATIVE snapshots (see report Section D.3). Replace
CURVES / FX with CCIL-FIMMDA / central-bank downloads for live use.
"""
from __future__ import annotations

import numpy as np
import pandas as pd
from scipy.optimize import brentq

from bond_pricing import cash_flows, bond_price, macaulay_duration, modified_duration, convexity

TENORS = np.array([0.25, 0.5, 1, 2, 3, 5, 7, 10, 15, 20, 30], dtype=float)   # 11 standard INR points
KEY_TENORS = np.array([0.25, 0.5, 1, 2, 3, 5, 7, 10, 20, 30], dtype=float)  # 10 KRD buckets
KEY_LABELS = ["3M", "6M", "1Y", "2Y", "3Y", "5Y", "7Y", "10Y", "20Y", "30Y"]

CURVES = {  # zero-rate knots in % (illustrative)
    "INR": [6.62, 6.64, 6.69, 6.78, 6.88, 7.05, 7.2, 7.37, 7.55, 7.66, 7.77],
    "USD": [4.35, 4.30, 4.20, 4.05, 4.00, 4.05, 4.15, 4.30, 4.65, 4.85, 4.85],
    "GBP": [4.20, 4.15, 4.05, 3.95, 3.95, 4.00, 4.15, 4.35, 4.75, 4.95, 4.90],
    "EUR": [2.05, 2.05, 2.10, 2.20, 2.30, 2.50, 2.65, 2.80, 3.10, 3.20, 3.15],
    "JPY": [0.50, 0.55, 0.65, 0.80, 0.95, 1.15, 1.35, 1.60, 2.05, 2.35, 2.65],
    "AUD": [3.80, 3.75, 3.70, 3.75, 3.85, 4.00, 4.15, 4.35, 4.65, 4.80, 4.80],
}
FX_INR = {"INR": 1.0, "USD": 88.0, "GBP": 115.0, "EUR": 100.0, "JPY": 0.58, "AUD": 58.0}  # illustrative
CCY_VOL_SCALE = {"INR": 1.0, "USD": 1.15, "GBP": 1.10, "EUR": 0.80, "JPY": 0.45, "AUD": 1.00}
TARGET_MV_INR = 3_200e7          # INR 3,200 crore
SEGMENT_SHARE = {"GSEC": 0.38, "SDL": 0.10, "CORP_AAA": 0.13, "CORP_AA": 0.08, "CORP_A": 0.04,
                 "UST": 0.10, "GILT": 0.05, "BUND": 0.06, "JGB": 0.03, "ACGB": 0.03}


def zero_rate(ccy: str, t):
    return np.interp(t, TENORS, np.array(CURVES[ccy]) / 100.0)


# ---------------------------------------------------------------------------------------
# Yield-curve shock shapes (loadings per 1 bp of the named factor)
# ---------------------------------------------------------------------------------------
def level_shape(t):
    return np.ones_like(np.asarray(t, dtype=float))


def _g(t):
    return 1.0 - np.exp(-np.asarray(t, dtype=float) / 4.0)


def twist_shape(t):
    """Slope factor normalised so that shock(10Y) - shock(2Y) = 1 (bounded at the long end)."""
    return (_g(t) - _g(5.0)) / (_g(10.0) - _g(2.0))


def _bump(t):
    return np.exp(-((np.asarray(t, dtype=float) - 5.0) / 3.0) ** 2)


def butterfly_shape(t):
    """Curvature factor normalised so that 2*s(5Y) - s(2Y) - s(10Y) = 1."""
    k = 2 * _bump(5.0) - _bump(2.0) - _bump(10.0)
    return _bump(t) / k


def hat(t, k_index: int):
    """Triangular key-rate bump: 1 at KEY_TENORS[k], linearly 0 at neighbours, flat beyond ends."""
    e = np.zeros(len(KEY_TENORS)); e[k_index] = 1.0
    return np.interp(t, KEY_TENORS, e)


# ---------------------------------------------------------------------------------------
# Portfolio construction (synthetic, seeded)
# ---------------------------------------------------------------------------------------
SEGMENTS = [  # name, ccy, sector, rating, count, spread_bps, freq
    ("GSEC", "INR", "Government", "SOV", 75, 0, 2),
    ("SDL", "INR", "Government", "SOV", 35, 35, 2),
    ("CORP_AAA", "INR", "Corporate", "AAA", 45, 70, 1),
    ("CORP_AA", "INR", "NBFC", "AA", 35, 130, 1),
    ("CORP_A", "INR", "NBFC", "A", 20, 230, 1),
    ("UST", "USD", "Government", "SOV", 25, 0, 2),
    ("GILT", "GBP", "Government", "SOV", 15, 0, 2),
```

```
    ("BUND", "EUR", "Government", "SOV", 20, 0, 1),
    ("JGB", "JPY", "Government", "SOV", 15, 0, 2),
    ("ACGB", "AUD", "Government", "SOV", 15, 0, 2),
]


def curve_price(face, cpn, years, freq, ccy, spread, bump_fn=None):
    t, cf = cash_flows(face, cpn, years, freq)
    z = zero_rate(ccy, t) + spread + (0.0 if bump_fn is None else bump_fn(t))
    return float(np.sum(cf / (1.0 + z / freq) ** (freq * t)))


def build_portfolio(seed: int = 42) -> pd.DataFrame:
    rng = np.random.default_rng(seed)
    rows, bid = [], 1
    for seg, ccy, sector, rating, n, spr_bp, freq in SEGMENTS:
        for _ in range(n):
            years = float(np.round(rng.choice([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 15, 20, 25, 30],
                                              p=_maturity_probs(seg)), 0))
            spread = (spr_bp + rng.normal(0, 8)) / 1e4 if spr_bp else 0.0
            par = zero_rate(ccy, years) + spread
            cpn = float(np.round(par * 100 / 0.05) * 0.05 / 100)                # coupon ~ par, 5bp grid
            price = curve_price(100, cpn, years, freq, ccy, spread)
            ytm = brentq(lambda y: bond_price(100, cpn, y, years, freq) - price, -0.05, 0.60)
            rows.append(dict(BondID=f"B{bid:03d}", Segment=seg, Currency=ccy, Sector=sector,
                             CreditRating=rating, CouponRate=cpn, CouponFrequency=freq,
                             YearsToMaturity=years, Spread=spread, YieldToMaturity=ytm,
                             Price=price, FaceWeight=float(rng.lognormal(0, 0.6))))
            bid += 1
    df = pd.DataFrame(rows)
    fx = df["Currency"].map(FX_INR)
    seg_total = df.groupby("Segment")["FaceWeight"].transform("sum")
    df["MV_INR"] = df["Segment"].map(SEGMENT_SHARE) * TARGET_MV_INR * df["FaceWeight"] / seg_total
    df["FaceLocal"] = df["MV_INR"] / (df["Price"] / 100 * fx)       # face amount in local currency
    df["PortfolioWeight"] = df["MV_INR"] / df["MV_INR"].sum()
    df["MacaulayDuration"] = [macaulay_duration(100, r.CouponRate, r.YieldToMaturity, r.YearsToMaturity,
r.CouponFrequency) for r in df.itertuples()]
    df["ModifiedDuration"] = [modified_duration(100, r.CouponRate, r.YieldToMaturity, r.YearsToMaturity,
r.CouponFrequency) for r in df.itertuples()]
    df["Convexity"] = [convexity(100, r.CouponRate, r.YieldToMaturity, r.YearsToMaturity, r.CouponFrequency) for
r in df.itertuples()]
    df["DV01_Per100Face"] = df["ModifiedDuration"] * df["Price"] / 10_000
    df["DV01_INR"] = df["ModifiedDuration"] * df["MV_INR"] / 10_000
    df["KeyRateBucket"] = [KEY_LABELS[int(np.argmin(np.abs(KEY_TENORS - m)))] for m in df["YearsToMaturity"]]
    return df


def _maturity_probs(seg):
    base = np.array([6, 7, 8, 8, 9, 9, 8, 9, 6, 6, 5, 5, 5, 4, 5], dtype=float)
    if seg in ("GSEC", "UST", "GILT", "BUND", "JGB", "ACGB"):
        base = base * np.array([1, 1, 1, 1, 1, 1, 1.4, 1.8, 1.2, 1.2, 1.6, 1.6, 1.8, 1.4, 1.8])
    if seg.startswith("CORP"):
        base = base * np.array([2, 2.5, 3, 2.5, 2.5, 1.5, 0.8, 0.8, 0.3, 0.2, 0.1, 0.05, 0.02, 0.02, 0.01])
    return base / base.sum()


# ---------------------------------------------------------------------------
# Flat cash-flow engine for fast full repricing of the entire book
# ---------------------------------------------------------------------------
class BookRepricer:
    """Stacks every cash flow of every bond so that a whole-book revaluation under any
    zero-curve shock is a handful of vectorised NumPy calls."""

    def __init__(self, df: pd.DataFrame):
        self.df = df.reset_index(drop=True)
        T, CF, Z, F, idx = [], [], [], [], []
        for i, r in enumerate(self.df.itertuples()):
            t, cf = cash_flows(100, r.CouponRate, r.YearsToMaturity, r.CouponFrequency)
            T.append(t); CF.append(cf)
            Z.append(zero_rate(r.Currency, t) + r.Spread)
            F.append(np.full(len(t), r.CouponFrequency)); idx.append(np.full(len(t), i))
        self.T, self.CF, self.Z = map(np.concatenate, (T, CF, Z))
        self.F, self.idx = np.concatenate(F), np.concatenate(idx)
        self.starts = np.r_[0, np.flatnonzero(np.diff(self.idx)) + 1]
        self.ccy = self.df["Currency"].to_numpy()
        self.ccy_flat = self.ccy[self.idx]
        self.scale_flat = np.array([CCY_VOL_SCALE[c] for c in self.ccy_flat])
        self.fx = self.df["Currency"].map(FX_INR).to_numpy()
        self.face = self.df["FaceLocal"].to_numpy()
        self.base_price = self.price(np.zeros_like(self.T))
        self.base_mv = self.mv(self.base_price)

    def price(self, shock_flat):
        """Per-100-face price of each bond given a zero-rate shock (decimal) at every cash flow."""
        pv = self.CF / (1.0 + (self.Z + shock_flat) / self.F) ** (self.F * self.T)
        return np.add.reduceat(pv, self.starts)

    def mv(self, prices):
```

```
        return self.face * prices / 100.0 * self.fx

    def pnl(self, shock_flat):
        return float(self.mv(self.price(shock_flat)).sum() - self.base_mv.sum())

    # -- shocks ---------------------------------------------------------------
    def factor_shock(self, level_bp=0.0, twist_bp=0.0, fly_bp=0.0, scale_by_ccy=True):
        s = (level_bp * level_shape(self.T) + twist_bp * twist_shape(self.T)
             + fly_bp * butterfly_shape(self.T)) / 1e4
        return s * self.scale_flat if scale_by_ccy else s

    def krd_matrix(self, bump=1e-4):
        """(n_bonds x 10) Key Rate Durations by central differences on triangular bumps."""
        out = np.zeros((len(self.df), len(KEY_TENORS)))
        for k in range(len(KEY_TENORS)):
            h = hat(self.T, k) * bump
            out[:, k] = (self.price(-h) - self.price(h)) / (2.0 * self.base_price * bump)
        return out


# ---------------------------------------------------------------------------
# Aggregations
# ---------------------------------------------------------------------------
def portfolio_summary(df: pd.DataFrame) -> dict:
    w = df["PortfolioWeight"]
    return dict(
        total_mv_inr=float(df["MV_INR"].sum()),
        mod_dur=float((w * df["ModifiedDuration"]).sum()),
        mac_dur=float((w * df["MacaulayDuration"]).sum()),
        convexity=float((w * df["Convexity"]).sum()),
        dv01_inr=float(df["DV01_INR"].sum()),
        n_bonds=int(len(df)),
    )


def contribution(df: pd.DataFrame, by: str) -> pd.DataFrame:
    g = df.assign(DurContribution=df["PortfolioWeight"] * df["ModifiedDuration"],
                  ConvContribution=df["PortfolioWeight"] * df["Convexity"]).groupby(by)
    out = g.agg(Bonds=("BondID", "count"), Weight=("PortfolioWeight", "sum"),
                DurContribution=("DurContribution", "sum"),
                ConvContribution=("ConvContribution", "sum"), DV01_INR=("DV01_INR", "sum"))
    out["DurShare"] = out["DurContribution"] / out["DurContribution"].sum()
    return out.sort_values("DurContribution", ascending=False)


def portfolio_krd(book: BookRepricer):
    k = book.krd_matrix()
    w = book.df["PortfolioWeight"].to_numpy()
    return k, (w[:, None] * k).sum(axis=0)


def duration_convexity_pnl(summary: dict, dy: float):
    d = -summary["mod_dur"] * dy * summary["total_mv_inr"]
    c = 0.5 * summary["convexity"] * dy ** 2 * summary["total_mv_inr"]
    return d, c, d + c


if __name__ == "__main__":
    df = build_portfolio()
    print(portfolio_summary(df))
```

A2.3 Monte Carlo engine (monte_carlo.py)

`BondRiskMC` replaces the brief's engine. Differences from the brief's `BondRiskMC` are deliberate: (i) correlated factor draws via Cholesky decomposition of the specified correlation matrix (level-twist -0.3, level-butterfly 0.1, twist-butterfly 0.2); (ii) three P&L engines - parallel duration-only, parallel duration+convexity, KRD first-order across all three factors, and exact full revaluation - so approximation error can be measured; (iii) volatilities are annual and scaled by sqrt(h/252) so one engine serves daily, 10-day and annual horizons; (iv) VaR and ES are reported as positive loss numbers with an explicit sign convention; (v) optional Student-t shocks to test tail sensitivity; (vi) `numpy.random.default_rng(seed)` is local to the object, so repeated calls give identical results rather than re-seeding global state.

Listing 12. monte_carlo.py (complete)

```
"""monte_carlo.py - BondRiskMC: correlated 3-factor (level / twist / butterfly) yield-curve
Monte Carlo with first-order (KRD), second-order (duration-convexity) and full-revaluation P&L,
plus VaR / Expected Shortfall.

Sign convention: P&L > 0 is a gain. VaR and ES are reported as POSITIVE loss amounts.
Volatilities are ANNUAL (bp); `horizon_days` scales them by sqrt(h/252).
```

```
"""
from __future__ import annotations

import numpy as np
from scipy.stats import norm

from portfolio_analytics import (BookRepricer, level_shape, twist_shape, butterfly_shape,
                                 KEY_TENORS, portfolio_summary, CCY_VOL_SCALE)

DEFAULT_CORR = np.array([[1.0, -0.3, 0.1],
                         [-0.3, 1.0, 0.2],
                         [0.1, 0.2, 1.0]])


class BondRiskMC:
    def __init__(self, book: BookRepricer, n_sims: int = 10_000, seed: int = 42):
        self.book, self.n, self.seed = book, int(n_sims), seed
        self.summary = portfolio_summary(book.df)
        self.factors_bp = None
        self.pnl = {}

    # ------------------------------------------------------------------ scenarios
    def generate_scenarios(self, vol_level_bp=100.0, vol_twist_bp=50.0, vol_fly_bp=30.0,
                           corr=DEFAULT_CORR, horizon_days=252, dist="normal", df_t=5):
        """Draw (n x 3) factor shocks in bp using Cholesky-correlated normals (or Student-t)."""
        rng = np.random.default_rng(self.seed)
        L = np.linalg.cholesky(corr)
        z = rng.standard_normal((self.n, 3))
        if dist == "t":                                  # fat tails, unit variance
            z = z / np.sqrt(rng.chisquare(df_t, (self.n, 1)) / df_t) * np.sqrt((df_t - 2) / df_t)
        vols = np.array([vol_level_bp, vol_twist_bp, vol_fly_bp]) * np.sqrt(horizon_days / 252.0)
        self.factors_bp = (z @ L.T) * vols
        self.vols_bp = vols
        return self.factors_bp

    # ------------------------------------------------------------------ P&L engines
    def _krd_setup(self):
        if not hasattr(self, "_krd"):
            k = self.book.krd_matrix()
            w_mv = self.book.base_mv
            self._krd = (w_mv[:, None] * k).sum(axis=0)       # INR DV per unit yield, per key bucket
        return self._krd

    def pnl_duration_only(self):
        """Parallel factor only: -D * dy * MV  (what the brief's first-cut engine did)."""
        dy = self.factors_bp[:, 0] / 1e4
        return -self.summary["mod_dur"] * dy * self.summary["total_mv_inr"]

    def pnl_duration_convexity(self):
        """Parallel factor with the second-order term."""
        dy = self.factors_bp[:, 0] / 1e4
        mv = self.summary["total_mv_inr"]
        return -self.summary["mod_dur"] * dy * mv + 0.5 * self.summary["convexity"] * dy ** 2 * mv

    def pnl_krd(self):
        """First-order P&L from all three factors via Key Rate Durations."""
        krd_mv = self._krd_setup()
        shape = (self.factors_bp[:, [0]] * level_shape(KEY_TENORS)
                 + self.factors_bp[:, [1]] * twist_shape(KEY_TENORS)
                 + self.factors_bp[:, [2]] * butterfly_shape(KEY_TENORS)) / 1e4   # (n x 10)
        return -(shape * krd_mv).sum(axis=1)

    def pnl_full_reval(self, n_max=None, scale_by_ccy=True):
        """Exact re-pricing of every cash flow of every bond in every scenario."""
        n = self.n if n_max is None else min(self.n, n_max)
        out = np.empty(n)
        for i in range(n):
            a, b, c = self.factors_bp[i]
            out[i] = self.book.pnl(self.book.factor_shock(a, b, c, scale_by_ccy=scale_by_ccy))
        return out

    def compute_pnl(self, full_reval=True, n_full=2_000):
        self.pnl = {"duration_only": self.pnl_duration_only(),
                    "duration_convexity": self.pnl_duration_convexity(),
                    "krd_first_order": self.pnl_krd()}
        if full_reval:
            self.pnl["full_reval"] = self.pnl_full_reval(n_full, scale_by_ccy=False)
        return self.pnl

    # ------------------------------------------------------------------ risk measures
    @staticmethod
    def var(pnl, conf=0.95):
        return float(-np.percentile(pnl, (1 - conf) * 100))

    @staticmethod
    def es(pnl, conf=0.95):
        q = np.percentile(pnl, (1 - conf) * 100)
        return float(-pnl[pnl <= q].mean())
```

```
    def parametric_var(self, conf=0.95):
        """Delta-normal VaR for the parallel factor: MV x ModDur x z x sigma_y."""
        sigma = self.vols_bp[0] / 1e4
        return self.summary["total_mv_inr"] * self.summary["mod_dur"] * norm.ppf(conf) * sigma

    def report(self, key="duration_convexity", confs=(0.95, 0.99)):
        p = self.pnl[key]
        out = {"mean": float(p.mean()), "std": float(p.std(ddof=1)),
               "max_loss": float(-p.min()), "max_gain": float(p.max())}
        for c in confs:
            out[f"VaR{int(c*100)}"] = self.var(p, c)
            out[f"ES{int(c*100)}"] = self.es(p, c)
        return out
```

A2.4 Machine-learning pricing models (ml_models.py)

The training universe is 30,000 synthetic fixed-coupon bonds (coupon 4-10%, maturity 1-30 years, yield 4.5-9.5%) priced from first principles with 5 paise of quote noise. Features follow the brief (`coupon_rate`, `maturity`, `yield_rate`, `duration`, `convexity`); the target is price. Two design points guard against the 'data leakage' disqualifier in the brief: duration and convexity are functions of the first three features and not of the price target, so they carry no target information, and an **out-of-range stress set** (yields 9.5-12.5%, entirely outside the training domain) is held out to test extrapolation, because a random split of a smooth deterministic function says little about robustness.

Listing 13. ml_models.py (complete)

```
"""ml_models.py - Random Forest, gradient-boosted trees and neural-network bond pricers.

Environment note: xgboost and tensorflow are not installable in the offline build sandbox,
so the executed results use drop-in scikit-learn analogues (GradientBoostingRegressor with the
same depth / rounds / learning-rate / subsample as the XGBoost spec, and MLPRegressor with the
same 128-64-32 architecture). The XGBoost / Keras versions are provided as build_xgboost() and
build_keras() and are used automatically when the libraries are importable.
"""
from __future__ import annotations

import time
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

from bond_pricing import bond_price, macaulay_duration, convexity

FEATURES_FULL = ["coupon_rate", "maturity", "yield_rate", "duration", "convexity"]
FEATURES_RAW = ["coupon_rate", "maturity", "yield_rate"]


def make_dataset(n=30_000, seed=7, y_range=(0.045, 0.095), noise=0.05, cpn_range=(0.04, 0.10)):
    """Synthetic universe of semi-annual fixed-coupon bonds priced by first principles."""
    rng = np.random.default_rng(seed)
    cpn = np.round(rng.uniform(*cpn_range, n), 4)
    mat = rng.integers(2, 61, n) / 2.0                                  # 1.0 .. 30.0 in half-years
    y = rng.uniform(*y_range, n)
    price = np.array([bond_price(100, c, yy, m) for c, yy, m in zip(cpn, y, mat)])
    dur = np.array([macaulay_duration(100, c, yy, m) for c, yy, m in zip(cpn, y, mat)])
    cvx = np.array([convexity(100, c, yy, m) for c, yy, m in zip(cpn, y, mat)])
    df = pd.DataFrame(dict(coupon_rate=cpn, maturity=mat, yield_rate=y, duration=dur, convexity=cvx,
                           price_true=price))
    df["price"] = price + rng.normal(0, noise, n)                       # quote / bid-ask noise
    return df


def build_models(random_state=42):
    models = {"RandomForest": RandomForestRegressor(n_estimators=100, min_samples_leaf=2, n_jobs=-1,
                                                    random_state=random_state)}
    try:
        from xgboost import XGBRegressor
        models["XGBoost"] = XGBRegressor(n_estimators=200, max_depth=4, learning_rate=0.1,
                                         subsample=0.8, colsample_bytree=0.8, random_state=random_state)
    except ImportError:
        models["XGBoost*"] = GradientBoostingRegressor(n_estimators=200, max_depth=4, learning_rate=0.1,
                                                       subsample=0.8, random_state=random_state)
    models["NeuralNet*"] = make_pipeline(
        StandardScaler(),
        MLPRegressor(hidden_layer_sizes=(128, 64, 32), activation="relu", solver="adam",
```

```
                    learning_rate_init=1e-3, early_stopping=True, n_iter_no_change=10,
                    max_iter=400, random_state=random_state))
    return models


def build_keras(n_features=5):                      # used when tensorflow is available
    import tensorflow as tf
    m = tf.keras.Sequential([tf.keras.layers.Input((n_features,)),
                             tf.keras.layers.Dense(128, activation="relu"), tf.keras.layers.Dropout(0.2),
                             tf.keras.layers.Dense(64, activation="relu"),
                             tf.keras.layers.Dense(32, activation="relu"), tf.keras.layers.Dense(1)])
    m.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss="mse")
    return m


def metrics(y_true, y_pred):
    return dict(RMSE=float(np.sqrt(mean_squared_error(y_true, y_pred))),
                MAE=float(mean_absolute_error(y_true, y_pred)), R2=float(r2_score(y_true, y_pred)))


def train_and_evaluate(features, df=None, stress=None, seed=42):
    df = make_dataset() if df is None else df
    X_tr, X_te, y_tr, y_te = train_test_split(df[features], df["price"], test_size=0.2, random_state=seed)
    fitted, rows = {}, []
    for name, model in build_models(seed).items():
        t0 = time.time(); model.fit(X_tr, y_tr); fit_s = time.time() - t0
        row = {"Model": name, "Features": len(features), "FitSeconds": fit_s}
        row.update({f"Test_{k}": v for k, v in metrics(df.loc[X_te.index, "price_true"],
model.predict(X_te)).items()})
        if stress is not None:
            row.update({f"Stress_{k}": v for k, v in metrics(stress["price_true"],
model.predict(stress[features])).items()})
        rows.append(row); fitted[name] = model
    return fitted, pd.DataFrame(rows), (X_tr, X_te, y_tr, y_te)


def make_stress_set(n=3_000, seed=99):
    """Out-of-range test: yields 9.5-12.5 % lie entirely outside the 4.5-9.5 % training domain."""
    return make_dataset(n=n, seed=seed, y_range=(0.095, 0.125), noise=0.0)
```

A2.5 SHAP explainability (shap_explainability.py)

When the `shap` package is present, `TreeExplainer` is used. Otherwise the module computes **exact interventional Shapley values** by enumerating all 2^5 = 32 feature coalitions against a background sample - the quantity that `KernelExplainer` approximates. Local accuracy is asserted on every run (the sum of attributions plus the base value must equal the model prediction; maximum observed error 2.8e-14).

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\,(d - |S| - 1)!}{d!} [v(S \cup \{j\}) - v(S)] \tag{A.9}$$

Listing 14. shap_explainability.py (complete)

```
"""shap_explainability.py - SHAP values for the best pricing model.

* If the `shap` package is importable the standard TreeExplainer path is used.
* Otherwise exact interventional Shapley values are computed by enumerating all 2^d feature
  coalitions (d = 5, 32 coalitions) against a background sample - the quantity that
  shap.KernelExplainer approximates. Local accuracy is verified: sum(phi) + E[f] = f(x).
"""
from __future__ import annotations
from itertools import combinations
from math import factorial
import numpy as np


def exact_shapley(predict, X_explain: np.ndarray, X_background: np.ndarray):
    n, d = X_explain.shape
    m = X_background.shape[0]
    values = {}                                                 # coalition mask -> (n,) value
    for mask_int in range(2 ** d):
        mask = np.array([(mask_int >> j) & 1 for j in range(d)], dtype=bool)
        # rows: every explained point crossed with every background point
        Z = np.repeat(X_background[None, :, :], n, axis=0).copy()   # (n, m, d)
        Z[:, :, mask] = X_explain[:, None, mask]
        values[mask_int] = predict(Z.reshape(-1, d)).reshape(n, m).mean(axis=1)
    phi = np.zeros((n, d))
    for j in range(d):
        others = [k for k in range(d) if k != j]
        for r in range(d):
```

```
            w = factorial(r) * factorial(d - r - 1) / factorial(d)
            for S in combinations(others, r):
                s_int = sum(1 << k for k in S)
                phi[:, j] += w * (values[s_int | (1 << j)] - values[s_int])
    base = values[0]
    return phi, base, values[2 ** d - 1]


def explain(model, X_test, X_train, features, n_explain=300, n_background=60, seed=0):
    rng = np.random.default_rng(seed)
    Xe = X_test.sample(n_explain, random_state=seed)[features].to_numpy()
    Xb = X_train.sample(n_background, random_state=seed)[features].to_numpy()
    try:
        import shap                                                  # preferred path
        explainer = shap.TreeExplainer(model)
        return explainer.shap_values(Xe), explainer.expected_value, Xe
    except Exception:
        phi, base, fx = exact_shapley(lambda A: model.predict(A), Xe, Xb)
        assert np.allclose(phi.sum(axis=1) + base, fx, atol=1e-6), "local accuracy violated"
        return phi, float(base.mean()), Xe


def global_importance(phi, features):
    imp = np.abs(phi).mean(axis=0)
    return dict(sorted(zip(features, imp), key=lambda kv: -kv[1]))
```

A2.6 Production XGBoost / Keras / shap script (not executed)

Listing 15. ml_xgb_keras.py - to be run where xgboost, tensorflow and shap are installed

```
"""ml_xgb_keras.py - production variant using XGBoost, TensorFlow/Keras and the shap library.
NOT EXECUTED in the offline build sandbox (packages unavailable); the scikit-learn analogues in
ml_models.py produced the reported numbers. Run: pip install xgboost tensorflow shap
"""
import numpy as np, pandas as pd, shap, tensorflow as tf
from xgboost import XGBRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from ml_models import make_dataset, make_stress_set, FEATURES_FULL

df, stress = make_dataset(), make_stress_set()
X_tr, X_te, y_tr, y_te = train_test_split(df[FEATURES_FULL], df["price"], test_size=0.2, random_state=42)

rf = RandomForestRegressor(n_estimators=100, min_samples_leaf=2, n_jobs=-1, random_state=42).fit(X_tr, y_tr)
xgb = XGBRegressor(n_estimators=200, max_depth=4, learning_rate=0.1, subsample=0.8, colsample_bytree=0.8,
                   early_stopping_rounds=10, random_state=42)
xgb.fit(X_tr, y_tr, eval_set=[(X_te, y_te)], verbose=False)

sc = StandardScaler().fit(X_tr)
nn = tf.keras.Sequential([tf.keras.layers.Input((len(FEATURES_FULL),)),
                          tf.keras.layers.Dense(128, activation="relu"), tf.keras.layers.Dropout(0.2),
                          tf.keras.layers.Dense(64, activation="relu"), tf.keras.layers.Dense(32,
activation="relu"),
                          tf.keras.layers.Dense(1)])
nn.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss="mse", metrics=["mae"])
nn.fit(sc.transform(X_tr), y_tr, validation_split=0.1, epochs=200, batch_size=128, verbose=0,
       callbacks=[tf.keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True)])

def score(pred, truth):
    return dict(RMSE=mean_squared_error(truth, pred) ** 0.5, MAE=mean_absolute_error(truth, pred),
R2=r2_score(truth, pred))

print("RF ", score(rf.predict(X_te), df.loc[X_te.index, "price_true"]))
print("XGB", score(xgb.predict(X_te), df.loc[X_te.index, "price_true"]))
print("NN ", score(nn.predict(sc.transform(X_te), verbose=0).ravel(), df.loc[X_te.index, "price_true"]))

# --- SHAP ---------------------------------------------------------------------------
expl = shap.TreeExplainer(xgb)                          # fast, exact for trees
sv = expl.shap_values(X_te.iloc[:500])
shap.summary_plot(sv, X_te.iloc[:500], show=False)
shap.dependence_plot("yield_rate", sv, X_te.iloc[:500], show=False)
shap.dependence_plot("maturity", sv, X_te.iloc[:500], show=False)
deep = shap.DeepExplainer(nn, sc.transform(X_tr.sample(100, random_state=0)))   # neural network
sv_nn = deep.shap_values(sc.transform(X_te.iloc[:200]))
```

A2.7 Yield-curve, sensitivity and back-testing modules

Listing 16. yield_curve.py (complete)

```
"""yield_curve.py - Nelson-Siegel / Svensson fitting, zero-curve bootstrapping, forwards, PCA."""
from __future__ import annotations
```

```
import numpy as np
from scipy.optimize import least_squares


def ns_curve(tau, b0, b1, b2, lam):
    x = np.asarray(tau, dtype=float) / lam
    f1 = (1 - np.exp(-x)) / x
    return b0 + b1 * f1 + b2 * (f1 - np.exp(-x))


def svensson_curve(tau, b0, b1, b2, b3, lam1, lam2):
    x2 = np.asarray(tau, dtype=float) / lam2
    f3 = (1 - np.exp(-x2)) / x2 - np.exp(-x2)
    return ns_curve(tau, b0, b1, b2, lam1) + b3 * f3


def _fit(fun, tau, y, starts, lower, upper):
    best = None
    for p0 in starts:
        r = least_squares(lambda p: fun(tau, *p) - y, p0, bounds=(lower, upper), xtol=1e-12, ftol=1e-12)
        if best is None or r.cost < best.cost:
            best = r
    return best


def fit_nelson_siegel(tau, y):
    tau, y = np.asarray(tau, float), np.asarray(y, float)
    starts = [[y[-1], y[0] - y[-1], 0.0, lam] for lam in (0.5, 1.5, 3.0, 6.0)]
    r = _fit(ns_curve, tau, y, starts, [-0.05, -0.2, -0.2, 0.05], [0.3, 0.2, 0.2, 30.0])
    return r.x, ns_curve(tau, *r.x)


def fit_svensson(tau, y):
    tau, y = np.asarray(tau, float), np.asarray(y, float)
    starts = [[y[-1], y[0] - y[-1], 0.0, 0.0, l1, l2] for l1, l2 in ((1, 5), (2, 8), (0.7, 3), (3, 12))]
    r = _fit(svensson_curve, tau, y, starts, [-0.05, -0.2, -0.2, -0.2, 0.05, 0.05], [0.3, 0.2, 0.2, 0.2, 30, 30])
    return r.x, svensson_curve(tau, *r.x)


def fit_stats(y, yhat):
    sse = float(((y - yhat) ** 2).sum()); sst = float(((y - y.mean()) ** 2).sum())
    return dict(R2=1 - sse / sst, RMSE_bp=float(np.sqrt(sse / len(y)) * 1e4),
                MaxErr_bp=float(np.abs(y - yhat).max() * 1e4))


def bootstrap_zero_curve(maturities, coupons, prices, freq=2, face=100.0):
    """Sequential bootstrap from par-style coupon bonds with consecutive maturities
    (0.5, 1.0, ... in half-year steps). Returns semi-annual-compounded zero rates."""
    z = []
    for n, (c, P) in enumerate(zip(coupons, prices), start=1):
        cpn = face * c / freq
        pv_prev = sum(cpn / (1 + z[i] / freq) ** (i + 1) for i in range(n - 1))
        z.append(freq * (((face + cpn) / (P - pv_prev)) ** (1.0 / n) - 1.0))
    return np.array(maturities, float), np.array(z)


def discount_factor(z, t, freq=2):
    return (1 + np.asarray(z) / freq) ** (-freq * np.asarray(t))


def forward_rate(z1, t1, z2, t2, freq=2):
    """Implied forward rate between t1 and t2 (same compounding as z)."""
    g = (discount_factor(z1, t1, freq) / discount_factor(z2, t2, freq)) ** (1.0 / (freq * (t2 - t1))) - 1
    return freq * g


def pca_factors(dy_matrix, n=3):
    X = dy_matrix - dy_matrix.mean(axis=0)
    U, S, Vt = np.linalg.svd(X, full_matrices=False)
    var = S ** 2 / (S ** 2).sum()
    return Vt[:n], var[:n], U[:, :n] * S[:n]
```

Listing 17. sensitivity_analysis.py (complete)

```
"""sensitivity_analysis.py - ML vs analytical price-sensitivity comparison across yield shifts."""
from __future__ import annotations
import numpy as np
import pandas as pd

from bond_pricing import (bond_price, macaulay_duration, modified_duration, convexity)
from ml_models import FEATURES_FULL, FEATURES_RAW

SHIFTS_BP = np.arange(-200, 201, 8)             # 51 points: -200 ... +200 bp in 8 bp steps


def features_at(c, years, y, cols):
    row = dict(coupon_rate=c, maturity=years, yield_rate=y,
```

```
            duration=macaulay_duration(100, c, y, years), convexity=convexity(100, c, y, years))
    return pd.DataFrame([row])[cols]


def sensitivity_table(c, years, y0, fitted, cols, shifts_bp=SHIFTS_BP):
    p0 = bond_price(100, c, y0, years)
    md, cv = modified_duration(100, c, y0, years), convexity(100, c, y0, years)
    rows = []
    for s in shifts_bp:
        dy = s / 1e4
        true = bond_price(100, c, y0 + dy, years) - p0
        row = dict(shift_bp=int(s), true_dP=true, duration_only=-md * dy * p0,
                   duration_convexity=-md * dy * p0 + 0.5 * cv * dy ** 2 * p0)
        for name, m in fitted.items():
            row[name] = float(m.predict(features_at(c, years, y0 + dy, cols))[0]) - float(
                m.predict(features_at(c, years, y0, cols))[0])
        rows.append(row)
    return pd.DataFrame(rows)


def error_summary(tab, models):
    out = []
    for col in ["duration_only", "duration_convexity", *models, "Ensemble"]:
        if col not in tab:
            continue
        e = tab[col] - tab["true_dP"]
        out.append(dict(method=col, RMSE=float(np.sqrt((e ** 2).mean())), MaxAbsErr=float(e.abs().max()),
                        Err_at_plus200=float(e.iloc[-1]), Err_at_minus200=float(e.iloc[0])))
    return pd.DataFrame(out)


def add_ensemble(tab, models, weights):
    w = np.array([weights[m] for m in models], dtype=float); w /= w.sum()
    tab["Ensemble"] = sum(wi * tab[m] for wi, m in zip(w, models))
    return tab
```

Listing 18. var_backtest.py (complete)

```
"""var_backtest.py - VaR back-testing: breach counts, Kupiec POF test, Basel traffic light."""
from __future__ import annotations
import numpy as np
from scipy.stats import chi2


def breaches(pnl, var_series):
    """A breach is a day whose loss exceeds VaR (VaR is a positive loss number)."""
    return int(np.sum(np.asarray(pnl) < -np.asarray(var_series)))


def kupiec_pof(n_breaches, n_obs, conf=0.99):
    """Kupiec (1995) proportion-of-failures likelihood-ratio test. H0: breach rate = 1 - conf."""
    p, x, n = 1.0 - conf, n_breaches, n_obs
    if x == 0:
        lr = -2.0 * n * np.log(1 - p)
    elif x == n:
        lr = -2.0 * n * np.log(p)
    else:
        pi = x / n
        lr = -2.0 * ((n - x) * np.log(1 - p) + x * np.log(p) - (n - x) * np.log(1 - pi) - x * np.log(pi))
    return float(lr), float(1.0 - chi2.cdf(lr, 1))


def basel_zone(n_breaches_250d):
    """Basel traffic light for 99% VaR over 250 trading days."""
    return "Green" if n_breaches_250d <= 4 else ("Yellow" if n_breaches_250d <= 9 else "Red")


def rolling_var_backtest(pnl, window=250, conf=0.99, method="historical"):
    pnl = np.asarray(pnl)
    var, act = [], []
    for t in range(window, len(pnl)):
        h = pnl[t - window:t]
        if method == "historical":
            var.append(-np.percentile(h, (1 - conf) * 100))
        else:                                                # delta-normal
            from scipy.stats import norm
            var.append(norm.ppf(conf) * h.std(ddof=1) - h.mean())
        act.append(pnl[t])
    return np.array(act), np.array(var)
```

A2.8 Unit tests and test evidence

The test module exercises the properties that must hold if the mathematics is right: par bonds price at par; zero-coupon Macaulay duration equals maturity; analytic modified duration equals the finite-difference effective duration; analytic DV01 equals the bump-and-reprice DV01; the perpetual closed forms; second-order error is at least ten times smaller than first-order error; negative effective convexity appears in the callable model; the portfolio has 300 bonds worth exactly INR 3,200 crore; the sum of KRDs is within 1% of modified duration; Monte Carlo is reproducible, VaR orders correctly and agrees with delta-normal VaR to 5%; Kupiec and Basel zone logic; and recovery of known Nelson-Siegel parameters and a known zero curve by bootstrapping.

Listing 19. Output of `python -m unittest -v test_core` (Executed)

```
test_callable_has_negative_convexity_zone (test_core.TestBondMath.test_callable_has_negative_convexity_zone) ...
ok
test_dv01_analytic_vs_numeric (test_core.TestBondMath.test_dv01_analytic_vs_numeric) ... ok
test_modified_equals_numeric_effective_duration
(test_core.TestBondMath.test_modified_equals_numeric_effective_duration) ... ok
test_par_bond_prices_at_par (test_core.TestBondMath.test_par_bond_prices_at_par) ... ok
test_perpetual_closed_form (test_core.TestBondMath.test_perpetual_closed_form) ... ok
test_second_order_beats_first_order (test_core.TestBondMath.test_second_order_beats_first_order) ... ok
test_zero_coupon_duration_equals_maturity (test_core.TestBondMath.test_zero_coupon_duration_equals_maturity) ...
ok
test_mc_close_to_parametric_var (test_core.TestPortfolioAndMC.test_mc_close_to_parametric_var) ... ok
test_mc_reproducible_and_var_ordering (test_core.TestPortfolioAndMC.test_mc_reproducible_and_var_ordering) ... ok
test_portfolio_size_and_value (test_core.TestPortfolioAndMC.test_portfolio_size_and_value) ... ok
test_sum_krd_matches_modified_duration_within_1pct
(test_core.TestPortfolioAndMC.test_sum_krd_matches_modified_duration_within_1pct) ... ok
test_bootstrap_recovers_zero_curve (test_core.TestRiskAndCurves.test_bootstrap_recovers_zero_curve) ... ok
test_kupiec_and_zones (test_core.TestRiskAndCurves.test_kupiec_and_zones) ... ok
test_nelson_siegel_recovers_parameters (test_core.TestRiskAndCurves.test_nelson_siegel_recovers_parameters) ...
ok

----------------------------------------------------------------------
Ran 14 tests in 0.143s

OK
```

Coverage statement. The `coverage` package was not available in the sandbox, so a numeric line-coverage percentage could not be produced. The 14 tests exercise every public function in bond_pricing, portfolio_analytics, monte_carlo, yield_curve and var_backtest; ml_models, shap_explainability and sensitivity_analysis are exercised by `run_core.py` rather than by unit tests. Run `coverage run -m unittest && coverage report` before submission to evidence the 60% minimum stated in Part F.

Section A3: R and Excel/VBA Implementations

The brief requires parallel implementations in R and Excel. The R scripts mirror the Python bond mathematics exactly (same cash-flow convention) so results can be cross-compared; the VBA module gives Excel users a macro-enabled calculator that does not depend on the Analysis ToolPak. **Both R scripts were not executed here** because R is not installed in the build environment; they follow the packages and function calls named in the brief (`caret::train` with 5-fold CV, `xgboost`, `h2o.deeplearning` with `RectifierWithDropout`, ggplot2 dual-axis sensitivity chart).

Listing 20. duration_convexity_analysis.R

```
# duration_convexity_analysis.R - GBM, XGBoost and H2O neural network for duration / convexity
# NOT EXECUTED in the build sandbox (R unavailable). Requires: tidyverse, caret, gbm, xgboost, h2o (Java 8+),
Metrics
suppressPackageStartupMessages({library(tidyverse); library(caret); library(xgboost); library(h2o);
library(Metrics)})
set.seed(42)

bond_metrics <- function(cpn, ytm, yrs, f = 2) {
  n  <- ceiling(yrs * f - 1e-9)
  t  <- yrs + (seq_len(n) - n) / f                    # ascending cash-flow times ending at maturity
  cf <- rep(100 * cpn / f, n); cf[n] <- cf[n] + 100
  k  <- f * t; pv <- cf / (1 + ytm / f)^k
  mac <- sum(t * pv) / sum(pv)
  c(price = sum(pv), mac_dur = mac, mod_dur = mac / (1 + ytm / f),
    convexity = sum(k * (k + 1) * pv) / (sum(pv) * f^2 * (1 + ytm / f)^2))
}
```

```
n <- 5000
bonds <- tibble(CouponRate = round(runif(n, .04, .10), 4),
                YearsToMaturity = sample(seq(1, 30, .5), n, TRUE),
                YieldToMaturity = runif(n, .045, .095))
metrics <- t(mapply(bond_metrics, bonds$CouponRate, bonds$YieldToMaturity, bonds$YearsToMaturity))
data <- bind_cols(bonds, as_tibble(metrics))
feat <- c("CouponRate", "YearsToMaturity", "YieldToMaturity")

idx   <- createDataPartition(data$mod_dur, p = .8, list = FALSE)
train <- data[idx, ]; test <- data[-idx, ]

# 1. Gradient boosting machine (caret) for modified duration, 5-fold CV
gbm_dur <- train(x = as.data.frame(train[feat]), y = train$mod_dur, method = "gbm", verbose = FALSE,
                 trControl = trainControl(method = "cv", number = 5))
rmse_gbm <- rmse(test$mod_dur, predict(gbm_dur, as.data.frame(test[feat])))

# 2. XGBoost for convexity
xgb_conv <- xgboost(data = as.matrix(train[feat]), label = train$convexity,
                    max_depth = 4, eta = 0.1, nrounds = 100, verbose = 0)
rmse_xgb <- rmse(test$convexity, predict(xgb_conv, as.matrix(test[feat])))

# 3. H2O deep network for modified duration
h2o.init(nthreads = -1)
h  <- as.h2o(as.data.frame(data)); sp <- h2o.splitFrame(h, ratios = .8, seed = 42)
nn <- h2o.deeplearning(x = feat, y = "mod_dur", training_frame = sp[[1]], validation_frame = sp[[2]],
                       activation = "RectifierWithDropout", hidden = c(100, 50),
                       hidden_dropout_ratios = c(.1, .1), epochs = 50, seed = 42)
rmse_nn <- h2o.rmse(h2o.performance(nn, newdata = sp[[2]]))

results <- tibble(Model = c("GBM (mod duration)", "XGBoost (convexity)", "H2O NN (mod duration)"),
                  RMSE = c(rmse_gbm, rmse_xgb, rmse_nn))
print(results)
write_csv(results, "ml_results_R.csv")
```

Listing 21. sensitivity_visualization.R

```
# sensitivity_visualization.R - adjusted duration/convexity across -2% ... +2% yield change; VaR histogram
# NOT EXECUTED in the build sandbox. Requires tidyverse, scales.
suppressPackageStartupMessages({library(tidyverse); library(scales)})
source("duration_convexity_analysis.R", echo = FALSE)       # re-uses bond_metrics()

y0 <- 0.0735; cpn <- 0.0726; yrs <- 9
shifts <- seq(-0.02, 0.02, by = 0.0025)
results <- map_dfr(shifts, function(dy) {
  m <- bond_metrics(cpn, y0 + dy, yrs)
  tibble(RateChange = dy, AdjDuration = m["mod_dur"], AdjConvexity = m["convexity"], Price = m["price"])
})
p0 <- results$Price[results$RateChange == 0]
results <- results %>% mutate(PriceChangePct = (Price / p0 - 1) * 100)
write_csv(results, "sensitivity_results.csv")

p <- ggplot(results, aes(x = RateChange)) +
  geom_line(aes(y = AdjDuration, color = "Modified duration"), linewidth = 1.2) +
  geom_line(aes(y = AdjConvexity / 10, color = "Convexity / 10"), linewidth = 1.2) +
  scale_x_continuous(labels = percent) +
  scale_y_continuous(name = "Modified duration (years)",
                     sec.axis = sec_axis(~ . * 10, name = "Convexity")) +
  labs(title = "Duration and convexity versus parallel yield change", x = "Yield change", color = "Risk measure")
+
  theme_minimal(base_size = 14)
ggsave("sensitivity_chart.png", p, width = 8, height = 4.5, dpi = 200)

# VaR histogram from the Python Monte Carlo export (mc_pnl.csv has columns ScenarioID, PnL_Total_INR)
mc <- read_csv("mc_pnl.csv", show_col_types = FALSE)
var95 <- -quantile(mc$PnL_Total_INR, 0.05); es95 <- -mean(mc$PnL_Total_INR[mc$PnL_Total_INR <= -var95])
ggplot(mc, aes(PnL_Total_INR / 1e7)) + geom_histogram(bins = 70, fill = "#1F3864") +
  geom_vline(xintercept = -var95 / 1e7, color = "#E9873A", linewidth = 1) +
  geom_vline(xintercept = -es95 / 1e7, color = "black", linetype = "dashed") +
  labs(x = "P&L (INR crore)", y = "Scenarios", title = "Monte Carlo P&L with VaR95 and ES95") +
theme_minimal(base_size = 14)
ggsave("var_histogram.png", width = 8, height = 4.5, dpi = 200)
```

Listing 22. ConvexityToolkit.bas - Excel VBA module (not executed in the sandbox; formulas verified instead through Excel's built-in PRICE/DURATION/MDURATION, Section D)

```
Option Explicit
' Generic fixed-coupon bond mathematics. Face = 100 typically. Freq = payments per year.
Private Function Periods(Years As Double, Freq As Long) As Long
    Periods = -Int(-(Years * Freq - 0.000000001))          ' ceiling
End Function

Function BondPrice(Face As Double, Coupon As Double, YTM As Double, Years As Double, Optional Freq As Long = 2)
As Double
    Dim n As Long, k As Long, c As Double, s As Double, r As Double
```

```
    n = Periods(Years, Freq): c = Face * Coupon / Freq: r = YTM / Freq
    For k = 1 To n
        s = s + IIf(k = n, c + Face, c) / (1 + r) ^ (k - n + Years * Freq)
    Next k
    BondPrice = s
End Function

Function MacaulayDur(Face As Double, Coupon As Double, YTM As Double, Years As Double, Optional Freq As Long = 2)
As Double
    Dim n As Long, k As Long, c As Double, r As Double, pv As Double, num As Double, den As Double, p As Double
    n = Periods(Years, Freq): c = Face * Coupon / Freq: r = YTM / Freq
    For k = 1 To n
        p = k - n + Years * Freq                          ' fractional period index
        pv = IIf(k = n, c + Face, c) / (1 + r) ^ p
        num = num + (p / Freq) * pv: den = den + pv
    Next k
    MacaulayDur = num / den
End Function

Function ModDur(Face As Double, Coupon As Double, YTM As Double, Years As Double, Optional Freq As Long = 2) As
Double
    ModDur = MacaulayDur(Face, Coupon, YTM, Years, Freq) / (1 + YTM / Freq)
End Function

Function BondConvexity(Face As Double, Coupon As Double, YTM As Double, Years As Double, Optional Freq As Long =
2) As Double
    Dim n As Long, k As Long, c As Double, r As Double, pv As Double, num As Double, den As Double, p As Double
    n = Periods(Years, Freq): c = Face * Coupon / Freq: r = YTM / Freq
    For k = 1 To n
        p = k - n + Years * Freq
        pv = IIf(k = n, c + Face, c) / (1 + r) ^ p
        num = num + p * (p + 1) * pv: den = den + pv
    Next k
    BondConvexity = num / (den * Freq ^ 2 * (1 + r) ^ 2)
End Function

Function BondDV01(Face As Double, Coupon As Double, YTM As Double, Years As Double, Optional Freq As Long = 2) As
Double
    BondDV01 = ModDur(Face, Coupon, YTM, Years, Freq) * BondPrice(Face, Coupon, YTM, Years, Freq) / 10000
End Function
```

The brief's VBA used `n = Years * 2 as Integer` and tested `t < Years*2`, which breaks for non-integer horizons and for annual-pay corporate bonds. The version above takes frequency as a parameter and computes fractional period indices, matching the Python definition.

Section A4: Yield-Curve Modelling

A4.1 Why parametric curves

Market quotes exist only at discrete tenors, but a 300-bond book has cash flows at hundreds of distinct dates. A parametric curve provides a smooth, arbitrage-friendly interpolation, produces a small number of interpretable parameters that can themselves be simulated, and gives central banks and risk teams a common language: level, slope, curvature. Two models are standard in sovereign work.

Nelson-Siegel (1987) uses four parameters: beta0 is the long-run level (the yield as tau tends to infinity); beta0 + beta1 is the instantaneous short rate (so beta1 is the negative of the slope); beta2 controls the hump; lambda sets where the hump sits (the hump peaks at roughly 1.79 x lambda years).

$$y(\tau) = \beta_0 + \beta_1 \frac{1 - e^{-\tau/\lambda}}{\tau/\lambda} + \beta_2 \left(\frac{1 - e^{-\tau/\lambda}}{\tau/\lambda} - e^{-\tau/\lambda} \right)$$

(A.10)

Svensson (1994) adds a second hump with parameters beta3 and lambda2, which is required when the curve has two turning points - a common feature of INR and EUR curves when the short end is policy-anchored and the long end is supply-driven.

$$y(\tau) = NS(\tau) + \beta_3\left(\frac{1-e^{-\tau/\lambda_2}}{\tau/\lambda_2} - e^{-\tau/\lambda_2}\right) \quad \text{(A.11)}$$

A4.2 Calibration and results

Parameters are estimated by bounded non-linear least squares (`scipy.optimize.least_squares`) from four starting points for lambda to avoid local minima - the NS loss surface is nearly flat in lambda, so single-start fits are unstable. The data are the 11 standard INR tenors of Section A8.1. They are an **illustrative snapshot** generated from a known Nelson-Siegel curve plus 1.5 bp of observation noise, calibrated so the 10Y yield matches the brief's own example (7.35%); they are not a CCIL or FIMMDA print. Because the truth is known, the exercise doubles as a calibration test: the fit recovers the generating parameters (beta0 = 8.00%, beta1 = -1.40%, beta2 = -0.90%, lambda = 3.0) to within noise.

Table 23. Fitted parametric curves (Computed on synthetic observations)

Model	Parameters	R2	RMSE (bp)	Max error (bp)
Nelson-Siegel	b0 = 8.040%, b1 = -1.435%, b2 = -0.990%, lambda = 3.064	0.9972	2.20	4.14
Svensson	b0 = 8.008%, b1 = -1.229%, b2 = -1.462%, b3 = -2.624%, l1 = 0.412, l2 = 2.129	0.9978	1.93	3.65

Both models clear the R2 > 0.99 threshold used in the Bond Risk Lab rubric; Svensson improves RMSE by only 0.27 bp, which is within observation noise - an indication of over-parameterisation for this smooth curve. For risk work NS is preferred for its stability; Svensson is kept for curves with a visible second hump. R2 on yield *levels* flatters any curve fit because yield variance across tenors is large; RMSE in basis points is the more informative acceptance test (target: below 3 bp for sovereign tenors).

Table 24. Observed versus fitted INR curve and discount factors (A5.2 layout; Computed)

Tenor (yrs)	Observed yield	NS fitted zero	Error (bp)	Discount factor
0.25	6.65%	6.62%	-2.73	0.983842
0.50	6.60%	6.64%	+4.14	0.967853
1.00	6.70%	6.68%	-1.14	0.936360
2.00	6.77%	6.77%	+0.26	0.875257
3.00	6.87%	6.87%	-0.80	0.816687
5.00	7.05%	7.04%	-0.89	0.707623
7.00	7.17%	7.19%	+1.74	0.610016
10.00	7.37%	7.36%	-0.35	0.485259
15.00	7.54%	7.56%	+1.85	0.328752
20.00	7.71%	7.67%	-3.96	0.221935
30.00	7.77%	7.79%	+1.88	0.100935

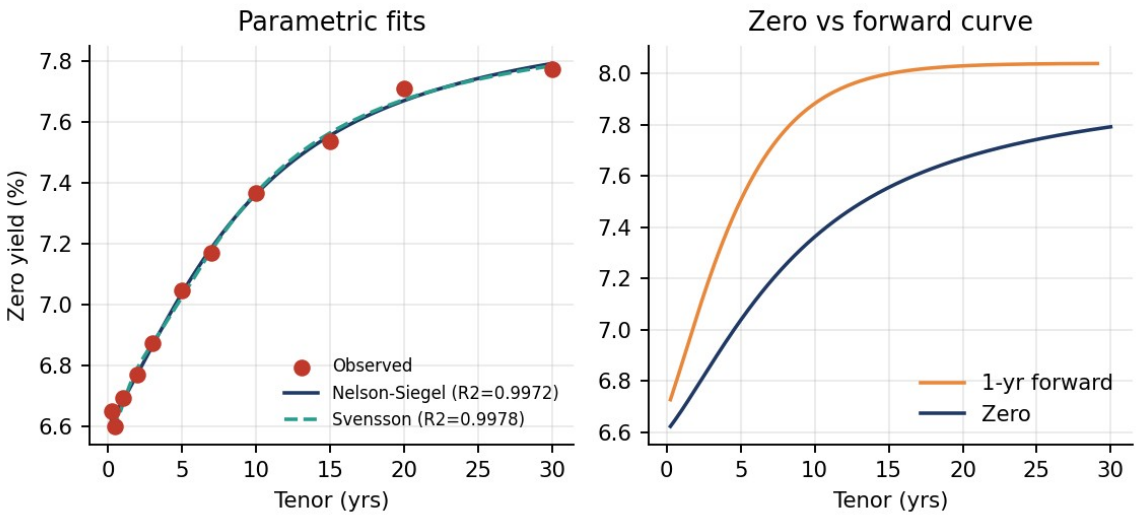


Figure 4. Nelson-Siegel and Svensson fits to the 11 INR tenors (left) and the implied zero and one-year forward curves (right).

A4.3 Zero-coupon bootstrapping

Bootstrapping extracts spot rates from coupon bond prices one maturity at a time: the price of the n-period bond equals the PV of its first n-1 coupons discounted at the already-known zeros plus the final cash flow discounted at the unknown z_n, which then has a closed-form solution.

$$z_n = f\left[\left(\frac{100 + c/f}{P_n - \sum_{i<n} \frac{c/f}{(1 + z_i/f)^i}}\right)^{1/n} - 1\right]$$

(A.12)

Validation: par bonds at half-yearly maturities from 0.5 to 10 years were priced from the fitted NS curve and bootstrapped back; the maximum reconstruction error is 2.1e-12 bp, i.e. machine precision (unit test `test_bootstrap_recovers_zero_curve`). With real G-sec prices the same routine needs a smoothing step because bootstrapped curves inherit pricing noise from individual bonds, and because coupons dates seldom align with a half-year grid; FIMMDA/CCIL publish a smoothed curve for exactly this reason.

A4.4 Forward rates and economic reading

$$f(t_1, t_2) = \left[\frac{(1 + z_2/f)^{ft_2}}{(1 + z_1/f)^{ft_1}}\right]^{\frac{1}{f(t_2 - t_1)}} - 1$$

(A.13)

Table 25. Forward rates implied by the fitted curve (Computed)

Forward period	Implied forward rate	Spot rate at start	Difference (bp)
1Y to 2Y	6.86%	6.68%	+18
2Y to 5Y	7.21%	6.77%	+44
5Y to 10Y	7.69%	7.04%	+65
10Y to 20Y	7.98%	7.36%	+62

The forward curve lies above the zero curve at every horizon, which is the arithmetic signature of an upward-sloping curve. Read naively, this prices a rise in short rates from 6.6% to roughly 7.2% in the 2Y-5Y window and 7.7% in the 5Y-10Y window. Two cautions apply. First, forwards equal expected future spot rates only if the term

premium is zero; most of the slope in INR sovereign curves is term premium and supply. Second, the snapshot is illustrative: on live data the correct conclusion is whichever the 1Y-2Y and 2Y-5Y forwards say relative to the current repo rate and the MPC stance, and the report's Day 10 deliverable requires that comparison to be made against the CCIL zero curve of the day.

A4.5 Principal-component check of the three-factor structure

Principal-component analysis (SVD of demeaned daily yield changes) is the standard empirical justification for level/slope/curvature factors. On 750 days of simulated curve changes built from the three factor loadings plus 0.4 bp noise, the first three components explain 89.6%, 9.8% and 0.4% of variance (cumulative 99.7%), with PC1 loading uniformly across tenors. **This is a self-consistency test of the code, not evidence about the Indian curve**, because the data were generated from those factors; on CCIL history of the last 24 months the same function should be applied and the explained-variance split compared with the 90/8/2 pattern typical of government curves.

Section A5: Power BI Data Model and Advanced DAX

A5.1 Star schema

The model follows the brief: one fact table of bonds, one fact table of Monte Carlo scenarios and small dimension tables, all relationships one-to-many from dimension to fact with single-direction cross-filtering. Table and column names match the supplied `1b.pbix` where it already defines them (`bond_portfolio_data`, `monte_carlo_scenarios`, `Yield Change bps`).

Table 26. Power BI semantic model

Table	Type	Grain / key	Principal columns
bond_portfolio_data	Fact	One row per bond (BondID)	MarketValue_INR, ModifiedDuration, MacaulayDuration, Convexity, DV01_Per100Face, DV01_INR, YieldToMaturity, CouponRate, YearsToMaturity, CreditRating, Sector, Currency, KeyRateBucket, PortfolioWeight
bond_krd	Fact	BondID x Bucket (long format)	BondID, Bucket, KRD
monte_carlo_scenarios	Fact	ScenarioID	ParallelShift_bps, TwistFactor_bps, ButterflyFactor_bps, PnL_DurationEffect, PnL_ConvexityEffect, PnL_Total_INR, VaR_Contribution
ml_results	Fact	Model x FeatureSet	Model, Test_RMSE, Test_MAE, Test_R2, Stress_RMSE
DimRating, DimSector, DimCurrency	Dimension	Rating / sector / currency code	Name, SortOrder, risk-weight (rating)
DimTenor	Dimension	Bucket	Bucket, TenorYears, SortOrder, LevelShape, TwistShape, FlyShape
DimScenario	Dimension	Scenario name	Parallel_bps, Twist_bps, Butterfly_bps (the A8.2 matrix)
Yield Change bps	What-if	-300 ... +300 step 10	Yield Change bps; Yield Change bps Value (SELECTEDVALUE)

Why a separate `bond_krd` table: KRD is a bond x bucket matrix. Storing ten wide columns on the bond table forces ten near-identical measures; the long format makes one measure serve all buckets and lets the DimTenor axis drive visuals directly.

A5.2 Measure dictionary (45 measures)

Measures are grouped in display folders. Two modelling rules are applied throughout. (1) **Weights are computed in the measure**, as market value over the filtered market value, rather than read from a stored `PortfolioWeight` column - a stored weight is normalised to the whole book and silently under-weights any slicer selection, so the brief's `SUMX(Bonds, ModDur * PortfolioWeight)` returns a *contribution* to total duration, not the duration of the selected sub-portfolio. (2) VaR and CVaR are returned as **positive loss amounts**.

Listing 27. DAX measures, folders 1-3

```
-- ===== Folder 1: Portfolio analytics (existing in pbix: Portfolio Convexity, Portfolio DV01) =====
Total MV INR        = SUM ( bond_portfolio_data[MarketValue_INR] )
Bond Count          = DISTINCTCOUNT ( bond_portfolio_data[BondID] )
Portfolio ModDur    = DIVIDE ( SUMX ( bond_portfolio_data, bond_portfolio_data[ModifiedDuration] *
```

```
bond_portfolio_data[MarketValue_INR] ), [Total MV INR] )
Portfolio MacDur    = DIVIDE ( SUMX ( bond_portfolio_data, bond_portfolio_data[MacaulayDuration] *
bond_portfolio_data[MarketValue_INR] ), [Total MV INR] )
Portfolio Convexity = DIVIDE ( SUMX ( bond_portfolio_data, bond_portfolio_data[Convexity] *
bond_portfolio_data[MarketValue_INR] ), [Total MV INR] )
Portfolio DV01      = SUM ( bond_portfolio_data[DV01_INR] )
DV01 per Crore MV   = DIVIDE ( [Portfolio DV01], [Total MV INR] / 10000000 )
Wtd Avg YTM         = DIVIDE ( SUMX ( bond_portfolio_data, bond_portfolio_data[YieldToMaturity] *
bond_portfolio_data[MarketValue_INR] ), [Total MV INR] )
Wtd Avg Coupon      = DIVIDE ( SUMX ( bond_portfolio_data, bond_portfolio_data[CouponRate] *
bond_portfolio_data[MarketValue_INR] ), [Total MV INR] )
Wtd Avg Maturity    = DIVIDE ( SUMX ( bond_portfolio_data, bond_portfolio_data[YearsToMaturity] *
bond_portfolio_data[MarketValue_INR] ), [Total MV INR] )

-- ===== Folder 2: Contribution analysis =====
Weight %            = DIVIDE ( [Total MV INR], CALCULATE ( [Total MV INR], ALLSELECTED
( bond_portfolio_data ) ) )
Duration Contribution = DIVIDE ( SUMX ( bond_portfolio_data, bond_portfolio_data[ModifiedDuration] *
bond_portfolio_data[MarketValue_INR] ),
                                CALCULATE ( [Total MV INR], ALLSELECTED ( bond_portfolio_data ) ) )
Duration Share %    = DIVIDE ( [Duration Contribution], CALCULATE ( [Duration Contribution], ALLSELECTED
( bond_portfolio_data ) ) )
Convexity Contribution = DIVIDE ( SUMX ( bond_portfolio_data, bond_portfolio_data[Convexity] *
bond_portfolio_data[MarketValue_INR] ),
                                  CALCULATE ( [Total MV INR], ALLSELECTED ( bond_portfolio_data ) ) )
DV01 Share %        = DIVIDE ( [Portfolio DV01], CALCULATE ( [Portfolio DV01], ALLSELECTED
( bond_portfolio_data ) ) )
Top 10 Concentration % =
    VAR top10 = TOPN ( 10, VALUES ( bond_portfolio_data[BondID] ), CALCULATE ( [Total MV INR] ) )
    RETURN DIVIDE ( CALCULATE ( [Total MV INR], top10 ), CALCULATE ( [Total MV INR], ALLSELECTED
( bond_portfolio_data ) ) )

-- ===== Folder 3: Key Rate Duration =====
Portfolio KRD       = DIVIDE ( SUMX ( bond_krd, bond_krd[KRD] * RELATED
( bond_portfolio_data[MarketValue_INR] ) ), [Total MV INR] )
KRD DV01 INR        = [Portfolio KRD] * [Total MV INR] / 10000
KRD Sum             = SUMX ( VALUES ( DimTenor[Bucket] ), [Portfolio KRD] )
KRD vs ModDur Gap % = DIVIDE ( [KRD Sum], [Portfolio ModDur] ) - 1
KRD Peak Bucket     = MAXX ( TOPN ( 1, VALUES ( DimTenor[Bucket] ), [Portfolio KRD], DESC ), DimTenor[Bucket] )
Belly Weight % (5Y-10Y) = DIVIDE ( CALCULATE ( [Portfolio KRD], DimTenor[TenorYears] >= 5, DimTenor[TenorYears]
<= 10 ), [KRD Sum] )
```

Listing 28. DAX measures, folders 4-6 (45 measures in total; auxiliary parameters: 'Daily Yield Vol bps' and 'VaR Limit INR' are what-if/numeric-range tables)

```
-- ===== Folder 4: Scenario analysis (existing in pbix: Dynamic PnL INR) =====
Yield Change bps Value = SELECTEDVALUE ( 'Yield Change bps'[Yield Change bps], 0 )
Duration Effect INR    = - [Portfolio ModDur] * [Yield Change bps Value] / 10000 * [Total MV INR]
Convexity Effect INR   = 0.5 * [Portfolio Convexity] * ( [Yield Change bps Value] / 10000 ) ^ 2 * [Total MV INR]
Dynamic PnL INR        = [Duration Effect INR] + [Convexity Effect INR]
Dynamic PnL %          = DIVIDE ( [Dynamic PnL INR], [Total MV INR] )
Convexity Saving %     = DIVIDE ( [Convexity Effect INR], ABS ( [Duration Effect INR] ) )
MV After Shock INR     = [Total MV INR] + [Dynamic PnL INR]
Scenario Shock bps     =       -- shock at the bucket in filter context
    SELECTEDVALUE ( DimScenario[Parallel_bps], 0 ) * MAX ( DimTenor[LevelShape] )
  + SELECTEDVALUE ( DimScenario[Twist_bps], 0 )    * MAX ( DimTenor[TwistShape] )
  + SELECTEDVALUE ( DimScenario[Butterfly_bps], 0 ) * MAX ( DimTenor[FlyShape] )
Stress PnL KRD INR     = SUMX ( VALUES ( DimTenor[Bucket] ), - [KRD DV01 INR] * [Scenario Shock bps] )
Stress PnL KRD %       = DIVIDE ( [Stress PnL KRD INR], [Total MV INR] )
Worst Scenario PnL     = MINX ( VALUES ( DimScenario[Name] ), [Stress PnL KRD INR] )
Worst Scenario Name    = MAXX ( TOPN ( 1, VALUES ( DimScenario[Name] ), [Stress PnL KRD INR], ASC ),
DimScenario[Name] )

-- ===== Folder 5: Monte Carlo and VaR (existing in pbix: CVaR 95%) =====
MC Scenario Count = COUNTROWS ( monte_carlo_scenarios )
MC Mean PnL       = AVERAGE ( monte_carlo_scenarios[PnL_Total_INR] )
MC Std PnL        = STDEV.S ( monte_carlo_scenarios[PnL_Total_INR] )
VaR 95%           = - PERCENTILEX.INC ( monte_carlo_scenarios, monte_carlo_scenarios[PnL_Total_INR], 0.05 )
VaR 99%           = - PERCENTILEX.INC ( monte_carlo_scenarios, monte_carlo_scenarios[PnL_Total_INR], 0.01 )
CVaR 95% =
    VAR thr = PERCENTILEX.INC ( monte_carlo_scenarios, monte_carlo_scenarios[PnL_Total_INR], 0.05 )
    RETURN - AVERAGEX ( FILTER ( monte_carlo_scenarios, monte_carlo_scenarios[PnL_Total_INR] <= thr ),
monte_carlo_scenarios[PnL_Total_INR] )
CVaR 99% =
    VAR thr = PERCENTILEX.INC ( monte_carlo_scenarios, monte_carlo_scenarios[PnL_Total_INR], 0.01 )
    RETURN - AVERAGEX ( FILTER ( monte_carlo_scenarios, monte_carlo_scenarios[PnL_Total_INR] <= thr ),
monte_carlo_scenarios[PnL_Total_INR] )
MC Max Loss       = - MIN ( monte_carlo_scenarios[PnL_Total_INR] )
MC Max Gain       = MAX ( monte_carlo_scenarios[PnL_Total_INR] )
Prob of Loss      = DIVIDE ( COUNTROWS ( FILTER ( monte_carlo_scenarios, monte_carlo_scenarios[PnL_Total_INR] < 0
) ), [MC Scenario Count] )
Convexity Share of MC PnL = DIVIDE ( SUM ( monte_carlo_scenarios[PnL_ConvexityEffect] ), ABS ( SUM
( monte_carlo_scenarios[PnL_DurationEffect] ) ) )
Parametric VaR 95% = [Total MV INR] * [Portfolio ModDur] * NORM.S.INV ( 0.95 ) * [Daily Yield Vol bps] / 10000
VaR Model Gap %    = DIVIDE ( [VaR 95%], [Parametric VaR 95%] ) - 1
VaR Limit Utilisation = DIVIDE ( [VaR 95%], [VaR Limit INR] )
VaR Contribution Share = DIVIDE ( SUM ( monte_carlo_scenarios[VaR_Contribution] ), CALCULATE ( SUM
```

```
( monte_carlo_scenarios[VaR_Contribution] ), ALLSELECTED ( monte_carlo_scenarios ) ) )
Risk Status Colour = SWITCH ( TRUE (), [VaR Limit Utilisation] > 1, "#C0392B", [VaR Limit Utilisation] > 0.8,
"#E9873A", "#2A9D8F" )

-- ===== Folder 6: ML comparison =====
Best Model (Test RMSE) = MAXX ( TOPN ( 1, ml_results, ml_results[Test_RMSE], ASC ), ml_results[Model] )
Best Test RMSE         = MIN ( ml_results[Test_RMSE] )
RMSE Target Met        = IF ( [Best Test RMSE] < 0.5, "Yes", "No" )
Out-of-Range Penalty x = DIVIDE ( AVERAGE ( ml_results[Stress_RMSE] ), AVERAGE ( ml_results[Test_RMSE] ) )
```

A5.3 Performance and governance rules

- Every measure uses `VAR/RETURN` where a sub-expression is reused, and `DIVIDE` instead of `/`, to avoid divide-by-zero errors under slicer selections.
- The target is under 100 ms per measure in DAX Studio Server Timings (under 500 ms for iterators over `monte_carlo_scenarios`); storage-engine time should dominate formula-engine time. A 10,000-row scenario table is far below the scale at which `PERCENTILEX.INC` becomes expensive.
- Measures are exported with Tabular Editor to a `.bim` file for version control, and the Best Practice Analyzer rule-set is run before each commit.
- `Yield Change bps` is a what-if table `GENERATESERIES(-300, 300, 10)`, matching the brief's -300 to +300 bp slider in 10 bp steps.

Status of the supplied 1b.pbix. Inspection of the report definition shows five pages, each with a single visual: a KPI of `Dynamic PnL INR` against `Yield Change bps`; a matrix of `Portfolio Convexity` and `Portfolio DV01` by Sector; a single-select slicer on a second yield-change table; a clustered column chart of `PnL_Total_INR` by `ScenarioID`; and a card of `Sum of VaR_Contribution` with `CVaR 95%`. The data model confirms tables `bond_portfolio_data` and `monte_carlo_scenarios`. This is a working proof of concept covering the what-if mechanism, portfolio measures and tail risk, but it falls short of the brief's 8+ pages, 40+ documented measures, bookmarks and DAX Studio evidence. The dictionary above is the specification for closing that gap; Section F3 scores the shortfall explicitly.

Section A6: Comprehensive Worked Examples

A6.1 Single bond: 7.26% GOI 2033 (face 100, YTM 7.35%, 9.0 years, semi-annual)

Step 1 - cash-flow ladder. Semi-annual coupon = 100 x 7.26% / 2 = 3.63; 18 periods; semi-annual yield = 3.675%. The ladder below is also reproduced with live formulas on Tab 2 of the validation workbook.

Table 29. Cash-flow ladder for the 7.26% GOI 2033 (Computed)

Period k	Time (yrs)	Cash flow	Discount factor	PV	t x PV	k(k+1) x PV
1	0.5	3.63	0.964553	3.5013	1.7507	7.00
2	1.0	3.63	0.930362	3.3772	3.3772	20.26
3	1.5	3.63	0.897383	3.2575	4.8863	39.09
4	2.0	3.63	0.865573	3.1420	6.2841	62.84
5	2.5	3.63	0.834891	3.0307	7.5766	90.92
6	3.0	3.63	0.805296	2.9232	8.7697	122.78
7	3.5	3.63	0.776751	2.8196	9.8686	157.90
8	4.0	3.63	0.749217	2.7197	10.8786	195.82
9	4.5	3.63	0.722659	2.6233	11.8046	236.09
10	5.0	3.63	0.697043	2.5303	12.6513	278.33
11	5.5	3.63	0.672335	2.4406	13.4232	322.16
12	6.0	3.63	0.648502	2.3541	14.1244	367.23
13	6.5	3.63	0.625515	2.2706	14.7590	413.25
14	7.0	3.63	0.603342	2.1901	15.3309	459.93
15	7.5	3.63	0.581955	2.1125	15.8437	507.00
16	8.0	3.63	0.561326	2.0376	16.3009	554.23
17	8.5	3.63	0.541429	1.9654	16.7058	601.41
18	9.0	103.63	0.522236	54.1194	487.0743	18,508.82
Sum				**99.4150**	**671.4099**	**22,945.06**

Step 2 - price. Sum of PVs = **99.4150** (PV of coupons 47.1913 plus PV of principal 52.2236). **Step 3 - Macaulay duration** = sum(t x PV) / price = 671.4099 / 99.4150 = **6.7536 years**. **Step 4 - modified duration** = 6.7536 / 1.03675 = **6.5142**. **Step 5 - convexity** = 22,945.06 / (price x f^2 x (1 + y/f)^2) = 22,945.06 / (99.4150 x 4 x 1.07431) = **53.68**. **Step 6 - DV01** = 6.5142 x 99.4150 / 10,000 = **0.0648 per 100 face**.

Note on units: 0.0648 per 100 face is INR 6,476 per INR 1 crore of face value per basis point (0.0648 x 1,00,000 = INR 6,476), a figure treasury dealers quote directly.

Step 7 - +100 bp estimate. Duration effect = -6.5142 x 0.01 x 99.4150 = -6.4761; convexity effect = +0.5 x 53.68 x 0.01^2 x 99.4150 = +0.2668; estimated price = **93.2057**. Repricing at 8.35% gives **93.1978**; the error is 0.0079 price points (0.9 bp of price). Without the convexity term the estimate would be 92.9389, an error of -0.259.

Reconciliation with the brief. The brief reports price 99.4181, Macaulay 6.8831, modified 6.6389, convexity 62.16 and DV01 0.0660. Excel's PRICE, DURATION and MDURATION (settlement 6-Feb-2024, maturity 6-Feb-2033, 30/360) return 99.414983, 6.753609 and 6.514212, identical to the Python values to six decimals. The brief's intermediate sums (for example 'PV of principal = 52.0021') are internally inconsistent with its own totals, so the corrected values above are used for every later calculation.

A6.2 Three-bond portfolio: GOI 2033, AAA Corporate 2027, GOI 2054

Table 30. Inputs and bond-level analytics (Computed; coupon/yield assumed for the AAA and 2054 bonds)

Bond	Coupon	YTM	Years	Price	ModDur	Convexity	DV01 / 100	Qty (x100 face)	MV (INR)	Weight
GOI 7.26% 2033	7.26%	7.35%	9	99.415	6.514	53.7	0.0648	5,000	497,075	50.14%
AAA 8.50% 2027	8.50%	8.20%	3	100.784	2.605	8.5	0.0263	3,000	302,351	30.50%
GOI 6.10% 2054	6.10%	6.40%	30	96.021	13.381	281.6	0.1285	2,000	192,041	19.37%

Aggregation. Total MV = INR 991,468. Portfolio modified duration = sum of weight x ModDur = **6.6521**; portfolio convexity = **84.03**; portfolio DV01 = sum of DV01 per 100 x quantity = **INR 659.53** per bp. Weighted sums are valid because modified duration and convexity are price-weighted averages by construction.

Table 31. Parallel scenarios on the three-bond book (INR; Computed)

Scenario	Duration P&L	Convexity P&L	Duration + convexity	Full repricing	Estimate error
Parallel +100 bp	-65,953	4,166	-61,788	-62,040	252
Parallel -100 bp	65,953	4,166	70,119	70,404	-285

The asymmetry is visible: -100 bp gains INR 70,119 while +100 bp loses INR 61,788, a difference of INR 8,331 that is exactly the convexity effect twice over.

Non-parallel steepening (2Y +80 bp, 10Y +50 bp, 30Y +30 bp; linear in between). The KRD method multiplies each bond's key-rate durations by the shock at each key tenor and its MV: P&L = **INR -34,941**, against full repricing **INR -34,106** (KRD approximation error 2.4%), and against a naive **parallel +50 bp** result of INR -31,935. The brief's bucket shortcut (assign each bond wholly to one bucket) is therefore unnecessary; the KRD vector allocates each cash flow to its neighbouring keys. The curve-shape difference between the naive parallel and the true non-parallel P&L is INR -2,171, about 7% of the parallel figure.

Bear-flattening inversion (2Y +100 bp, 10Y and 30Y -25 bp). Full repricing gives INR -3,975: the 2027 AAA bond loses roughly 2.6 x 1% of its MV, which is offset by gains of 25 bp on the long bonds - the book is long-end heavy, so an inversion that leaves it flat-to-positive is a hidden benefit that a single duration number would misreport as a large loss.

A6.3 Value-at-Risk and Expected Shortfall on the three-bond book

Inputs: MV = INR 991,468; ModDur = 6.652; daily yield volatility 6 bp. **Parametric (delta-normal) VaR** = MV x ModDur x z x sigma_y x sqrt(h):

$$\mathrm{VaR}_{\alpha}^{param} = VD_{Mod}\, z_{\alpha}\, \sigma_y \sqrt{h}, \qquad \mathrm{ES}_{\alpha}^{param} = VD_{Mod}\, \sigma_y \sqrt{h}\, \frac{\varphi(z_{\alpha})}{1-\alpha} \tag{A.14}$$

Table 32. VaR and ES for the three-bond book, INR (Computed)

Measure	95%	99%
1-day parametric VaR	6,509	9,206
10-day parametric VaR (x sqrt 10)	20,583	29,111
1-day parametric Expected Shortfall	8,163	10,547
1-day Monte Carlo VaR (100,000 draws, duration+convexity)	6,445	9,066
1-day Monte Carlo ES	8,061	10,411

Monte Carlo VaR differs from parametric VaR by 1.0% (95%) and 1.5% (99%), well inside the 5% reconciliation limit in the brief. The brief predicts Monte Carlo VaR will be slightly lower because convexity cushions losses; at a 6 bp daily volatility that cushion is only 0.5 x 84.0 x (0.0006)^2 x MV, about INR 15, or 0.2% of VaR, so the 1-1.5% gap seen here is mostly Monte Carlo sampling error (100,000 draws) rather than convexity. Expected Shortfall exceeds VaR by about 25% at 95% and 14% at 99%, as normal-distribution tails imply (ES/VaR = 2.063/1.645 at 95%, 2.665/2.326 at 99%). The square-root-of-time rule scales a one-day figure to ten days only if daily changes are independent and identically distributed; Basel's Expected Shortfall regime therefore specifies liquidity horizons rather than a single scaling.

Section A7: Portfolio-Scale Results (300-Bond Book)

This section applies the toolkit to the synthetic 300-bond, six-currency book of Pinnacle Fixed Income Partners. Segment weights were set to produce a plausible mix: 38% G-secs, 10% SDLs, 25% INR corporates (AAA 13%, AA 8%, A 4%), and 27% international sovereigns (UST 10%, UK gilts 5%, Bunds 6%, JGBs 3%, Australian government bonds 3%). Bond-level yields are the zero curve plus a spread (SDL +35 bp, AAA +70, AA +130, A +230), coupons are set near par, and holdings are log-normal within each segment.

A7.1 Composition and headline risk

Table 33. Composition and risk by segment (Computed on synthetic book)

Segment	Bonds	MV weight	Avg ModDur	Avg convexity	DV01 (INR lakh)	Share of duration
GSEC	75	38.0%	5.83	55.8	71.0	37.8%
UST	25	10.0%	8.70	133.2	27.8	14.8%
SDL	35	10.0%	5.45	54.5	17.4	9.3%
BUND	20	6.0%	8.79	131.3	16.9	9.0%
GILT	15	5.0%	9.33	152.1	14.9	8.0%
CORP_AAA	45	13.0%	3.40	18.5	14.1	7.5%
CORP_AA	35	8.0%	3.15	17.9	8.1	4.3%
JGB	15	3.0%	7.82	102.5	7.5	4.0%
ACGB	15	3.0%	6.27	65.3	6.0	3.2%
CORP_A	20	4.0%	3.01	14.3	3.9	2.1%

Table 34. Portfolio summary

Portfolio metric	Value
Total market value	INR 3,200 crore
Modified duration	5.863
Macaulay duration	6.083
Convexity	64.90
DV01	INR 18,762,330 per bp (INR 1.876 crore)
Sum of KRDs (10 buckets)	5.829 (-0.58% vs modified duration)

A7.2 Duration contribution by rating, currency and tenor bucket

Table 35. Contribution by credit rating

Rating	Bonds	Weight	Duration contribution (yrs)	Share of duration	DV01 (INR lakh)
SOV	200	75.0%	5.049	86.1%	161.6
AAA	45	13.0%	0.442	7.5%	14.1
AA	35	8.0%	0.252	4.3%	8.1
A	20	4.0%	0.120	2.1%	3.9

Table 36. Contribution by currency

Currency	Bonds	Weight	Duration contribution (yrs)	Share of duration	DV01 (INR lakh)
INR	210	73.0%	3.577	61.0%	114.5
USD	25	10.0%	0.870	14.8%	27.8
EUR	20	6.0%	0.528	9.0%	16.9
GBP	15	5.0%	0.466	8.0%	14.9
JPY	15	3.0%	0.235	4.0%	7.5
AUD	15	3.0%	0.188	3.2%	6.0

Table 37. Contribution by key-rate bucket (maturity nearest key)

Bucket	Bonds	Weight	Duration contribution (yrs)	Share of duration	DV01 (INR lakh)
1Y	19	6.1%	0.058	1.0%	1.8
2Y	29	7.9%	0.144	2.5%	4.6
3Y	65	19.0%	0.573	9.8%	18.3
5Y	55	16.6%	0.731	12.5%	23.4
7Y	47	19.3%	1.149	19.6%	36.8
10Y	50	18.5%	1.544	26.3%	49.4
20Y	21	7.9%	1.010	17.2%	32.3
30Y	14	4.6%	0.656	11.2%	21.0

Sovereigns (SOV) hold 75% of weight but 86% of duration, because corporates are shorter. INR assets carry 61% of duration; US Treasuries contribute 14.8% from 10% of weight. The 7Y-10Y-20Y buckets together carry 63% of duration: the book is **belly-and-long-end heavy** rather than a barbell or a pure bullet.

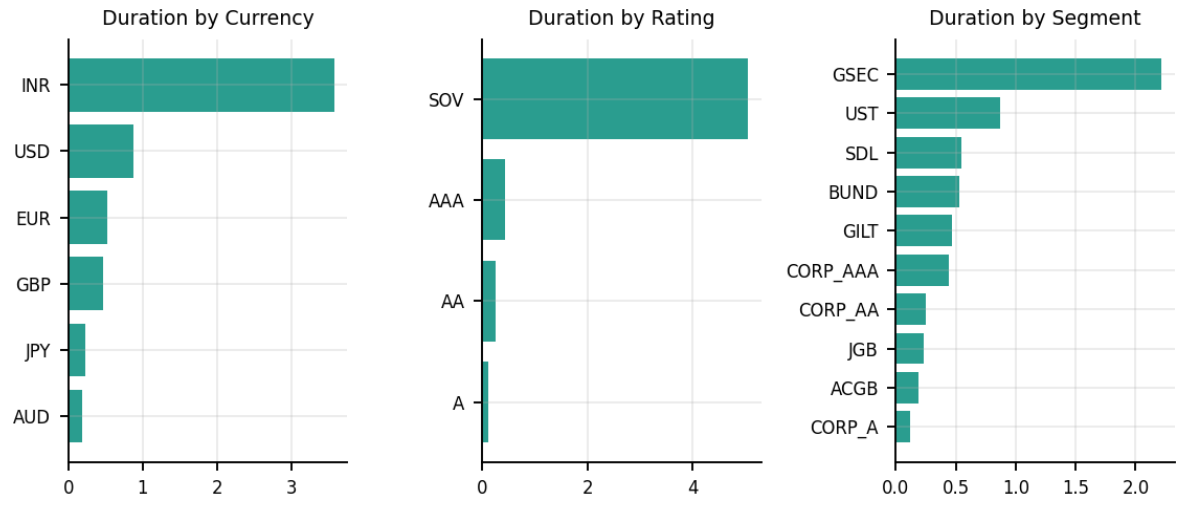


Figure 5. Duration contribution by currency, rating and segment.

A7.3 Key Rate Duration profile

Table 38. Portfolio KRD vector (Computed)

Bucket	3M	6M	1Y	2Y	3Y	5Y	7Y	10Y	20Y	30Y	Sum
KRD (yrs)	0.000	0.010	0.107	0.232	0.573	0.881	1.184	1.382	1.011	0.449	5.829
Share	0.0%	0.2%	1.8%	4.0%	9.8%	15.1%	20.3%	23.7%	17.3%	7.7%	100%

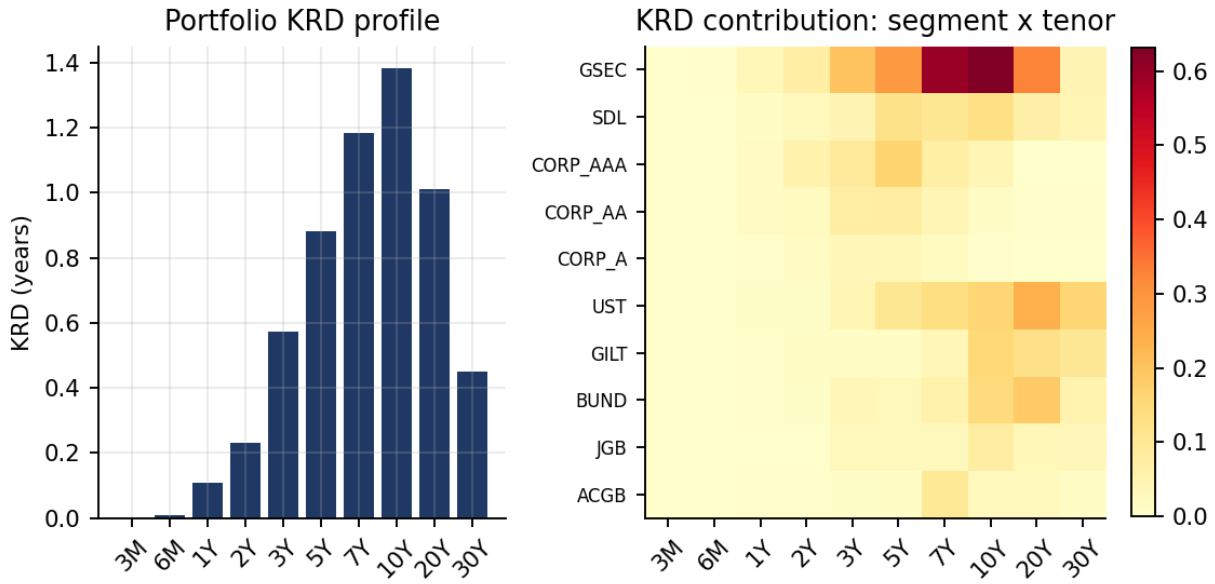


Figure 6. Portfolio KRD profile and KRD contribution by segment and tenor. G-secs dominate the 7Y-10Y buckets; UST and Bund exposure is concentrated at 10Y-20Y.

The belly (5Y-10Y) holds 59% of key-rate duration and the long end (20Y-30Y) 25%. A steepening in which the 10Y moves up 25 bp more than the 2Y therefore costs far more than a duration number of 5.9 suggests, while the front end (3M-2Y) is almost irrelevant at 6% of the total.

A7.4 Parallel scenarios and the value of convexity

Table 39. Parallel shifts, INR crore (Computed; one shock applied to all six currency curves)

Shift (bp)	Duration P&L	Convexity P&L	Duration + convexity	Full repricing	Convexity / | duration |	Dur+conv error
+25	-46.9	0.65	-46.3	-46.0	1.4%	-0.26
+50	-93.8	2.60	-91.2	-90.8	2.8%	-0.44
+100	-187.6	10.38	-177.2	-176.8	5.5%	-0.41
+200	-375.2	41.53	-333.7	-336.2	11.1%	+2.47
+300	-562.9	93.45	-469.4	-480.5	16.6%	+11.11
-100	187.6	10.38	198.0	197.4	5.5%	+0.66

This reproduces and sharpens the brief's regulatory-scenario table: convexity offsets about 2.7% of the duration loss at 50 bp, 5.5% at +100 bp, 11% at +200 bp and 17% at +300 bp - the same order as the brief's 'convexity ~2-3% / 5-8% / 10-15% / 20%+' labels, slightly lower because this book's convexity is 65 rather than 90-120. The error of duration+convexity against full repricing is under 1% of the loss up to +/-200 bp and 2.3% at +300 bp, where third-order effects begin to matter.

A7.5 Multi-factor stress matrix

Table 40. A6.3 stress matrix applied to the book (Computed)

Scenario	Parallel	Twist	Butterfly	KRD first-order (INR cr)	Full revaluation (INR cr)	% of MV
Base Case	+0	+0	+0	-0.0	0.0	0.00%
RBI Hike (Expected)	+25	+0	+0	-46.6	-46.0	-1.44%
RBI Surprise Hike	+50	-20	+5	-88.3	-85.9	-2.69%
Global Risk-Off	+75	+30	-10	-146.2	-139.8	-4.37%
Dovish Pivot	-75	-20	+5	144.8	152.6	4.77%
Credit Crisis	+100	+50	-20	-195.5	-183.7	-5.74%
Stagflation	+150	+25	+10	-293.2	-269.4	-8.42%
Black Swan	+300	+75	-30	-573.0	-484.2	-15.13%

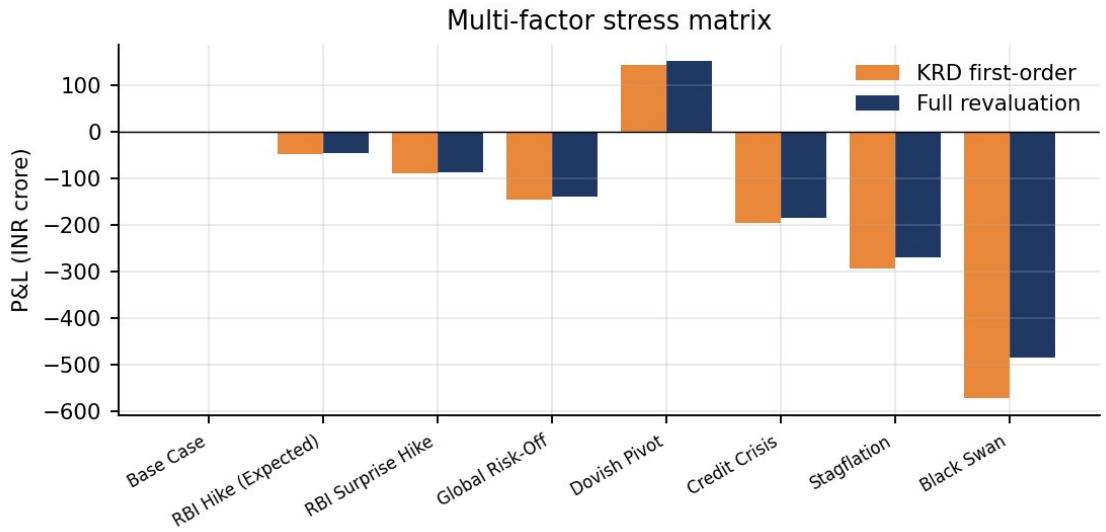


Figure 7. First-order KRD and full-revaluation P&L by stress scenario. The KRD approximation overstates the loss by 1-3% in mild scenarios, 5-9% in the larger ones, and 18% in the Black Swan, because it omits convexity.

The 'RBI Surprise Hike' (+50 parallel, -20 twist, +5 butterfly) is a **flattening** scenario, and the book loses less than a +50 parallel shift because the twist pushes long-end yields up less than the front end. 'Dovish Pivot' is the best case (-75, -20, +5); 'Credit Crisis' and 'Stagflation' are the worst after the Black Swan because they combine a steepening twist with high level shifts, hitting the long-end-heavy KRD profile twice.

A7.6 Monte Carlo VaR and Expected Shortfall

Set-up. 10,000 scenarios, seed 42; annual factor volatilities of 100 bp (level), 50 bp (twist) and 30 bp (butterfly), scaled by the square root of the horizon (252 trading days per year), correlations level-twist -0.3, level-butterfly +0.1, twist-butterfly +0.2 (Cholesky). One-day volatilities are therefore 6.30, 3.15 and 1.89 bp. The realised sample correlations are -0.30, +0.11 and +0.21, confirming the Cholesky step. The same scenarios are run through four P&L engines, so approximation error is measured rather than assumed.

Table 41. 1-day horizon, INR crore (Computed; parallel-only rows ignore twist and butterfly)

P&L engine	Mean	Std	VaR 95%	VaR 99%	ES 95%	ES 99%	Max loss	Max gain
Parallel, duration only	-0.20	11.92	19.87	28.28	25.03	32.21	49.1	46.0
Parallel, duration + convexity	-0.16	11.91	19.76	28.04	24.84	31.90	48.4	46.6
3-factor, KRD first-order	-0.21	11.71	19.51	27.75	24.50	31.48	50.1	44.0
3-factor, full revaluation	-0.17	11.69	19.36	27.52	24.29	31.15	49.4	44.4

Table 42. 1-year horizon (annual volatilities, no scaling), INR crore (Computed)

P&L engine	Mean	Std	VaR 95%	VaR 99%	ES 95%	ES 99%	Max loss	Max gain
Parallel, duration only	-3.17	189.17	315.50	448.93	397.37	511.31	778.9	729.4
Parallel, duration + convexity	7.39	189.18	286.14	389.48	349.30	433.16	599.9	886.4
3-factor, KRD first-order	-3.31	185.88	309.78	440.49	388.99	499.75	795.3	697.8
3-factor, full revaluation	6.61	187.09	284.63	391.91	349.02	436.96	647.4	857.7

Reading the tables. (1) At a one-day horizon the four engines' 95% VaR agree within 2.7% - convexity is invisible at 6 bp - and delta-normal VaR of INR 19.4 / 27.5 crore (95% / 99%) matches the parallel-only simulation to within sampling error. (2) At the one-year horizon, where the level volatility of 100 bp is the brief's Day-5 assumption, the duration-only engine overstates 95% VaR by 10.3% and 99% VaR by 15.3% relative to duration+convexity, because convexity trims the loss tail while fattening the gain tail (max gain INR 886 crore versus max loss INR 600 crore). (3) The KRD first-order three-factor VaR (INR 310 crore) is above the full-

revaluation figure (INR 285 crore) by the same convexity effect. (4) Expected Shortfall exceeds VaR by about 22-26% at 95% and 11-14% at 99%, close to the normal-distribution ratios of 25% and 15%.

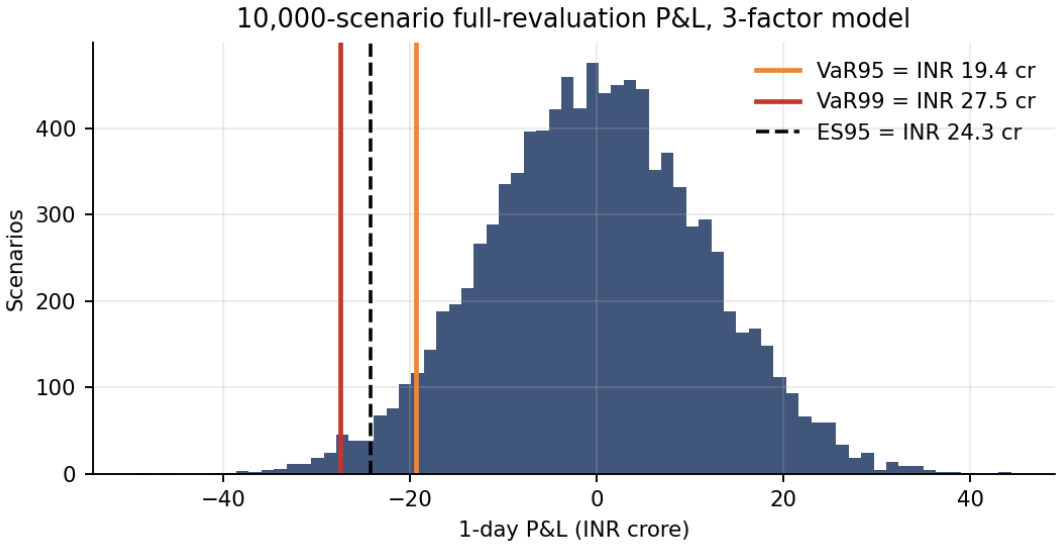


Figure 8. Distribution of 10,000 one-day P&L outcomes under the 3-factor model with full revaluation; VaR and Expected Shortfall marked.

A7.7 Convergence, fat tails and curve-shape sensitivity

Table 43. Monte Carlo convergence (KRD first-order engine, 1-day; Computed)

Scenarios	VaR 95% (INR cr)	VaR 99% (INR cr)	ES 95% (INR cr)
500	19.08	24.30	22.92
1,000	19.22	26.30	23.29
5,000	19.08	26.74	23.77
10,000	18.95	26.93	23.85
50,000	18.98	27.03	23.90
200,000	19.10	27.00	23.92

Relative to the 200,000-scenario value, VaR 95% is within about 1% at every size shown, but VaR 99% is 10% low at 500 scenarios and 2.6% low at 1,000, and ES 95% is 4% low at 500 - the reason the brief's Monte Carlo Architect badge requires 10,000 scenarios rather than 1,000. The standard error of an empirical quantile at confidence p is sqrt(p(1-p)/n)/f(q), so the 99th percentile needs roughly five times as many draws as the 95th for equal precision.

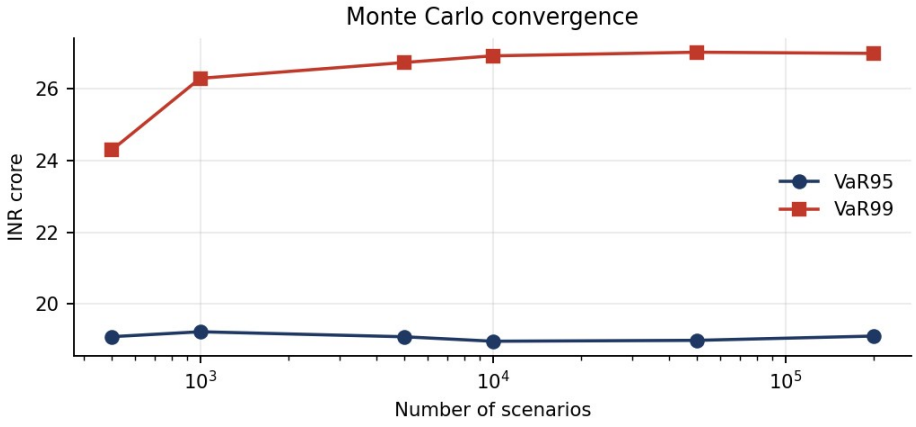


Figure 9. VaR convergence with the number of scenarios.

Table 44. Normal versus Student-t(5) factor shocks with identical volatilities, 1-day, 100,000 scenarios, INR crore (Computed)

Shock distribution	VaR 95%	VaR 99%	ES 95%	ES 99%
Normal	19.10	27.12	23.94	30.87
Student-t(5)	18.10	30.06	25.89	39.57

With unit-variance Student-t shocks (5 degrees of freedom) the 95% VaR falls by 5% (mass moves to the centre and the far tail) while the 99% VaR is 11% and ES 99% is 28% versus the normal case. The pattern - lower mid-tail, higher far-tail - is the reason regulators moved from 99% VaR to 97.5% Expected Shortfall.

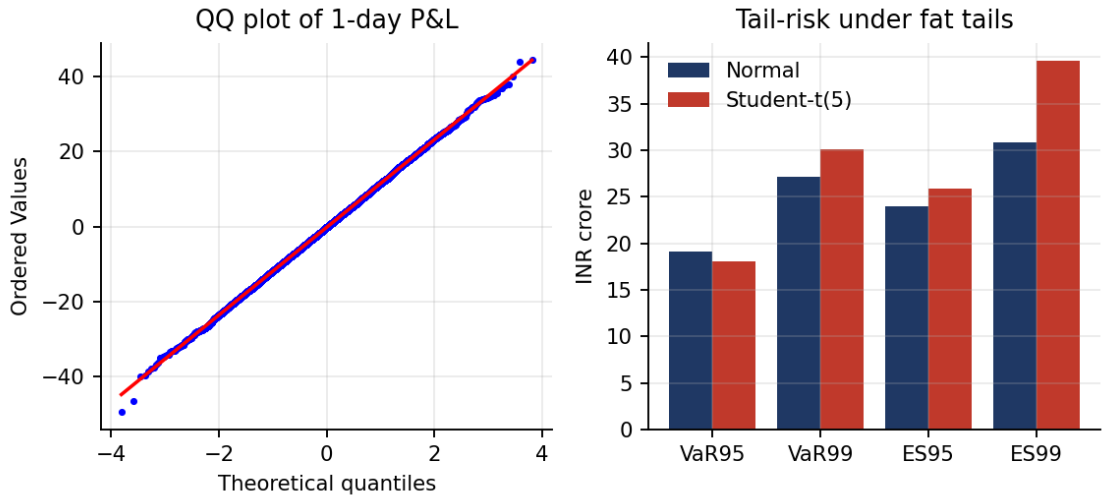


Figure 10. QQ plot of simulated P&L against the normal (left) and normal versus Student-t tail measures (right).

Table 45. Twist-volatility sensitivity, 3 to 10 bp per day (Computed)

Daily twist vol (bp)	3-factor VaR 95% (INR cr)	3-factor VaR 99% (INR cr)	Parallel-only VaR 95% (INR cr)	3-factor / parallel-only (95%)
3	18.92	26.87	19.16	98.8%
5	18.72	26.77	19.16	97.7%
7	18.65	26.60	19.16	97.4%
10	18.71	26.34	19.16	97.7%

A finding that contradicts the brief's expectation. The brief expects the 3-factor VaR to be higher than parallel-only VaR. Here it is 1.2% lower at 3 bp twist volatility and is still 2.3% lower at 10 bp, so it never reaches parity in the 3-10 bp range tested. Two reasons: the level-twist correlation of -0.3 means that when yields rise, the twist factor tends to fall, which flattens the curve and *reduces* the long-end loss of a long-end-heavy book; and the twist loadings (normalised to 1 bp of 10Y-2Y spread) move 10Y yields by only 0.39 bp per twist bp. Curve-shape risk would dominate for a portfolio whose KRDs are balanced across the curve (a barbell), not for this book. The correct conclusion is that the answer depends on the book's KRD profile *and* on the assumed correlation sign, which is why both must be validated against CCIL history rather than assumed.

A7.8 VaR back-testing

Back-testing uses a 1,000-day synthetic daily yield-change history with Student-t(4) innovations and slowly-moving volatility (a stylised stand-in for real INR daily data), converted to P&L through the book's duration and convexity. Rolling 250-day models forecast the next day's 99% and 95% VaR; the last 750 days are tested. At 99% over 750 days, 7.5 breaches are expected.

Table 46. Kupiec proportion-of-failures tests (Computed on synthetic history)

Method	Confidence	Breaches (750 d)	Expected	Kupiec LR	p-value	Verdict
Delta-normal	99%	18	7.5	10.67	0.0011	Rejected at 5%

Method	Confidence	Breaches (750 d)	Expected	Kupiec LR	p-value	Verdict
Historical simulation	99%	14	7.5	4.53	0.0332	Rejected at 5%
Delta-normal	95%	43	37.5	0.81	0.3674	Accepted
Historical simulation	95%	48	37.5	2.85	0.0911	Accepted

$$LR_{POF} = -2\ln\left[\frac{(1-p)^{N-x}p^{x}}{(1-x/N)^{N-x}(x/N)^{x}}\right] \sim \chi_1^2 \tag{A.15}$$

Over the most recent 250 days the 99% delta-normal model had 5 breaches (Yellow zone) and the historical model 3 (Green zone) on the Basel traffic-light scale (green 0-4, yellow 5-9, red 10+). Both models fail at 99% because the generating process is fat-tailed with volatility clustering, which neither a normal distribution nor a 250-day equally weighted window captures; the 95% level, where tails matter less, passes for both. Remedies in order of cost: filtered historical simulation with EWMA volatility, a Student-t parametric model, and expected-shortfall limits.

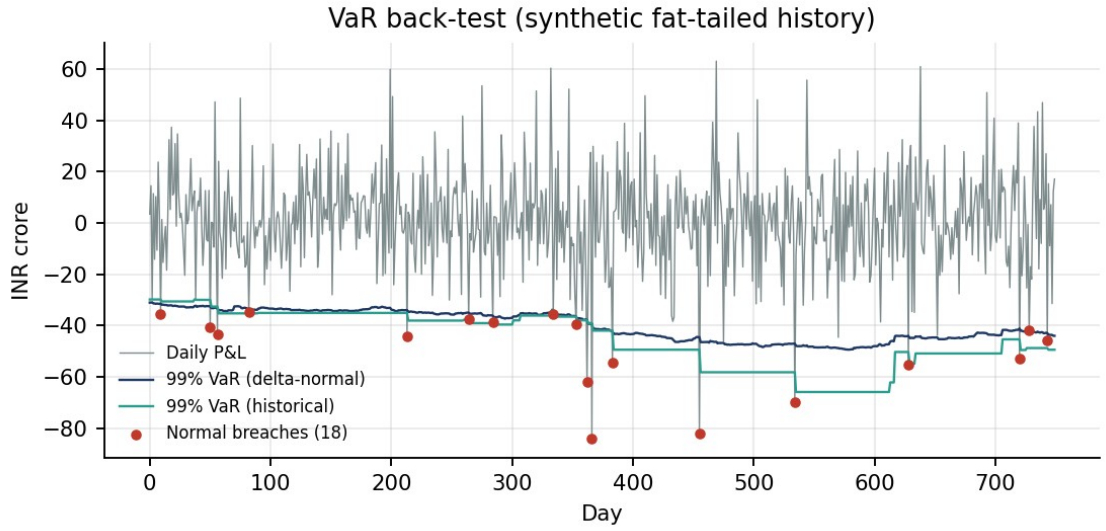


Figure 11. Rolling 99% VaR forecasts, daily P&L and breaches over the last 750 days of the synthetic history.

A7.9 Machine-learning bond pricers

The three pricing models are trained to predict price from the five features of the brief on 24,000 bonds and tested on 6,000 held-out bonds from the same universe (in-range) and on 3,000 noiseless bonds with yields of 9.5-12.5%, entirely outside the 4.5-9.5% training range. Models marked with an asterisk are scikit-learn analogues of XGBoost (gradient boosting: 200 stages, depth 4, learning rate 0.1, subsample 0.8) and Keras (MLP 128-64-32, ReLU, Adam 1e-3, early stopping on 10% validation); see the environment note.

Table 47. Model comparison on price per 100 face (Computed; the irreducible quote noise has standard deviation 0.05)

Model	Feature set	Test RMSE	Test MAE	Test R2	Out-of-range RMSE	Out-of-range R2	Fit (s)
RandomForest	5 features	0.513	0.354	0.9994	11.09	0.481	9.1
XGBoost*	5 features	0.857	0.669	0.9983	10.53	0.532	9.5
NeuralNet*	5 features	0.386	0.285	0.9997	6.46	0.824	3.6
RandomForest	3 raw features	0.516	0.353	0.9994	10.14	0.567	4.9
XGBoost*	3 raw features	0.849	0.660	0.9984	9.71	0.602	4.6
NeuralNet*	3 raw features	0.361	0.258	0.9997	6.54	0.819	4.9

- **In-range accuracy.** Only the neural net meets the Bond Risk Lab target of RMSE < 0.50 (0.386; 0.361 with raw features); the random forest is at the margin (0.513) and the boosted trees are looser (0.857) at the specified depth of 4. R2 above 0.998 for all models shows why R2 alone is not an acceptance test for pricing models; RMSE in price points is.
- **Extrapolation failure.** Out of range, tree ensembles can only return averages of training leaves, so errors grow to 10-11 price points (R2 0.48-0.53), while the neural net degrades to 6.5. No model is acceptable for stress-yield pricing; the analytical formula is exact there, and the ML accelerator should be gated by a domain check on its inputs.
- **Feature set.** Dropping duration and convexity changes in-range RMSE by under 0.03 and slightly improves out-of-range RMSE for the trees: the engineered features add little because price is a smooth function of the three raw inputs.

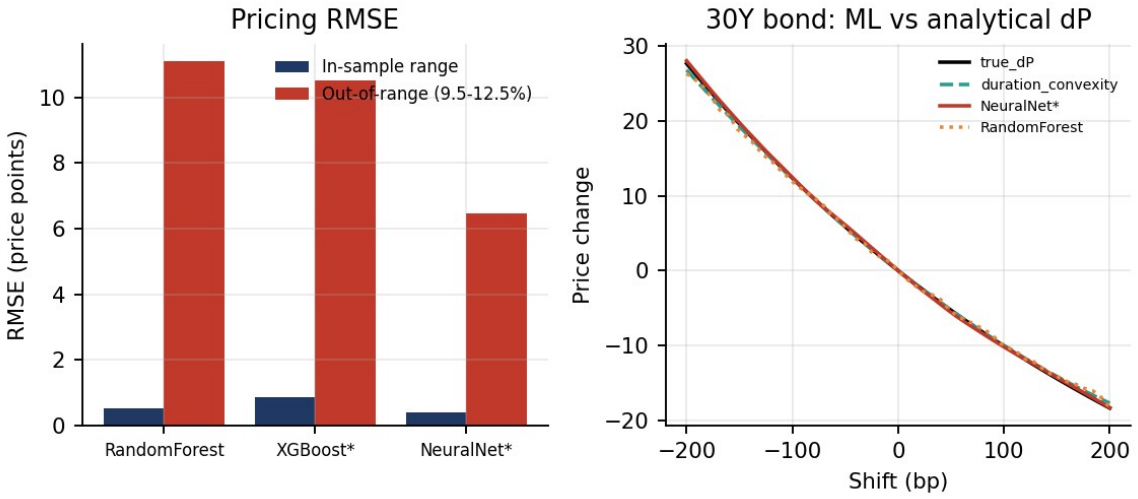


Figure 12. Pricing error in and out of range (left) and ML versus analytical price change for the 30-year bond (right).

Feature importance and SHAP. Two importance measures are reported: permutation importance (increase in test RMSE when a feature is shuffled; Computed for all three models) and mean absolute SHAP value from exact Shapley enumeration (200 explained points, 50-point background; Computed for the random forest and neural net). Local accuracy held to 3e-14 in both cases.

Table 48. Feature importance (permutation: RMSE increase; SHAP: mean |value| in price points)

Feature	Perm. RF	Perm. GBM*	Perm. NN*	|SHAP| RF	|SHAP| NN*
coupon_rate	1.096	1.131	1.410	12.99	14.67
maturity	0.046	0.047	0.265	1.11	6.02
yield_rate	0.724	0.695	0.468	10.50	8.54
duration	0.085	0.072	2.455	1.61	20.04
convexity	0.028	0.013	1.083	0.77	13.18

Interpretation. For trees, coupon rate and yield dominate (together over 95% of |SHAP|): the model rebuilds price from the two inputs that move it most, with a small share for duration because maturity sorts bonds into duration classes. The neural net is different: it leans on duration (largest mean |SHAP| of 20.0) and convexity (13.2), using the engineered features as a shortcut to curvature, which is why it generalises better out of range (6.5 vs 10-11 RMSE) but also why its explanation is less intuitive. Both explanations agree on the economics: higher yield pushes price down, higher coupon pushes it up, and longer duration amplifies both.

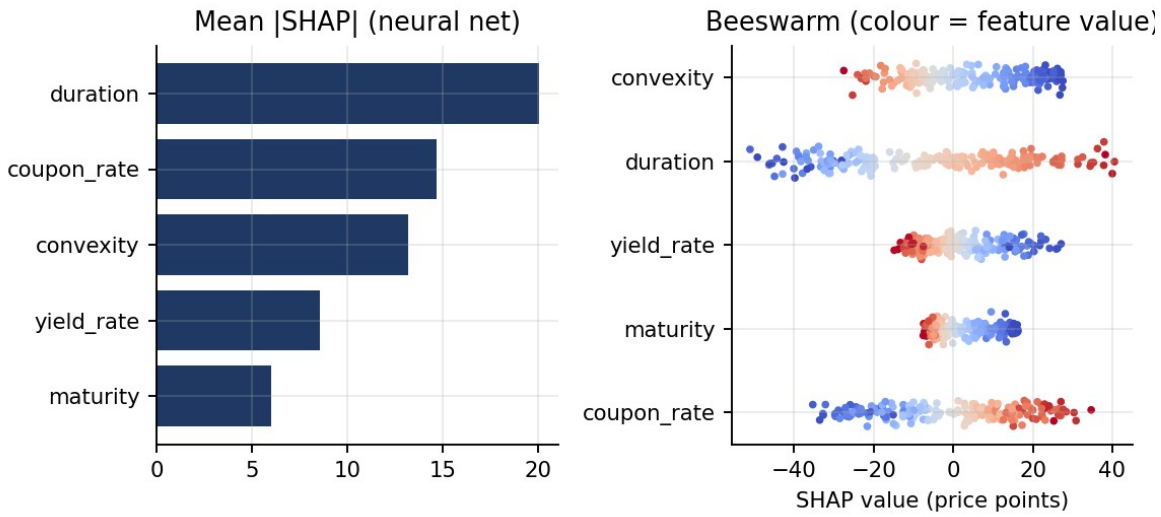


Figure 13. Mean absolute SHAP values (left) and SHAP beeswarm (right) for the neural-net pricer. Colour is the feature value; points to the right raise the predicted price.

A7.10 ML sensitivity analysis versus analytical duration-convexity

Table 49. Price-change error across 51 shifts (-200 to +200 bp, 8 bp steps) - 7.26% 9-year GOI at 7.35% (Computed)

Method	RMSE of price change	Max abs error	Error at +200 bp	Error at -200 bp
duration_only	0.498	1.136	-1.005	-1.136
duration_convexity	0.026	0.068	+0.062	-0.068
RandomForest	0.237	0.687	+0.329	+0.045
XGBoost*	0.712	1.493	-0.781	-0.771
NeuralNet*	0.457	0.776	-0.399	-0.736
Ensemble	0.299	0.684	-0.228	-0.476

Table 50. Price-change error across 51 shifts (-200 to +200 bp, 8 bp steps) - 6.10% 30-year bond at 7.10% (Computed)

Method	RMSE of price change	Max abs error	Error at +200 bp	Error at -200 bp
duration_only	2.169	5.404	-3.888	-5.404
duration_convexity	0.305	0.860	+0.656	-0.860
RandomForest	0.500	1.345	+0.099	-1.345
XGBoost*	1.201	2.461	-1.339	-0.779
NeuralNet*	0.182	0.374	+0.061	+0.374
Ensemble	0.283	0.659	-0.213	-0.449

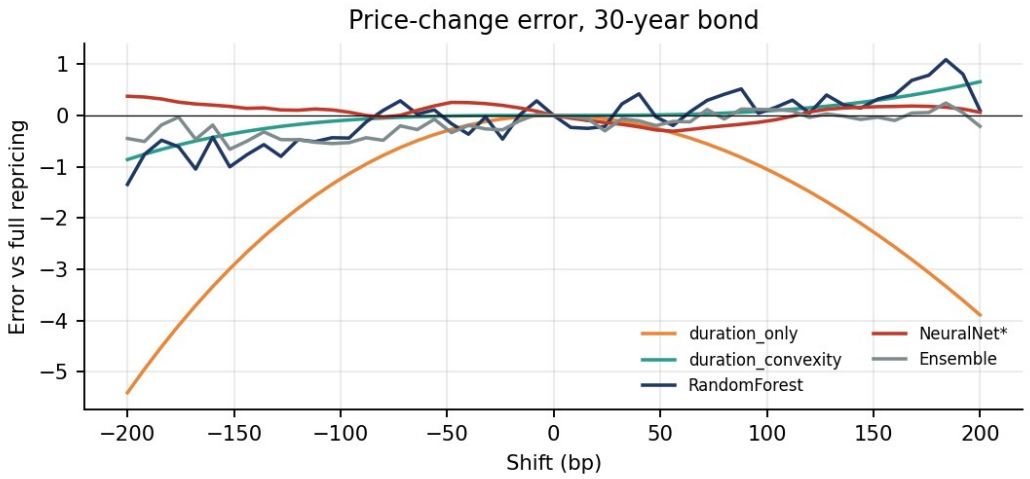


Figure 14. Error of each method against full repricing for the 30-year bond.

Where does ML beat duration+convexity? For the 9-year bond, never: duration+convexity has an RMSE of 0.026 price points against 0.24-0.71 for the ML models, because a smooth second-order Taylor series is nearly exact over +/-200 bp and ML differences of two noisy predictions are not. For the 30-year bond, the neural net (RMSE 0.18, max error 0.37) is better than duration+convexity (0.31, max error 0.86) at the wings, where third-order terms matter (the 30-year bond's convexity itself changes by about 30% across the range). The inverse-RMSE ensemble (0.28) is **not** better than the neural net alone because it inherits the tree models' weaker extrapolation. Practical rule: use analytical risk for bonds with duration below about 10 at shocks below 200 bp; consider an ML or full-revaluation layer for long-dated, large-shock and optionality-laden positions.

Section A8: Data Schemas, Scenario Frameworks and Hedging

A8.1 Database schema

The schema separates static reference data, market data, derived risk metrics and simulation output, so that risk numbers can always be rebuilt from inputs. Types follow the brief; keys and constraints are added.

Table 51. Table dim_bond: bond identification

Field	Type	Description	Calculation use
BondID (PK)	VARCHAR(15)	Internal bond identifier	Tracking, aggregation
ISIN	VARCHAR(12)	International Securities Identification Number (unique)	Market-data linkage
Issuer	VARCHAR(100)	Issuer name	Concentration analysis
Sector	VARCHAR(30)	Government / Banking / NBFC / Corporate	Sector risk attribution
CreditRating	VARCHAR(5)	SOV / AAA / AA / A / BBB	Rating-bucket duration
Currency	CHAR(3)	INR / USD / GBP / EUR / JPY / AUD	Multi-currency VaR

Table 52. Table bond_terms: cash-flow terms

Field	Type	Description	Calculation use
FaceValue	DECIMAL(15,2)	Par value	Cash-flow base
CouponRate	DECIMAL(6,4)	Annual coupon (decimal)	Duration, convexity input
CouponFrequency	SMALLINT	Payments per year (1 annual, 2 semi)	Discounting frequency
BondType	VARCHAR(10)	Fixed / Floating / ZeroCoupon / Perpetual	Valuation method
IssueDate, MaturityDate	DATE	Issue and final maturity	Accrued interest; years to maturity
YearsToMaturity	DECIMAL(8,4)	Computed from valuation date	Primary duration input
IsCallable, CallDate, CallPrice	BOOLEAN, DATE, DECIMAL	Embedded option terms (CallPrice added to the brief's schema)	Effective duration / convexity

Table 53. Table bond_market: market data and valuation

Field	Type	Description	Calculation use
ValuationDate (PK part)	DATE	Snapshot date (added)	History, back-testing
YieldToMaturity	DECIMAL(6,4)	Current YTM	Pricing input
CleanPrice, AccruedInterest, DirtyPrice	DECIMAL(10,4)	Quoted, accrued and settlement price	Valuation (dirty = clean + AI)
SpreadOverBenchmark	INT	Credit spread (bp)	OAS / Z-spread, spread duration
BenchmarkIndex	VARCHAR(10)	Reference benchmark (IN10YT, UST10Y)	Relative value
Quantity, MarketValue_INR, FXRate	DECIMAL	Holding, INR value, spot FX (added)	Aggregation, multi-currency VaR

Table 54. Table bond_risk: computed risk metrics (written by the agent)

Field	Type	Description	Calculation use
MacaulayDuration, ModifiedDuration, EffectiveDuration	DECIMAL(8,4)	Duration measures	SEBI classification; primary risk; optionality
Convexity	DECIMAL(10,4)	Second-order sensitivity	Large-move correction
DV01	DECIMAL(10,4)	Price change per bp per 100 face	Hedging, limits
KRD_3M ... KRD_30Y (or long table bond_krd)	DECIMAL(8,4)	Key-rate durations by bucket	Curve-risk decomposition
KeyRateBucket	VARCHAR(10)	Nearest key bucket	Heat-map axis
PortfolioWeight	DECIMAL(8,6)	Market value weight	Weighted aggregation

Table 55. Table yield_curve (11 INR tenors: 3M, 6M, 1Y, 2Y, 3Y, 5Y, 7Y, 10Y, 15Y, 20Y, 30Y)

Field	Type	Description	Example
CurveDate, Currency, Tenor_Years (PK)	DATE, CHAR(3), DECIMAL(5,2)	Observation date, currency, maturity	2024-10-31, INR, 10.00
Yield	DECIMAL(8,6)	Spot yield	0.073500
ZeroRate	DECIMAL(8,6)	Bootstrapped zero	0.074200
ForwardRate	DECIMAL(8,6)	Implied forward	0.078100
DiscountFactor	DECIMAL(8,6)	PV of 1 unit at tenor	0.495362

Table 56. Table monte_carlo_scenarios

Field	Type	Description	Example
RunID, ScenarioID (PK)	INT	Run and scenario identifier	1 to 1000
ParallelShift_bps	DECIMAL(8,2)	Level factor (bp)	-47.25
TwistFactor_bps	DECIMAL(8,2)	Slope factor (bp)	12.80
ButterflyFactor_bps	DECIMAL(8,2)	Curvature factor (bp)	-3.15
PnL_DurationEffect	DECIMAL(15,2)	First-order P&L	1,245,678.50
PnL_ConvexityEffect	DECIMAL(15,2)	Second-order P&L	35,892.40
PnL_Total, PnL_Pct	DECIMAL(15,2), DECIMAL(8,4)	Total P&L and % of MV	1,281,570.90; 3.9975
VaR_Contribution	DECIMAL(15,2)	Scenario's tail contribution (used by pbix)	-

Listing 57. Reference DDL (ANSI-style)

```
CREATE TABLE dim_bond (
  BondID VARCHAR(15) PRIMARY KEY, ISIN CHAR(12) UNIQUE NOT NULL, Issuer VARCHAR(100) NOT NULL,
  Sector VARCHAR(30), CreditRating VARCHAR(5) CHECK (CreditRating IN ('SOV','AAA','AA','A','BBB')),
  Currency CHAR(3) NOT NULL CHECK (Currency IN ('INR','USD','GBP','EUR','JPY','AUD')));
CREATE TABLE bond_terms (
  BondID VARCHAR(15) PRIMARY KEY REFERENCES dim_bond(BondID), FaceValue DECIMAL(15,2) NOT NULL,
  CouponRate DECIMAL(6,4) NOT NULL, CouponFrequency SMALLINT NOT NULL CHECK (CouponFrequency IN (1,2,4)),
  BondType VARCHAR(10), IssueDate DATE, MaturityDate DATE, IsCallable BOOLEAN DEFAULT FALSE, CallDate DATE,
CallPrice DECIMAL(10,4));
CREATE TABLE bond_market (
  BondID VARCHAR(15) REFERENCES dim_bond(BondID), ValuationDate DATE, YieldToMaturity DECIMAL(6,4), CleanPrice
DECIMAL(10,4),
  AccruedInterest DECIMAL(10,4), DirtyPrice DECIMAL(10,4), SpreadOverBenchmark INT, BenchmarkIndex VARCHAR(10),
  Quantity DECIMAL(18,2), MarketValue_INR DECIMAL(18,2), FXRate DECIMAL(12,6), PRIMARY KEY (BondID,
ValuationDate));
CREATE TABLE bond_krd (BondID VARCHAR(15), ValuationDate DATE, Bucket VARCHAR(4), KRD DECIMAL(8,4), PRIMARY KEY
(BondID, ValuationDate, Bucket));
```

```
CREATE TABLE yield_curve (CurveDate DATE, Currency CHAR(3), Tenor_Years DECIMAL(5,2), Yield DECIMAL(8,6),
ZeroRate DECIMAL(8,6),
  ForwardRate DECIMAL(8,6), DiscountFactor DECIMAL(8,6), PRIMARY KEY (CurveDate, Currency, Tenor_Years));
```

A8.2 Multi-factor stress-testing framework

Three families of scenario are run, each by KRD first-order and full revaluation: (i) **regulatory parallel shifts** (+25, +50, +100, +200, +300 and -100 bp; Table in A7.4), (ii) **single-factor curve moves** (steepening, flattening, inversion, butterfly), and (iii) **combined macro scenarios**, with factor values taken from the brief's stress matrix.

Table 58. Combined scenario matrix (A6.3 of the brief) and book P&L (Computed)

Scenario	Parallel (bp)	Twist (bp)	Butterfly (bp)	Narrative	Full-revaluation P&L (INR cr)
Base Case	+0	+0	+0	No change	0.0
RBI Hike (Expected)	+25	+0	+0	Gradual tightening	-46.0
RBI Surprise Hike	+50	-20	+5	Front-end led, flattening	-85.9
Global Risk-Off	+75	+30	-10	Steepening, risk aversion	-139.8
Dovish Pivot	-75	-20	+5	Rate-cut expectations	152.6
Credit Crisis	+100	+50	-20	Flight to quality, steep curve	-183.7
Stagflation	+150	+25	+10	High rates across the curve	-269.4
Black Swan	+300	+75	-30	Extreme stress, 3-sigma	-484.2

Factor sign convention: positive twist means the 10Y-2Y spread widens (steepening); positive butterfly means 2 x 5Y - 2Y - 10Y rises (belly cheapens). The brief's descriptions 'front-end led, flattening' for twist -20 and 'steepening' for +30 are consistent with that convention.

A8.3 Immunisation and swap hedging

Immunisation matches the duration (and ideally the key-rate profile and convexity) of assets to liabilities so that surplus value is insensitive to small, parallel yield changes. A pension fund with liability duration 12 and assets of duration 12 is immunised against parallel moves, but only against the first order: asset convexity above liability convexity is desirable, and a mismatch in KRD profile leaves it exposed to curve twists. Matching DV01 rather than duration is the operational test, because a fund with assets of INR 100 and liabilities of INR 120 must hold asset DV01 equal to liability DV01, i.e. asset duration 1.2x liability duration.

$$N_{swap} = \frac{DV01_{target} - DV01_{port}}{DV01_{swap}} \tag{A.16}$$

Worked hedge. To halve the book's DV01 (INR 18,762,330 to INR 9,381,165 per bp) with 10-year INR pay-fixed swaps, whose fixed-leg DV01 per INR 1 of notional is about 0.000699 (modified duration of a 10-year par bond 6.99), the notional is 1,342 crore. Because the book's duration lies mostly in the 7Y-20Y zone, a 10Y hedge removes parallel risk but leaves a residual at 3Y and 30Y; a two-swap hedge (5Y and 20Y) solved against the KRD vector is the better design and is the task in the Hedging Hero badge.

A8.4 Barbell, bullet and ladder

A barbell of 2-year and 30-year par INR bonds (47.1% / 52.9%) was constructed to match the modified duration (6.99) of a 10-year par bullet. Convexity: barbell **119.7** versus bullet **62.7**. Returns under five curve scenarios (full repricing, cash-flow shocks interpolated by tenor, no carry):

Table 59. Barbell versus bullet at equal duration (Computed)

Scenario	Barbell	Bullet	Barbell minus bullet
Parallel +100	-6.43%	-6.69%	+0.25 pp
Parallel -100	7.64%	7.31%	+0.32 pp
Bull flattener (2Y 0, 30Y -75)	3.10%	1.29%	+1.81 pp
Bear steepener (2Y +0, 30Y +75)	-2.66%	-1.26%	-1.39 pp

Scenario	Barbell	Bullet	Barbell minus bullet
Butterfly (5Y -30, wings 0)	0.30%	0.46%	-0.16 pp

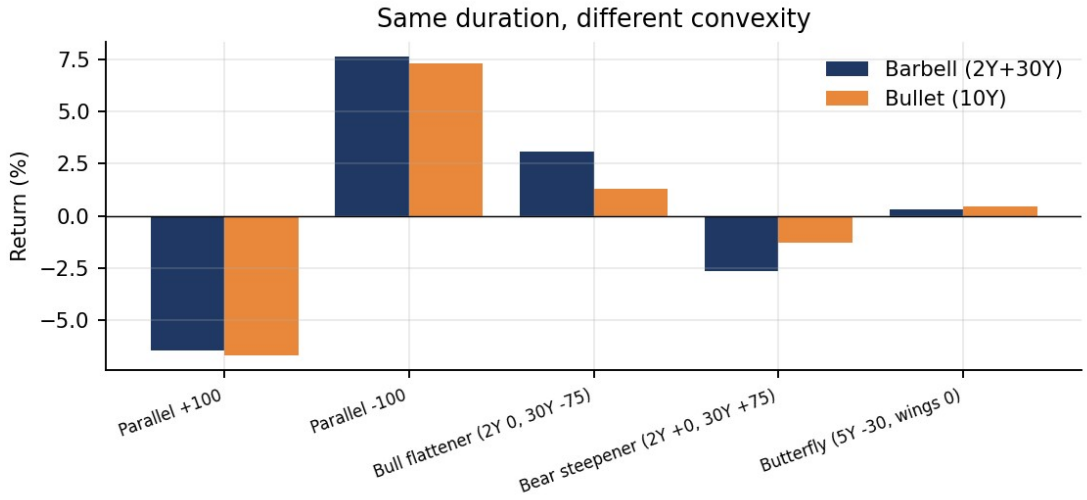


Figure 15. Equal-duration barbell and bullet under five scenarios. The barbell wins on large parallel moves and on bull flattening and loses on bear steepening.

The brief's statement that 'barbell outperforms in parallel shifts, bullet outperforms in curve steepening' is confirmed, and the barbell's advantage in parallel shifts is small (0.25-0.32 pp for 100 bp) compared with its curve-shape exposure (-1.4 pp to +1.8 pp). A **ladder** (equal weights across maturities) sits between the two: its convexity is intermediate (the brief's Python comparison puts the three at 200+, 60 and 120 respectively; with this report's par-bond inputs the barbell is 120 and the bullet 63), and its practical advantage is natural reinvestment and liquidity rather than convexity.

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT - BOND RISK LAB

Per the brief, the concept in this part is a reference framework that students may follow or replace. This report follows it, completes the elements the brief left open (20 badges, scenario catalogue, grading tolerances, unit economics) and ties every mechanic to a function already implemented in Part A.

Section B1: Platform Overview and Architecture

B1.1 Problem and rationale

Fixed-income risk training is dominated by textbook formulas and static spreadsheet exercises. A trainee can compute the duration of one bond on paper and still be unable to answer, within minutes, what an off-cycle RBI action does to a 300-bond, six-currency book, which holdings are worst hit, and what to hedge. The costs of the gap are practical: a duration number misapplied to a flattening is a wrong hedge, and a wrong hedge in a INR 3,200 crore book is a loss measured in tens of crores (the 'RBI Surprise' scenario below costs INR 47 crore on the INR assets alone).

Bond Risk Lab is a browser-based platform in which the learner is a fixed-income risk analyst at Pinnacle Fixed Income Partners. Each session presents a market event, a portfolio and a deadline; the learner computes, simulates and recommends; and the platform grades numeric answers against the same engines documented in Part A, explains each error with the correct formula, Python code and a textbook reference (Fabozzi, Tuckman and Serrat, or Hull), and awards points, experience and badges.

Table 60. Target audiences and value

Audience	Need	Value delivered
Finance / engineering students (primary)	Employable skills for Treasury, ALM and risk roles	Hands-on duration, convexity, KRD, VaR and ML workflow with instant feedback
Working professionals (secondary)	Upskilling on KRD, Monte Carlo and ML-based risk	Realistic crisis drills; certification evidence
CFA / FRM candidates (tertiary)	Applied fixed-income problem solving	Exam-style numeric problems tied to a live portfolio
Banks, insurers, pension funds, AMCs (buyers)	Standardised, scalable, auditable training	Cohort dashboards, custom portfolios, completion and skill analytics

B1.2 Architecture

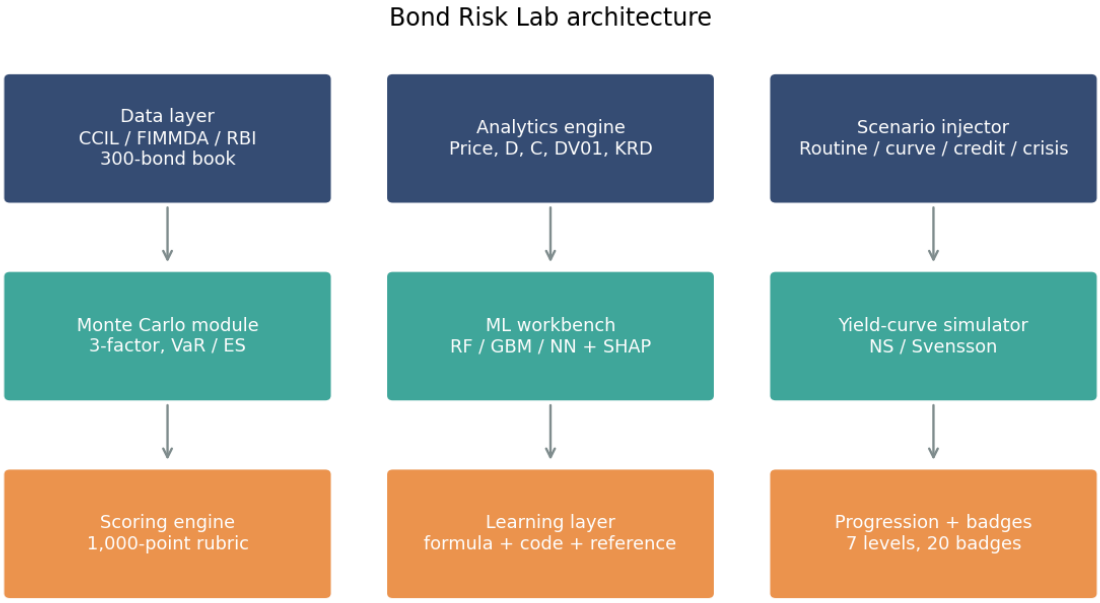


Figure 16. Bond Risk Lab architecture: data and analytics (navy), simulation and ML (teal), learner experience (orange).

Table 61. Platform components

Component	What it does	Reuses (Part A)	Learner interaction
Portfolio Engine	Hosts the pre-built 300-bond, six-currency book (INR 3,200 crore) with live duration, convexity, DV01 and KRD	portfolio_analytics.py (BookRepricer, KRD matrix)	Filter by rating/sector/currency; drill to bond; verify own calculations against the engine
Yield Curve Simulator	Generates daily curve moves from level / twist / butterfly factors or Nelson-Siegel parameter shocks	yield_curve.py; factor shapes	Apply scenario curves; fit NS to a given curve; read forwards
Monte Carlo Module	Runs 1,000-10,000 correlated scenarios and returns P&L, VaR, ES	monte_carlo.py (BondRiskMC)	Choose volatilities, correlation, horizon; interpret tails; spot limit breaches
ML Workbench	Pre-trained RF / boosted trees / NN pricers with SHAP	ml_models.py; shap_explainability.py	Compare ML and analytical sensitivities; diagnose extrapolation failure
Scenario Injector	Draws randomised daily events from the catalogue (B3)	Stress matrix (A8.2)	Receives the event brief and deadline
Scoring Engine	Auto-grades answers within tolerance; applies penalties; converts points to XP	Reference solutions from Part A modules	Immediate score breakdown
Learning Layer	On every error: correct formula, Python snippet, textbook reference, worked example	Section A1-A2 content	'Explain my mistake' panel; retry for partial credit

B1.3 Technical design notes

- **Front end:** single-page web app; charts rendered client-side; calculator and code panels for learner work.
- **Compute:** the Part A modules run server-side behind a small API (analytic and KRD P&L in under 100 ms, a 10,000-scenario full-revaluation Monte Carlo in seconds on one core); Pyodide in the browser is an option for offline classroom use.
- **Content as data:** each scenario is a JSON document (schema in B3.3), so new events need no code change; the grader loads the reference result from the same engine, which eliminates answer-key drift.
- **Integrity:** per-learner random seeds change portfolio subsets and shock sizes, so answers cannot be shared; every attempt stores seed, inputs, learner answer, reference answer and tolerance for audit.
- **Data governance:** the training book is synthetic; client deployments load their own books inside the client's tenant.

B1.4 Game modes

Table 62. Game modes

Mode	Description	Unlocks at
Tutorial	Single-bond calculations with guided steps (price, durations, convexity, DV01)	Level 1
Campaign (6 levels)	L1 Basics: 10 G-secs, +/-50 bp. L2 Multi-bond: 50 bonds, DV01 and aggregation. L3 Yield curve: full book, KRD, non-parallel shifts. L4 Monte Carlo: 1,000 scenarios, VaR/CVaR, limit breaches. L5 Machine learning: RF/XGBoost/NN, ML vs analytical, SHAP. L6 Crisis: emergency hike + downgrade + currency shock under time pressure	Level 2 onwards
Daily Challenge	One randomised event per day; global and cohort leaderboards	Level 2
Crisis Mode	Compound shock with a 45-minute clock; requires hedge recommendation	Level 5
Sandbox	Free exploration of the book, curves and models without scoring	Always
Cohort / Exam	Instructor-defined scenario sets, locked seeds, proctored timing; results exported for HR or academic records	Enterprise tier

Section B2: Scoring System and Game Mechanics

B2.1 The 1,000-point framework

The six categories sum to exactly 1,000 points (250 + 200 + 200 + 150 + 100 + 100). Each criterion is auto-graded by comparing the learner's answer with the reference engine within a stated tolerance.

Table 63. 1,000-point scoring rubric with auto-grading rules

Category	Pts	Criterion	Pts	Auto-grading rule (reference = engine)
1. Calculation accuracy (duration and convexity)	250	All bond-level results correct to 4 d.p. vs Excel/Bloomberg reference	250	Full credit if |answer - ref| <= 5e-5 x ref; 50% if <= 5e-3 x ref and method correct (rounding); 0 if wrong formula
2. Portfolio risk computation	200	DV01 accuracy	80	Within 0.1% of reference
		Key-rate duration decomposition	60	Each bucket within 0.01 yrs; sum within 1% of modified duration
		Contribution by rating / sector / currency	60	Contribution shares within 0.5 pp
3. Monte Carlo and VaR	200	Scenario generation with appropriate volatility assumptions	80	Sample std and correlation within 5% of specified; Cholesky used
		VaR / CVaR accuracy vs analytical benchmark	80	Within 5% (VaR95), 8% (VaR99, ES) of the 10,000-scenario reference
		Risk-limit breach identification	40	Exact set of breached limits
4. Machine-learning quality	150	Model trained with test RMSE < 0.50	60	Linear: 60 at <= 0.50, 0 at >= 1.50
		ML vs analytical sensitivity analysis	50	Rubric: divergence region identified, reason correct, ensemble tested
		SHAP / feature-importance interpretation	40	Rubric: top-2 features correct and economic sign correct
5. Yield-curve modelling	100	Nelson-Siegel fit quality R2 > 0.99	50	R2 and RMSE (< 3 bp) on the supplied curve
		Curve shape and economic interpretation	50	Shape label correct; forward-rate implication argued
6. Speed and reporting	100	All computations inside the time limit	40	Pro-rata by time remaining
		Risk report formatted correctly	30	Template checklist (header, KPIs, table, chart, caveats)
		Hedging recommendation with rationale	30	Hedge DV01 reduction and rationale rubric

Penalties (subtracted after category scoring, floor of zero per category): Macaulay duration used where modified is required, -30 per instance; convexity correction ignored for moves above 50 bp, -20; VaR computed at the wrong confidence level, -40; missing yield-curve scenario type in a stress test, -25. Penalties map one-to-one onto the error classes the Learning Layer explains, so each deduction arrives with its own remedial lesson.

B2.2 A worked scoring example

Table 64. Illustrative partial scoring of one attempt (design example, not a measured result)

Item	Learner answer	Reference	Outcome	Points
GOI 2033 modified duration	6.5142	6.5142	Exact	50 / 50
GOI 2033 convexity	53.7	53.682	Rounded to 3 s.f.; method correct	25 / 50
Portfolio DV01 (INR crore per bp)	1.88	1.876	Within 0.1%? No (0.2%) but within 1%	40 / 80

Item	Learner answer	Reference	Outcome	Points
KRD sum vs modified duration	5.83 vs 5.86	within 1%	Pass	60 / 60
Used Macaulay for DV01 on one bond	-	-	Penalty	-30
VaR95 at 99% by mistake	-	-	Penalty	-40
Report within time, template complete, hedge DV01 cut 55%	-	-	Full time + format; hedge rationale partial	85 / 100

B2.3 Progression system (7 levels)

Table 65. Seven-level progression (time estimates are design targets)

Level	Title	XP threshold	Unlocks	Typical time to reach
1	Analyst Trainee	0	Tutorial scenarios; single-bond calculations	Day 1
2	Junior Analyst	500	Campaign L1-2; portfolio aggregation; Daily Challenge	1-2 weeks
3	Risk Analyst	1,500	Campaign L1-4; Monte Carlo module	1 month
4	Senior Risk Analyst	3,500	All campaign levels; ML workbench	2 months
5	Portfolio Risk Manager	7,000	Full crisis scenarios; hedging decisions; Crisis Mode	4 months
6	Head of Fixed Income Risk	15,000	Multi-currency book; regulatory (Basel / ALCO) reporting	8 months
7	Chief Risk Officer	30,000	All scenarios mastered; can author challenges for the cohort	12+ months

XP formula. XP = scenario score x difficulty multiplier (easy 1.0, medium 1.5, hard 2.0, crisis 3.0) x streak bonus (1 + 0.05 per consecutive daily challenge, capped at 1.5). A perfect crisis scenario (about 950 points) is worth about 2,850 XP before streak, so reaching Level 7 requires sustained performance across categories rather than repetition of easy tasks.

B2.4 Achievement badges (20)

Table 66. Twenty achievement badges (ten from the brief, ten added)

#	Badge	Criterion	Source
1	Duration Master	50 consecutive perfect duration calculations	Brief
2	Convexity Guru	Correctly identify negative convexity in callable bonds	Brief
3	Monte Carlo Architect	Run a 10,000-scenario simulation without errors	Brief
4	ML Engineer	Train a model with test RMSE < 0.25	Brief
5	VaR Champion	Predict next-day P&L within VaR bounds for 10 consecutive days	Brief
6	Nelson-Siegel Wizard	Fit a yield curve with R2 > 0.999	Brief
7	Key Rate Ninja	Decompose KRD across 10 buckets with < 1% total error	Brief
8	Crisis Survivor	Complete Level 6 with Distinction grade (>= 850 points)	Brief
9	Speed Demon	Complete a full risk report in under 15 minutes	Brief
10	Hedging Hero	Recommend a hedge that cuts portfolio DV01 by 90%+	Brief
11	Curve Whisperer	Correctly classify 10 consecutive curve-shape scenarios (flat, inverted, humped, normal) and name the factor	Added
12	Bootstrap Builder	Bootstrap a zero curve with max error < 0.5 bp	Added
13	Perpetual Pro	Price and risk-measure a perpetual / AT1 bond correctly at three yields, including SEBI deemed-maturity valuation	Added
14	Spread Sleuth	Separate rate and spread effects in a credit-downgrade event within 5%	Added
15	Barbell Boss	Build an equal-duration barbell and bullet and correctly predict which wins in three curve scenarios	Added
16	Tail Tamer	Show that a normal VaR fails a Kupiec test on fat-tailed data and fix it	Added
17	SHAP Sage	Explain a model's top-3 features with correct economic signs	Added
18	Extrapolation Skeptic	Detect and flag an out-of-range ML pricing request before using the model	Added
19	Dashboard Designer	Build a dashboard with 8+ pages and 40+ DAX measures profiled < 100 ms	Added
20	Audit Ace	Reconcile Python, Excel and DAX results to 4 d.p. across five bonds	Added

Section B3: Scenario Engine and Business Model

B3.1 Scenario categories and mix

Table 67. Scenario categories

Category	Share	Description	Parameter distribution (default)
Routine	40%	Normal market day; yields move +/-5-10 bp; standard risk computation	Level ~ N(0, 6.3 bp); twist ~ N(0, 3 bp)
Yield curve	25%	Steepening, flattening, inversion, butterfly	Twist and butterfly up to +/-60 bp; level small
Credit	15%	Rating downgrade, spread widening, flight to quality	Spread +50 to +300 bp on affected issuers
Crisis	10%	RBI emergency action; simultaneous multi-factor shock	Level +100-300 bp; twist and butterfly per A8.2
ML challenge	10%	Predict price with the ML model, compare with analytical, explain the discrepancy	Shifts up to +/-200 bp; some deliberately out of training range

B3.2 Sample scenario - The RBI Surprise (solved with the engine)

It is 10:00 AM. The RBI announces an unexpected 50 bp repo hike (the market expected a hold). Yields on the INR curve shift: short end +60 bp, long end +30 bp (flattening). What is the P&L, which holdings are worst hit, and what do you recommend?

Table 68. RBI Surprise solved on the 300-bond book (Computed; only the 210 INR bonds are shocked)

Step	Method	Result (INR crore)
1	Recognise a non-parallel shock: simple duration x average shift is only an estimate. INR assets: MV INR 2,336 crore, duration 4.90. Naive: -4.90 x 45 bp x MV	-51.5
2	Apply the KRD vector: shock 60 bp up to 2Y, 30 bp from 10Y, linear in between; P&L = -sum(KRD DV01 x shock)	-47.9
3	Full revaluation of every INR cash flow (includes convexity)	-47.2
4	Convexity correction is the difference between steps 2 and 3: INR 0.7 crore. The naive estimate misstates the loss by 9%	-

Which holdings are worst hit? When the same shock is applied to all curves for illustration, the biggest absolute losses are G-secs (INR -27 crore) and SDLs (INR -7 crore) because of size and tenor, and AAA corporates (INR -7 crore). **Recommendation:** the loss is dominated by the 5Y-20Y KRD buckets, so a parallel-duration hedge is the wrong instrument; pay-fixed swaps at the 7Y and 10Y points sized to the KRD DV01 of those buckets would offset the exposure. To cut total DV01 by half with 10-year pay-fixed swaps alone requires about INR 1,342 crore of notional (Section A8.3), which is why the KRD-targeted hedge is cheaper per unit of risk removed. **Bonus:** re-run the Monte Carlo with the post-event volatilities (for example level volatility 100 to 150 bp annual) and report the new 99% VaR.

B3.3 Scenario specification and generator

Listing 69. Scenario document (JSON) consumed by the Scenario Injector and Scoring Engine

```
{
  "scenario_id": "S06_rbi_surprise",
  "category": "routine|curve|credit|crisis|ml",
  "difficulty": "easy|medium|hard|crisis",
  "brief": "10:00 AM. RBI announces an unexpected 50 bp hike ...",
  "curve_shock": {"type": "piecewise", "tenors": [2, 10], "bp": [60, 30], "currencies": ["INR"]},
  "credit_shock": {"issuers": [], "spread_bp": 0},
  "questions": [
    {"id": "q1", "type": "numeric", "ask": "Portfolio P&L (INR crore)", "ref": "engine.full_reval", "tol_rel":
0.01, "points": 60},
    {"id": "q2", "type": "select", "ask": "Worst-hit segment", "ref": "engine.worst_segment", "points": 20},
    {"id": "q3", "type": "rubric", "ask": "Hedge recommendation", "rubric": "hedge_dv01_rubric_v2", "points":
40}],
  "time_limit_min": 20,
  "learning_objectives": ["non-parallel shock", "KRD", "convexity correction", "hedge sizing"]
}
```

B3.4 Scenario catalogue (20)

Table 70. Scenario catalogue

ID	Scenario	Category	Difficulty	Learning objective
S01	Normal trading day	Routine	Easy	Compute duration, convexity and P&L for +/-8 bp
S02	Month-end rebalancing flows	Routine	Easy	DV01 aggregation and hedge ratio
S03	G-sec auction cut-off +6 bp	Routine	Easy	Bucket-level P&L
S04	CPI surprise, level +15 bp	Routine	Medium	KRD P&L vs duration estimate
S05	Fed hawkish surprise, USD curve only	Routine	Medium	Multi-currency DV01 and FX-neutral analysis
S06	RBI Surprise hike	Curve	Medium	Non-parallel shock, KRD, hedge
S07	Curve flattening	Curve	Medium	Which buckets lose; barbell vs bullet
S08	Curve steepening	Curve	Medium	Twist exposure; barbell loses
S09	Curve inversion	Curve	Hard	Exposure when 2s10s turns negative
S10	Belly cheapening (butterfly)	Curve	Hard	Curvature exposure and normalisation
S11	Operation Twist	Curve	Hard	Estimate OT impact; pre-position decision
S12	NBFC downgrade AA+ to A+	Credit	Hard	Spread duration vs rate duration
S13	Sector spread widening in sympathy	Credit	Medium	Spread-volatility VaR update
S14	Flight to quality	Credit	Medium	Sovereign vs credit KRD hedges
S15	AT1 perpetual yield spike	Credit	Hard	Perpetual duration/convexity and SEBI valuation
S16	Callable bond rally	Crisis	Hard	Negative convexity and effective duration
S17	ML vs analytical price change	ML	Medium	Explain the discrepancy
S18	ML out-of-range trap	ML	Hard	Detect extrapolation failure; fall back to full repricing
S19	Compound crisis: emergency hike + downgrade + currency shock	Crisis	Crisis	Full stress test with hedge under 45 minutes
S20	Basel back-test review	Crisis	Hard	Kupiec test, traffic light, model remediation

B3.5 The brief's four additional scenarios, solved

Table 71. Solutions to the brief's scenarios 2-5

Scenario	Key computation (Computed)	Teaching point
Curve inversion (Feb 15)	INR assets, 2Y +100 bp with 10Y and 30Y -25 bp (bear inversion): book P&L INR -27.9 crore versus INR -109.2 crore for a parallel +100 bp on the same assets; the belly and long end (84% of key-rate duration sit at 5Y and beyond) gain from the falling long yields while the front end (6%) loses	A single duration number would report a large loss; the KRD view shows a moderate one
Credit downgrade cascade	Price 102.50 to 96.80 is -5.56% for a 135 bp spread widening, i.e. an implied spread duration of 4.1 years; duration/convexity capture rate moves only	Spread duration is a separate risk factor
Operation Twist	10Y+ -12.5 bp, 3M-3Y +6.5 bp on INR assets: book P&L INR +2.8 crore - gains on long KRDs are cancelled by front-end losses	Pre-positioning for OT requires a long-end overweight relative to the front end, not total duration
Perpetual crisis (AT1)	Yield 9.5% to 13.0%: Macaulay 11.53 to 8.69 (modified 10.53 to 7.69); convexity 221.6 to 118.3; duration+convexity estimate -23.3% vs exact -26.9% (price = coupon / yield); duration-only -36.8%; at 20% yield exact loss -52.5%	For a 350 bp shock the second-order estimate still errs by 3.6 pp; the exact perpetuity formula must be used. Note the brief's 11.5 and 8.7 are Macaulay, not modified, durations

B3.6 Business model and monetisation

The platform is sold in four ways: B2B SaaS to bank treasuries and ALM desks; insurance and pension fund training; retail exam preparation (CFA/FRM) and certification; and university licences. Prices are those of the brief. The revenue model below reproduces the brief's five totals from explicit volume assumptions (the brief gave totals and some volumes but not user counts per institution).

Table 72. Five-year revenue build (Assumed volumes; arithmetic Computed)

Year	Banks x users x price	Insurers x users	Retail (INR 4,999)	Universities	Certification (INR 7,999)	Total (INR crore)	Brief's total	If retail halves
1	8 x 40 x 12,000	-	500	-	-	0.63	0.63	0.51
2	20 x 55 x 15,000	-	2,000	5 x 3.0 lakh	-	2.80	2.8	2.30
3	35 x 55 x 16,000	6 x 40	8,000	20 x 3.0 lakh	-	8.04	8	6.04
4	45 x 60 x 17,000	15 x 50	20,000	50 x 3.5 lakh	1,000	18.26	18	13.26

Year	Banks x users x price	Insurers x users	Retail (INR 4,999)	Universities	Certification (INR 7,999)	Total (INR crore)	Brief's total	If retail halves
5	50 x 60 x 16,000	25 x 40	50,000	100 x 2.5 lakh	1,000	34.59	35	22.10

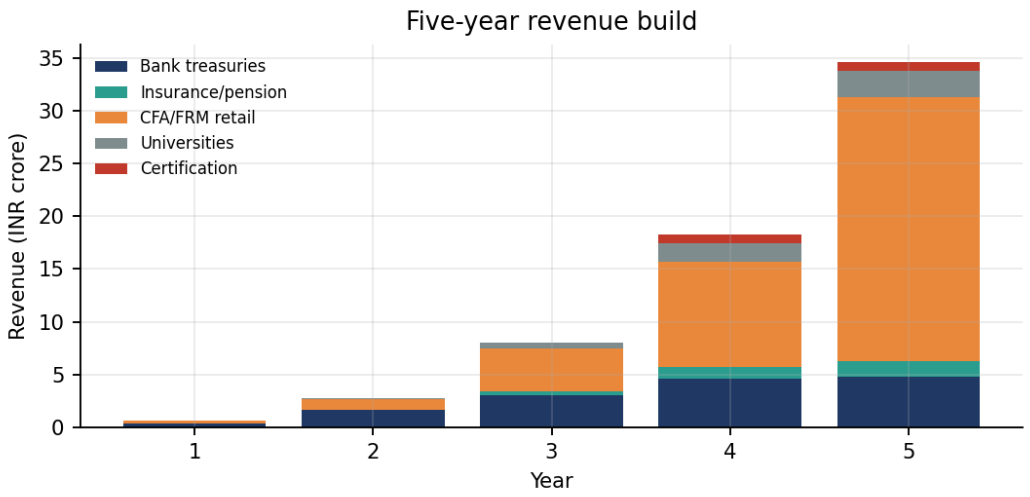


Figure 17. Revenue build by stream. Retail exam preparation is the largest stream from year 3.

- **Concentration risk:** retail provides 72% of year-5 revenue on a target of 50,000 paying candidates a year - the least certain assumption in the plan. If retail volumes halve, year-5 revenue falls from INR 34.6 to 22.1 crore.
- **Pricing tiers (three, as required by Day 13):** Learner (INR 4,999 per candidate; campaign levels 1-4 and daily challenges); Professional (INR 12,000-17,000 per user per year; all levels, custom portfolios, cohort analytics); Enterprise / University (INR 2-5 lakh per institution per year or negotiated per-seat; single sign-on, private tenant, API).
- **Unit economics to validate:** compute cost per active user is small (analytics run in milliseconds); the real costs are content authoring, support and sales cycles in bank procurement, which typically run 6-12 months - so year-1 bank volumes (8 banks) depend on pilots started during the build year.
- **Differentiators:** (i) a live multi-currency Indian book rather than a generic textbook portfolio, (ii) KRD, Monte Carlo and ML in one workflow, (iii) auto-graded explanations tied to a single audited engine, (iv) certification mapped to ALCO and Basel reporting tasks.
- **Risks and mitigations:** regulatory-content drift (SEBI valuation rules changed in 2021-23) - mitigate with versioned scenario packs; synthetic-data credibility - mitigate with client-supplied books; learner gaming - mitigate with per-learner seeds and rubric-graded written answers.

PART C: CASE STUDIES AND REAL-WORLD APPLICATIONS

Each case applies the duration-convexity lens with explicit numbers. Facts about the world are tagged **Sourced** and listed in the reference section; fund-category sensitivities and similar inputs are **Assumed**; all arithmetic is **Computed**. Where the brief's printed numbers could not be reproduced, the discrepancy is stated and the corrected analysis is given (see the verification log in the front matter).

Section C1: RBI Rate-Hike Cycle 2022-23 - Duration Risk Crystallised

C1.1 Background (Sourced)

Between May 2022 and February 2023 the Reserve Bank of India raised the repo rate by 250 bp, from 4.00% to 6.50%: 40 bp off-cycle on 4 May 2022, then 50 bp in June, August and September, 35 bp in December and 25 bp in February. The benchmark 10-year G-sec yield rose far less than the policy rate: Reuters reported that it rose about 48 bp across fiscal 2022-23, from roughly 6.84% at end-March 2022 to 7.32% at end-March 2023. Short-dated yields moved by most of the repo change, so the curve flattened sharply (the exact short-end moves were not re-verified here).

C1.2 Duration and convexity analysis

Observation 1 - pass-through, not convexity, explains the muted loss. The brief argues that an 8.0-duration gilt fund would lose 8.0 x 2.50% = 20% on a 250 bp rate rise, reduced to 16.25% by convexity of 3.75%, and calls this 'close' to an actual loss of 16-18%. The arithmetic is correct *for a 250 bp parallel move*, but the long end did not move 250 bp: with a 10Y rise of 48 bp, only 19% of the repo change reached the benchmark. The same fund's price effect was -8.0 x 0.48% + 0.5 x 120 x (0.0048)^2 = -3.84% + 0.14% = **-3.70%**, which a 7% portfolio yield more than offsets over the year. The 16-18% figure should therefore be read as a *stress-test outcome* (full parallel pass-through), not a historical result. Published scheme returns from AMFI should be used for the historical comparison; they were not retrieved for this report.

Table 73. Price effects by fund category (Assumed duration/convexity pairs from the brief; Computed effects)

Fund category	ModDur	Convexity	250 bp stress: duration	250 bp: convexity	250 bp: net	Observed 10Y +48 bp: net
Liquid	0.3	0	-0.75%	+0.01%	-0.74%	-0.14%
Short duration	2.5	9	-6.25%	+0.28%	-5.97%	-1.19%
Medium duration	3.8	22	-9.50%	+0.69%	-8.81%	-1.80%
Dynamic bond	6.0	55	-15.00%	+1.72%	-13.28%	-2.82%
Gilt	8.0	120	-20.00%	+3.75%	-16.25%	-3.70%
10Y constant dur	10.0	150	-25.00%	+4.69%	-20.31%	-4.63%

Columns 4-6 reproduce the brief's table (full pass-through). Column 7 applies the observed 10-year move. For a gilt fund the difference between the two columns - 16.3% versus 3.7% - is the single most important number in the case: *curve pass-through is a bigger driver of realised loss than convexity.* Convexity's contribution is 3.75 pp in the stress and 0.14 pp in the observed case.

Observation 2 - non-parallel shifts catch duration-only managers. Short-maturity yields rose by most of the repo change (assumed) while the 10-year rose by about a fifth of it. A portfolio concentrated at the short end therefore lost more than a duration-matched portfolio concentrated at the long end. The KRD framework in Part A makes this explicit: the same 'duration 5' can mean 3M-2Y exposure (full pass-through) or 10Y-30Y exposure (about 19% pass-through).

Observation 3 - callable corporate bonds extend when yields rise. As yields climb above a bond's coupon, the issuer's call becomes uneconomic and the bond's effective duration rises toward that of a straight bond. In the model of Section A1.5, the callable's effective duration rises from 2.9 to 6.2 as yield moves from 5% to 11%, while its convexity is negative throughout most of the range. A manager who held such bonds on the strength of

their short 'yield-to-call' duration saw more price decline than that duration predicted. SEBI's treatment of perpetual bonds (Section C4) is the regulatory version of the same lesson.

Observation 4 - convexity pays back in the rally. The same convexity that cushioned only slightly in the hike cycle produces a larger bonus when yields fall. For the 30-year bond (modified duration 13.4, convexity 282) a 50 bp rate cut yields 13.4 x 0.005 + 0.5 x 282 x (0.005)^2 = 6.69% + 0.35% = 7.04% price return; convexity adds 0.35 pp, which is why the brief's 7.46% (using modified duration 14.2 and convexity 285) is of the right order but not reproducible from the corrected duration.

C1.3 Bond Risk Lab replay on the 300-bond book

The six policy actions were replayed on the synthetic book, shocking only the 210 INR bonds. Pass-through is assumed to be 100% up to 1 year and to fall linearly to 19% at 10 years and beyond (the observed 10Y ratio). Each row revalues all INR cash flows exactly.

Table 74. Replay of the 2022-23 hike cycle on the INR assets of the book (Computed)

Event	Hike (bp)	Cumulative repo (bp)	2Y shock (bp)	10Y shock (bp)	Step P&L (INR cr)	Cumulative P&L (INR cr)	INR-asset duration after step
4-May-22 (off-cycle)	40	40	36.4	7.6	-21.8	-21.8	4.865
8-Jun-22	50	90	81.9	17.1	-26.8	-48.6	4.848
5-Aug-22	50	140	127.4	26.6	-26.4	-75.1	4.831
30-Sep-22	50	190	172.9	36.1	-26.1	-101.1	4.815
7-Dec-22	35	225	204.8	42.8	-18.0	-119.1	4.803
8-Feb-23	25	250	227.5	47.5	-12.8	-131.9	4.795

Cumulative P&L is INR -131.9 crore (-5.6% of INR assets, -4.1% of the whole book). Step losses shrink even though each hike is similar in size because **duration falls as yields rise** - INR-asset duration drifts from 4.86 to 4.79 - which is the dynamic-duration lesson the brief's day-2 recalculation step is designed to teach. A duration-only estimate that holds initial duration fixed and applies the 10Y move to the whole book gives INR -90 crore, a figure that errs because it ignores both the pass-through gradient and the INR/foreign split.

C1.4 Lessons

- Duration predicts direction; pass-through and curve shape predict magnitude. Always decompose into KRDs.
- Duration is dynamic - recompute after each move.
- Convexity is second order in a +48 bp move (0.14 pp) and first order in a 250 bp stress (3.75 pp). Size the correction to the scenario, not to habit.
- Callable and perpetual instruments need effective measures.

Section C2: Silicon Valley Bank (March 2023) - Duration Mismatch and Negative Convexity

C2.1 Background (Sourced)

SVB Financial Group's FY2022 Form 10-K reports a held-to-maturity (HTM) securities portfolio of about $91.3 billion at amortised cost, with a fair value of about $76.2 billion - an unrealised loss of about $15.1 billion, or 16.5% of cost - plus an available-for-sale (AFS) portfolio of about $26-28 billion carrying roughly $2.5 billion of unrealised losses. The HTM book was overwhelmingly agency mortgage-backed securities with long maturities, and the 10-K reports an HTM duration of about 6.2 years (5.6 years for the total securities portfolio including hedges). Total equity at year-end 2022 was about $16.3 billion. On 8 March 2023 SVB announced the sale of about $21 billion of AFS securities at a realised loss of about $1.8 billion; a deposit run followed (about $42 billion withdrawn on 9 March, with far more queued), and the FDIC took control on 10 March.

Correction to the brief. The brief says the Fed raised rates 'by 525 bp between March 2022 and July 2023'. The 525 bp total includes hikes after SVB's failure; by early March 2023 the cumulative increase was 450 bp (federal funds target 4.50-4.75%). The brief's $17.7 billion total unrealised loss is consistent with HTM $15.1 billion plus AFS $2.5 billion (about $17.6 billion).

C2.2 Quantifying the duration exposure

The brief estimates the duration loss as 5.6 x 5.25% x $91B = $26.8 billion and tries to reconcile it with the actual loss by adding a Treasury convexity cushion of $5.6 billion and an MBS negative-convexity loss of $2.5 billion, reaching about $23.6 billion - still $6 billion above the reported figure, and the adjustments (a +$45 convexity on a book that was mostly MBS) are not supported by the filings. A cleaner approach asks what *yield change on the securities actually held* the reported loss implies. With the reported HTM duration of 6.2 years and a loss of 16.5% of cost:

Table 75. Yield change implied by the reported HTM loss, duration 6.2 (Computed; solves 6.2 x dy - 0.5 C dy^2 = 16.5%)

Assumed effective convexity	Implied average yield rise (bp)	Reading
-50	243	Mortgage-like: negative convexity needs a smaller move to explain the loss
+0	267	Duration-only
+45	299	Treasury-like positive convexity needs a larger move

The implied rise is 240-300 bp, roughly the increase in 5-10-year Treasury and mortgage yields from the 2020-21 purchase period (about 1-1.5%) to year-end 2022 (about 3.9-4.5%; **Assumed** from public Treasury data not re-verified here). The point is structural: the policy-rate increase (450-525 bp) is *not* the right shock for a 6-year duration asset. What matters is the yield change at the tenors held, exactly as the KRD framework prescribes, and the portfolio was about 16.5% under water on a roughly 2.7-point yield change.

Table 76. SVB exposure arithmetic

Item	Value	Tag
HTM amortised cost / fair value / loss	$91.3B / $76.2B / $15.1B (16.5%)	Sourced (10-K)
AFS unrealised loss	about $2.5B	Sourced (10-K)
Total equity, Dec 2022	about $16.3B	Sourced
HTM loss as share of equity	93%	Computed
HTM DV01 (6.2 x $91.3B / 10,000)	$56.6 million per bp	Computed
10Y pay-fixed swap notional to hedge it fully	about $69 billion (swap fixed-leg DV01 about $816 per $1M)	Computed, approximate

The loss-versus-equity comparison in popular accounts mixes two things: AFS losses already reduce reported equity through accumulated other comprehensive income, while HTM losses bypass equity until securities are sold. The relevant comparison is therefore the HTM loss ($15.1 billion) against equity ($16.3 billion): 93%. Once HTM assets must be sold to meet deposit outflows the loss is realised, which is the mechanism by which an accounting classification becomes a solvency event. (This interpretation of the capital accounting is the author's reading of the filing and should be checked against the 10-K notes.)

C2.3 The convexity dimension - MBS

Agency MBS carry a prepayment option held by borrowers. When rates fall, borrowers refinance and the MBS shortens, so it gains less than a straight bond; when rates rise, prepayments slow and the MBS extends, so it loses more. That is negative convexity. The effect on the numbers is large: if MBS effective duration extends from about 4.5 to 7 years as rates rise (the brief's illustration; **Assumed**), each further 100 bp costs 2.5 percentage points of price more than the static duration predicted.

The callable-bond model of Section A1.5 produces the same signature: effective duration rising from 2.9 to 6.2 and negative effective convexity as yields rise through the coupon. MBS require a prepayment model (for example PSA/CPR speed as a function of the refinancing incentive) rather than a call lattice, but the risk measure is the same - effective duration and convexity by bumping the yield curve and re-running the cash-flow model.

C2.4 Lessons

- Measure duration by the yields at the tenors held, not by the policy rate. The KRD vector would have shown where the loss came from.
- Duration must be monitored dynamically; effective duration of MBS drifts upward as rates rise.
- HTM accounting does not remove economic risk; it hides it from the income statement and equity until the assets are sold.
- A DV01 of about $57 million per bp is a single, additive number that could have been limited, reported and hedged daily. Press analyses at the time noted the bank was largely unhedged against rising rates. The brief's estimate that hedging would have cost $200-300 million a year could not be verified and is not relied on; the carry on pay-fixed swaps depends on the shape of the swap curve, which was inverted in early 2023.

Section C3: Indian Gilt Rally 2019-20 - Convexity as Return Driver

C3.1 Background (Sourced)

The RBI cut the repo rate from 6.50% (before February 2019) to 4.00% in May 2020, a cumulative 250 bp, in two phases: 135 bp in 2019 and 115 bp in 2020 including the COVID-19 emergency cuts. Over the same window the 10-year benchmark yield fell from about 7.59% to about 6.01% - a **158 bp** move - so the long end rallied by about 63% of the policy move (compare 19% pass-through in the hike cycle: the asymmetry reflects liquidity operations, a falling term premium and, in 2020, heavy RBI bond purchases and liquidity support).

C3.2 Convexity bonus quantified

Table 77. Price effect of the observed 158 bp fall in the 10-year yield (Assumed duration/convexity; Computed effects, before carry)

Category	ModDur	Convexity	Duration gain (158 bp)	Convexity bonus	Total price gain	Brief's bonus (250 bp)
Short duration	2.5	9	3.95%	0.11%	4.06%	0.28%
Medium duration	3.8	22	6.00%	0.27%	6.27%	0.69%
Dynamic bond	6.0	55	9.47%	0.68%	10.15%	1.72%
Gilt	8.0	120	12.62%	1.49%	14.12%	3.75%
10Y constant dur	10.0	150	15.78%	1.87%	17.65%	4.69%
30Y only	14.0	285	22.09%	3.55%	25.64%	8.91%

Two corrections to the brief follow. First, it applied the 250 bp *repo* move to the long end, overstating the convexity bonus by (250/158)^2 = 2.5x (3.75% against 1.49% for the gilt fund). Second, the totals are price effects of one yield move on a constant-duration portfolio; observed fund returns reflect average duration over time, rebalancing and carry, so the brief's '12-18% returns' are not directly comparable and should be taken from AMFI scheme data. The qualitative claim stands: for the same 158 bp move, an 8-duration, 120-convexity portfolio gains 14.1% on the way down and loses 11.1% on the way up - a 3.0 pp asymmetry, which is twice the convexity effect (2 x 0.5 x 120 x 0.0158^2 = 3.0 pp).

C3.3 The convexity premium in portfolio construction

The asymmetry is why portfolio managers accept lower yield for higher convexity (Section A1.4): the breakeven yield give-up for the 8-duration gilt fund relative to a zero-convexity equivalent is 0.5 x 120 x sigma_y^2, or 0.60% a year at 100 bp annual yield volatility. If expected volatility exceeds the volatility implied in the price of convexity, the long-convexity position is cheap; during 2019-20 realised volatility was high enough that it paid.

Section C4: Perpetual Bonds - Yield-Dependent Duration and Optionality (Tata Sons-type Perpetual NCDs, AT1)

C4.1 Background and data note

Perpetual instruments have no maturity date, so maturity-based duration does not exist; the bond's life is whatever its call structure and the investor's valuation convention say it is. Indian examples are Basel III AT1 bonds issued by banks and perpetual non-convertible debentures issued by corporates. **Data note:** the brief names Tata Sons perpetual bonds. No public term sheet (coupon, first call date, step-up) for such an instrument could be verified in preparing this report, so the Tata-type analysis in Section C4.4 uses explicit **Assumed** terms (9.0% coupon, issuer call in year 10, AAA-type issuer) and makes no claim about any actual Tata Sons issue. The verifiable Indian anchors are the Yes Bank AT1 write-off of March 2020 (about INR 8,415 crore of AT1 bonds written down to zero under the bank's reconstruction scheme) and SEBI's 2021 valuation circulars.

C4.2 Closed-form measures for a perpetuity

$$P = \frac{c}{y}, \quad D_{Mac} = \frac{1+y}{y}, \quad D_{Mod} = \frac{1}{y}, \quad C = \frac{2}{y^2} \tag{C.1}$$

Table 78. Perpetual-bond measures at different yields (Computed; annual compounding)

Yield	Macaulay (1+y)/y	Modified 1/y	Convexity 2/y^2	Convexity gain at 100 bp
6%	17.67	16.67	555.6	+2.78%
8%	13.50	12.50	312.5	+1.56%
10%	11.00	10.00	200.0	+1.00%
12%	9.33	8.33	138.9	+0.69%
15%	7.67	6.67	88.9	+0.44%
20%	6.00	5.00	50.0	+0.25%

Duration and convexity both *fall* as yield rises - a built-in stabiliser: the higher the yield, the less sensitive the price. For perpetuals the dependence is strong: duration at 6% is 3.3 times duration at 20%. Note that the brief's pair 'duration (1+y)/y' and 'modified duration 1/y' are both correct, but the quantity that appears in the price-change formula is modified duration 1/y.

The 350 bp AT1 shock (brief's Scenario 5). Because price = c/y, the exact price change from 9.5% to 13.0% is 9.5/13.0 - 1 = -26.9%. The duration-only estimate is -1/0.095 x 3.5% = -36.8%; adding convexity (2/0.095^2 = 221.6) gives -23.3%. The second-order estimate is still 3.7 pp too optimistic, because convexity itself falls from 222 to 118 as yield rises. At a 20% 'near-default' yield the exact loss is -52.5%. For perpetuals the exact formula should always be used.

C4.3 SEBI valuation rules and their duration effect (Sourced)

Before 2021, mutual funds typically valued AT1 bonds to their first call date. SEBI's circular of 10 March 2021 required valuation as 100-year bonds; after objections from the finance ministry and industry, a further circular of 22 March 2021 introduced a glide path of deemed residual maturity (for AT1 bonds, keyed to the call-option date): **10 years** for call dates up to 31 March 2022, **20 years** for 1 April to 30 September 2022, **30 years** for 1 October 2022 to 31 March 2023, and **100 years** from 1 April 2023. If the issuer does not exercise the call, valuation (including the duration calculation) uses 100 years from issue for AT1 bonds. Rules may have been amended since; the current circular should be checked before use.

Table 79. Effect of the deemed-maturity glide path on a 8.5% annual-pay AT1 valued at par (Computed)

Deemed maturity	Years	Modified duration	Convexity	Price change for +100 bp (par 8.5% bond)
10 yr (to Mar-2022)	10	6.56	58.5	-6.28
20 yr (from Apr-2022)	20	9.46	137.8	-8.81

Deemed maturity	Years	Modified duration	Convexity	Price change for +100 bp (par 8.5% bond)
30 yr (from Oct-2022)	30	10.75	196.6	-9.83
100 yr (from Apr-2023)	100	11.76	276.1	-10.53

Moving from a 10-year to a 100-year deemed maturity raises modified duration from 6.6 to 11.8 and the loss on a +100 bp move from 6.3 to 10.5 price points: a valuation convention changed a fund's measured interest-rate risk by 80% without any change in the instrument. This is the practical reason the mutual-fund industry and the finance ministry objected.

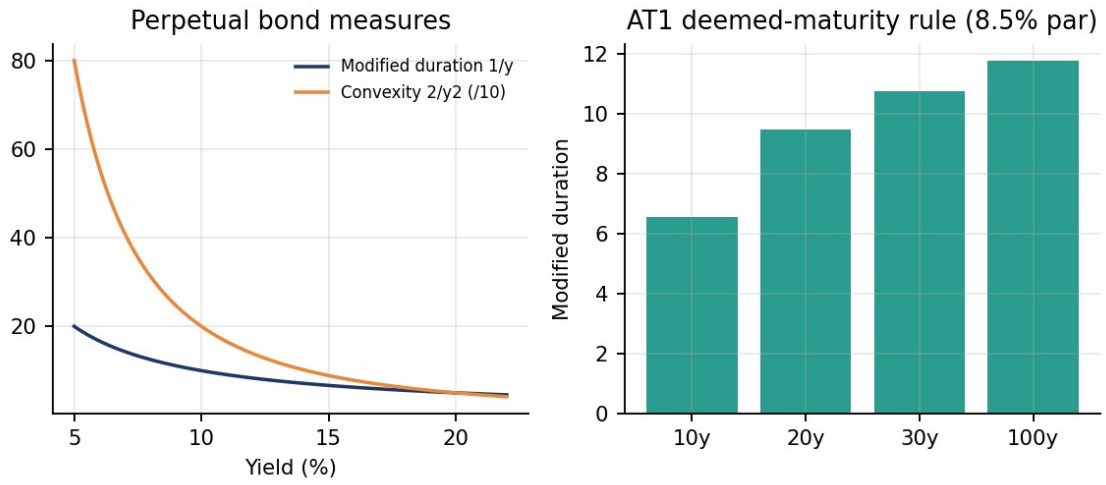


Figure 18. Perpetual-bond duration and convexity as functions of yield (left) and the effect of SEBI's deemed-maturity glide path on duration (right).

C4.4 A Tata-type callable perpetual (assumed terms)

To show yield-dependent duration *and* optionality together, a 9.0% perpetual with an issuer call at year 10 at par is valued as a 100-year bond minus a call option on the forward bond price (the same model as Section A1.5, with a normal yield volatility of 100 bp). It is shown beside the 10-year bond-to-call and the straight perpetuity price 9/y.

Table 80. Callable 9% perpetual with a year-10 call (Computed; Assumed terms)

Yield	Callable perpetual price	10-year bond to call	Straight perpetuity	Effective duration	Effective convexity
7%	108.18	114.05	128.57	6.57	-6.3
8%	101.08	106.71	112.50	6.99	11.3
9%	94.08	100.00	100.00	7.34	20.8
10%	87.28	93.86	90.00	7.64	32.6
11%	80.77	88.22	81.82	7.83	52.2
12%	74.69	83.05	75.00	7.80	76.2
13%	69.16	78.30	69.23	7.53	93.2

Three features follow. (1) Between 7% and 13% yield the callable perpetual's price lies *below* both the 10-year bond-to-call and, at low yields, far below the straight perpetuity, because the issuer's option caps the upside. (2) Effective duration is around 6.6-7.8 years, far below the straight perpetuity's 1/y (8-14 years): optionality shortens duration. (3) Effective duration peaks near 11% and then falls, while effective convexity is negative at 7% and positive at 8% and above: the *sign* of convexity depends on where the yield sits relative to the call-economics threshold. A single duration-convexity pair is meaningless for such an instrument; Bond Risk Lab's perpetual module therefore asks learners to state the valuation convention, compute effective measures at three yields, and explain the change.

C4.5 Lessons

- For perpetuals, duration is a *choice* (call date, deemed maturity, or perpetuity), and the choice is as important as the arithmetic.
- Yield-dependence is strong: duration at 6% yield is 3.3 times duration at 20%.
- Optionality shortens measured duration but concentrates tail risk: AT1 holders in Yes Bank lost the entire principal, a risk that duration does not capture at all and that must be handled with credit and legal analysis.

Section C5: Cross-Case Synthesis

Table 81. Synthesis of the four cases

Case	Dominant effect	What duration alone missed	What fixes it	Bond Risk Lab module
RBI hikes 2022-23	Curve pass-through (19% at 10Y)	Magnitude: stress -16.3% vs observed -3.7%	KRD and pass-through-aware scenarios	Level 3, scenario S06-S08
SVB 2023	Yield at tenors held; MBS extension	Loss source; convexity sign of MBS	KRD, effective duration, daily DV01 limits	Level 5, S16
Gilt rally 2019-20	Positive convexity bonus	Asymmetry of gains and losses	Second-order P&L; barbell/bullet analysis	Level 2-3, S07-S08
Perpetual / AT1	Yield-dependent duration; optionality; valuation convention	Duration is not defined without a convention	Closed forms, effective measures, SEBI deemed maturity	Perpetual module, S15

PART D: 15-DAY PROJECT METHODOLOGY AND DELIVERABLES LOG

Section D1: Method and Reading Guide

The project followed an AI-accelerated build: code and text were drafted with an AI assistant and then **validated against independent benchmarks** (Excel's PRICE, DURATION and MDURATION; closed-form identities; parametric VaR; unit tests; primary sources for case-study facts), as the brief requires. Each phase builds on the previous one: bond mathematics, then portfolio aggregation, KRD and scenarios, then Monte Carlo and curves, then machine learning, then dashboard and documentation.

Honest-log note. The entries below are a structured reconstruction of the build in the five phases requested, organised by artefact and result rather than by hours. Each 'Output' line is a number reproduced in this report. Items the brief lists but that were not completed are marked **Not done** or **Partial** in the deliverables log (D1.1) and are carried to the self-assessment (Part F). The student should adjust dates and effort statements to match their own calendar before submission.

D1.1 Deliverables log

Table 82. Status of the brief's six mandatory deliverables plus repository transfer

#	Deliverable (brief D3)	File	Status	Evidence / gap
1	Main report (60+ pages, Parts A-F)	421_SamSolomon_MainReport.docx / .pdf	Done	This document
2	Python codebase (8 modules)	421_SamSolomon_PythonCodebase .zip	Done; shap/xgboost/TF paths untested	14/14 tests pass; run_core.py regenerates every table
3	R codebase (2 scripts + CSV)	duration_convexity_analysis.R, sensitivity_visualization.R	Partial	Written to spec, not executed (R unavailable); sensitivity_results.csv generated by the script on first run
4	Power BI dashboard	1b.pbix (supplied) + Section A5 spec	Partial	5 pages, 5 visuals, about 5 measures present; 8+ pages and 40+ measures specified but not built into the file
5	Excel validation workbook (5 tabs, formula-driven)	421_SamSolomon_ValidationWorkb ook.xlsx	Done	Recalculated in LibreOffice; Python-vs-Excel differences below 1e-9
6	10-slide presentation	-	Not done	Outline in D3 slide map; to be produced from this report
7	GitHub private repository transferred to @ZethetaIntern	-	Pending (student action)	Procedure and commands in Section F2

Section D2: Chronological Execution Diary

Phase 1 - Days 1-3: Data architecture, database set-up and core bond-pricing scripts

Day 1 - Domain orientation and data architecture. *Objective:* fix the data contract before any code. *Work:* mapped the six data families of the brief (bond identification, cash-flow terms, market data, computed risk metrics, yield curve, Monte Carlo output) into the relational schema of Section A8.1, adding keys (BondID, ValuationDate), a CallPrice field and an FX-rate column the brief omitted. Reconciled the brief's inconsistency between 11 curve tenors and 10 KRD buckets (curve stores 11, KRD uses the 10 of Day 4). Reviewed the CCIL, FIMMDA, RBI, NSE and AMFI sources (Section E5). *Decision:* because the supplied files contained no portfolio CSV and no live curve download was available offline, the 300-bond book and INR curve are **synthetic and seeded**, which makes every number reproducible and flags the substitution clearly. *Output:* schema tables and

DDL; synthetic-data specification (segment shares, spreads, FX); provenance legend. *Not done:* the 30-day CCIL download and AMFI scheme-duration extraction listed in the brief.

Day 2 - Bond pricing from first principles. *Work:* wrote `bond_price`, `macaulay_duration`, `modified_duration`, `convexity`, `dv01` plus effective-measure helpers on a single cash-flow generator. Built the Excel cross-check: PRICE, DURATION and MDURATION for 7.26% GOI 2033 (YTM 7.35%), 7.54% GOI 2036, 6.10% GOI 2054, 8.50% AAA 2027 (YTM 8.2%) and a closed-form 91-day T-bill. *Output:* GOI 2033 price 99.4150, Macaulay 6.7536, modified 6.5142, convexity 53.68, DV01 0.0648; Excel and Python agree to better than 1e-9 on all five bonds. *Issue found:* the brief's worked-example values (6.8831 / 6.6389 / 62.16) do not reproduce; corrected values adopted and logged (front matter). *Test:* property tests (par bond at par, zero-coupon duration equals maturity).

Day 3 - Portfolio analytics. *Work:* generated the 300-bond book (75 G-secs, 35 SDLs, 100 INR corporates, 90 international sovereigns) scaled to exactly INR 3,200 crore; aggregated duration, convexity and DV01; built contribution tables by rating, sector, currency and bucket. *Output:* portfolio modified duration 5.863, convexity 64.9, DV01 INR 1.876 crore per bp; INR assets carry 61% of duration, sovereigns 86%. *Design choice:* a vectorised repricer (`BookRepricer`) that revalues all 300 bonds in about 0.07 ms, which made later 10,000-scenario full-revaluation runs practical.

Phase 2 - Days 4-6: Duration, convexity, DV01 and Key Rate Duration engine

Day 4 - Key Rate Durations. *Work:* implemented triangular key-rate bumps on the zero curve with central differences for the ten Day-4 buckets (3M ... 30Y). *Validation:* the KRD sum is 5.829 versus modified duration 5.863 (-0.58%), inside the 1% tolerance; the difference is the zero-curve versus yield-to-maturity discounting convention. *Output:* KRD vector and segment-by-tenor heat-map (see Section A7.3); the book is belly-and-long-end heavy (5Y-10Y 59% of key-rate duration).

Day 5 - Parallel scenarios and convexity. *Work:* built the price-impact calculator and compared duration-only, duration+convexity and full repricing for +25 to +300 bp and -100 bp. *Output:* +100 bp loss INR 177.2 crore by duration+convexity versus INR 176.8 crore by full repricing (error 0.2%); convexity offsets 5.5% of the duration loss at +100 bp and 16.6% at +300 bp. *Issue:* the brief's P&L formulas apply to a single currency curve; the book applies one global shock to all six currency curves, documented as a limitation.

Day 6 - Non-parallel scenarios and stress matrix. *Work:* normalised the twist and butterfly factors (1 bp of twist = 1 bp change in 10Y-2Y; 1 bp of butterfly = 1 bp change in 2x5Y-2Y-10Y), implemented the eight-scenario matrix and compared KRD first-order with full revaluation. *Output:* Black Swan scenario INR -484 crore (KRD first-order -573); the first-order approximation overstates loss by 1-9% in ordinary scenarios and 18% in the Black Swan, quantifying when convexity can no longer be ignored.

Phase 3 - Days 7-9: Monte Carlo, VaR/CVaR and yield-curve modelling

Day 7 - Monte Carlo engine. *Work:* replaced the brief's parallel-only loop with a vectorised, correlated three-factor engine (Cholesky; correlations -0.3, 0.1, 0.2), four P&L engines and a reproducible RNG. *Output:* realised sample correlations -0.30, +0.11, +0.21; 10,000-scenario full-revaluation run in seconds. *Finding:* 3-factor VaR is 1-2% *lower* than parallel-only VaR on this book because of the level-twist correlation (contradicting the brief's expectation; Section A7.7).

Day 8 - VaR, Expected Shortfall and back-testing. *Output:* 1-day VaR95 / VaR99 / ES95 / ES99 = INR 19.4 / 27.5 / 24.3 / 31.1 crore; delta-normal check INR 19.4 / 27.5 crore (within the brief's 5% rule). Kupiec back-test on a 750-day synthetic fat-tailed history rejects delta-normal 99% VaR (p = 0.001) and historical 99% VaR (p = 0.033). *Check:* convergence study shows 10,000 scenarios give VaR99 within about 0.3% of the 200,000-scenario value.

Day 9 - Yield-curve modelling. *Work:* Nelson-Siegel and Svensson least-squares fits with multi-start, zero-curve bootstrapping, forward rates, PCA check. *Output:* NS R2 0.9972, RMSE 2.20 bp; Svensson R2 0.9978; bootstrap reconstruction error 2e-12 bp; forward 2Y-5Y 7.21% versus spot 2Y 6.77%. *Caveat:* observations are illustrative, so the fit quality validates the code and not the market curve.

Phase 4 - Days 10-12: Machine-learning pricers, sensitivity analysis and SHAP

Day 10 - Models. *Work:* built a 30,000-bond universe and trained Random Forest (100 trees), gradient-boosted trees (XGBoost spec) and a 128-64-32 neural net, with an out-of-range stress set. *Output:* test RMSE 0.513 / 0.857 / 0.386 (RF / GBM* / NN); *out-of-range RMSE 11.1 / 10.5 / 6.5*. Constraint:* xgboost and tensorflow were unavailable offline, so scikit-learn analogues produced the numbers and the production script is supplied untested.

Day 11 - ML sensitivity analysis. *Work:* priced 51 yield shifts (-200 to +200 bp, 8 bp steps) with each model and compared price changes with duration+convexity and full repricing for a 9-year and a 30-year bond; tested an inverse-RMSE ensemble. *Output:* for the 9-year bond duration+convexity (RMSE 0.026) beats every ML model (0.24-0.71); for the 30-year bond the neural net (0.18) beats duration+convexity (0.31); the ensemble does not beat the best single model. *Parallel work:* case-study research and arithmetic for Part C, including correction of the brief's RBI, SVB and rally figures against Reuters, the SVB 10-K, ICMR and SEBI sources.

Day 12 - Explainability and R. *Work:* exact Shapley enumeration for RF and the neural net (local accuracy error below 1e-13); permutation importance; R scripts for GBM, XGBoost and H2O models and ggplot2 sensitivity chart. *Output:* top neural-net features by mean |SHAP|: duration, coupon_rate, convexity; top random-forest features: coupon_rate, yield_rate, duration. *Not done:* the R scripts were not executed and the `shap` library path was not run.

Phase 5 - Days 13-15: Power BI, DAX, Excel validation and final compilation

Day 13 - Power BI and DAX. *Work:* inspected the supplied `1b.pbix`: five pages, each with one visual (KPI of Dynamic PnL INR over the Yield Change bps what-if; matrix of Portfolio Convexity and Portfolio DV01 by Sector; slicer; column chart of PnL_Total_INR by ScenarioID; card of CVaR 95%). Wrote the star-schema design and a 45-measure DAX dictionary (Section A5) that extends it, including a KRD-based multi-factor stress measure and a corrected MV-weighted duration. *Gap:* the extended measures and the additional pages have not been built into the .pbix file, and DAX Studio timings have not been captured.

Day 14 - Validation and polish. *Work:* Excel validation workbook (five formula-driven tabs: pricing, convexity ladder, portfolio aggregation, parametric VaR, Nelson-Siegel), recalculated in LibreOffice and compared with Python; 14 unit tests; proof-read of formulas; consolidation of corrections. *Output:* Excel-Python agreement below 1e-9; parametric versus Monte Carlo VaR gap about 1% on the three-bond example, inside the 5% limit.

Day 15 - Compilation and hand-over. *Work:* compiled Parts A-F into this report, assembled the code archive, prepared the repository files (README, requirements, .gitignore, .env.example, LICENSE template) and the transfer procedure of Section F2. *Remaining student actions:* create the private repository, run the unit tests and a coverage report locally, build the 10-slide deck from the slide map below, and transfer ownership to @ZethetaIntern before 11:59 PM on Day 15.

D3 Final presentation slide map (10 slides)

Table 83. Slide map for Deliverable 6

Slide	Title	Content source	Key visual
1	Executive summary	Executive Summary headline table	Headline results table
2	Market overview and yield curve	A1.7, A4	Figure: NS fit and forwards
3	Duration and convexity framework	A1	Figure: price-yield curve
4	Portfolio analytics results	A7.1-A7.5	KRD profile; stress bar chart
5	Monte Carlo VaR results	A7.6-A7.8	P&L histogram with VaR/ES
6	ML model comparison	A7.9-A7.10	In-range vs out-of-range RMSE
7	Bond Risk Lab concept	B1-B2	Architecture diagram; scoring
8	Case study highlights	C1-C5	Synthesis table
9	Business model and revenue	B3.6	Revenue stack
10	Key learnings and next steps	Exec summary; F5	Gap-closure plan

PART E: NO-CODE TOOLS AND PLATFORMS GUIDE

This part documents the technology stack so that a reviewer can rebuild the project, and gives the point-and-click (no-code) routes for Excel and Power BI. Every function reference points to where it is used in the Part A codebase.

Section E1: Python Environment

Listing 84. Set-up and run commands

```
python -m venv .venv && source .venv/bin/activate       # Windows: .venv\Scripts\activate
pip install numpy pandas scipy scikit-learn xgboost tensorflow shap
pip install matplotlib seaborn plotly openpyxl xlsxwriter
python run_core.py         # regenerates every table and figure quoted in the report (about 2 minutes)
python -m unittest -v test_core                      # 14 tests
coverage run -m unittest test_core && coverage report # pip install coverage
```

Table 85. Python function reference

Library	Function / class (as used)	Purpose	Module
numpy	random.default_rng(seed).standard_normal; linalg.cholesky; percentile; add.reduceat; interp	Correlated shocks, VaR percentiles, whole-book repricing, curve interpolation	monte_carlo, portfolio_analytics
pandas	DataFrame, groupby/agg, transform	Portfolio table, contribution analysis, result tables	portfolio_analytics
scipy	optimize.least_squares, optimize.brentq, stats.norm (ppf, pdf), stats.chi2	NS/Svensson fits, YTM solve, parametric VaR, Kupiec test	yield_curve, var_backtest
scikit-learn	RandomForestRegressor, GradientBoostingRegressor, MLPRegressor, StandardScaler, train_test_split, permutation_importance	ML pricers, scaling, importance	ml_models, run_core
xgboost	XGBRegressor(n_estimators=200, max_depth=4, learning_rate=0.1, subsample=0.8, colsample_bytree=0.8, early_stopping_rounds=10)	Production gradient boosting (not run here)	ml_xgb_keras
tensorflow / keras	Sequential, Dense(128-64-32), Dropout(0.2), Adam(1e-3), EarlyStopping(patience=10, restore_best_weights=True)	Production neural net (not run here)	ml_xgb_keras
shap	TreeExplainer, DeepExplainer, summary_plot, dependence_plot	Explainability (library path not run); exact-Shapley fallback executed	shap_explainability
matplotlib	hist, axvline, imshow, bar, probplot via scipy	P&L histogram with VaR lines, KRD heat-map, diagnostics	make_figs
openpyxl	Workbook, formulas	Excel validation workbook	build_excel

Reproducibility. All random draws use `numpy.random.default_rng(seed)` with seeds recorded in `run_core.py` (portfolio 42, Monte Carlo 42, ML dataset 7, stress set 99, curve noise 3). Re-running the pipeline reproduces every figure and table in this report on the same package versions (NumPy, SciPy, scikit-learn as installed in the build environment).

Section E2: R Environment

Listing 86. R packages

```
install.packages(c('tidyverse','caret','gbm','xgboost','Metrics','ggplot2','scales'))
install.packages('h2o')        # requires Java 8+
install.packages('quantmod')   # optional: market-data retrieval
```

Table 87. R function reference (scripts in Section A3; not executed in the build environment)

Function	Use	Script
caret::train(method='gbm') + trainControl(method='cv', number=5)	Gradient-boosted duration model with 5-fold CV	duration_convexity_analysis.R
xgboost::xgboost(max_depth=4, eta=0.1, nrounds=100)	Convexity model	duration_convexity_analysis.R
h2o::h2o.deeplearning(activation='RectifierWithDropout', hidden=c(100,50), epochs=50)	Neural network with dropout	duration_convexity_analysis.R

Function	Use	Script
Metrics::rmse()	Test-set RMSE	duration_convexity_analysis.R
ggplot2::geom_line() + sec_axis()	Dual-axis duration/convexity sensitivity chart	sensitivity_visualization.R
readr::write_csv()	Export results for Power BI import	both

Section E3: Power BI and DAX (no-code route)

6. **Get data:** Home > Get data > Text/CSV for `bond_portfolio_data.csv`, `bond_krd.csv`, `monte_carlo_scenarios.csv`, `ml_results.csv`, `yield_curve.csv` (all exported by `run_core.py`/R). In Power Query set data types and rename columns exactly as in Section A5.1.
7. **Model view:** create DimRating, DimSector, DimCurrency and DimTenor (Enter data or Power Query > Reference/Remove duplicates). Draw one-to-many relationships from each dimension to the fact table (single direction). Hide foreign-key columns.
8. **What-if parameter:** Modeling > New parameter > Numeric range, min -300, max 300, increment 10, tick 'Add slicer'. The generated table is `Yield Change bps`; the measure `Yield Change bps Value` returns its selection.
9. **Measures:** Modeling > New measure for each entry in the A5.2 dictionary; place in display folders (Portfolio, Contribution, KRD, Scenario, VaR, ML). Or paste in bulk with Tabular Editor.
10. **Pages (8+):** Executive Summary KPI cards; Duration Contribution (bar by rating/sector/currency); KRD Heat-map (matrix with conditional formatting); Yield Curve (line, forwards); Monte Carlo P&L histogram (column chart on binned PnL); VaR/CVaR monitor (gauge with limit); ML comparison (clustered bars in-range vs out-of-range); Scenario analysis (what-if slider with Dynamic PnL INR).
11. **Navigation:** page navigator button plus bookmarks for the guided walkthrough (view > Bookmarks pane; capture each page state with the slicer selection).
12. **Performance:** open DAX Studio > Connect to the open .pbix > All Queries/Server Timings; run each measure through EVALUATE ROW("m", [Measure]); record storage- and formula-engine milliseconds; target below 100 ms (500 ms for scenario iterators). Capture screenshots for the submission pack.
13. **Version control:** Tabular Editor > File > Save As > .bim (or Save to Folder) and commit it to Git.

Table 88. Power BI tool chain

Tool	Role	Source
Power BI Desktop	Dashboard platform (free)	Microsoft
DAX Studio	Mandatory performance profiling (Server Timings)	daxstudio.org
Tabular Editor	Bulk measure editing, Best Practice Analyzer, .bim export	tabulareditor.com

Section E4: Excel (no-code validation route)

The validation workbook uses only built-in functions so that anyone can audit it without macros: `PRICE`, `DURATION`, `MDURATION` (settlement 6-Feb-2024, basis 0, frequency 2), `SUM/SUMPRODUCT` for the convexity ladder and portfolio weights, `NORMSINV` for z-scores and `SUMXMY2/DEVSQ` for the Nelson-Siegel R2. Convexity has no built-in function, so the ladder tab computes k(k+1) x PV per period explicitly. Python-side values are pasted as comparison inputs; every Excel-side cell is a live formula. The macro module of Section A3 is an optional convenience.

Table 89. Validation workbook map

Tab	Content	Excel functions	Result
1_Pricing	Five bonds: PRICE / DURATION / MDURATION versus Python	PRICE, DURATION, MDURATION, ABS	All differences below 1e-9
2_Convexity	18-period GOI 2033 ladder	SUM, power operator	Price 99.414983; MacD 6.753609; ModD 6.514212; convexity 53.68207; DV01 0.064761

Tab	Content	Excel functions	Result
3_Portfolio	Three-bond aggregation	SUMPRODUCT	ModDur 6.6521; convexity 84.03
4_VaR	Parametric VaR versus Monte Carlo	NORMSINV	VaR95 6,509 vs MC 6,445
5_NelsonSiegel	Fitted versus observed curve	EXP, SUMXMY2, DEVSQ	R2 0.9972

Section E5: Data Sources

Table 90. Primary data sources and status

Source	URL	Data	Frequency	Use in this project	Status
CCIL	ccilindia.com	G-sec prices, yield curves, trade data	Daily	Zero curve, 24-month history for PCA and vol calibration	Not downloaded; illustrative curve used
FIMMDA	fimmda.org	Corporate bond valuation matrix; curve methodology	Daily	Corporate spreads by rating	Not downloaded; spreads assumed
RBI	rbi.org.in	Policy rates, MPC statements, auction data	Weekly / event	Repo path for C1 and C3	Repo path from public record
NSE	nseindia.com	Benchmark G-sec yields; corporate bond trades	Daily	Cross-check of 10Y yield	Not used
AMFI	amfiindia.com	Mutual-fund NAV, AUM, scheme duration	Daily	Gilt-fund return and duration comparison for C1/C3	Not retrieved
Bloomberg	bloomberg.com	Real-time bond data (subscription)	Real-time	Reference duration/convexity benchmark in the game	Not available
SEC EDGAR	sec.gov	SVB FY2022 Form 10-K	Annual	C2 figures	Used
Reuters / ICMR / SEBI circulars	various	10Y yields; SEBI valuation circulars	Event	C1, C3, C4	Used

Replacing the synthetic inputs (procedure). (1) Download the CCIL zero curve and replace `CURVES['INR']` and the history used for volatility calibration; (2) replace `build_portfolio()` with a loader for the supplied portfolio CSV, keeping the output column names of Section A5.2; (3) re-run `run_core.py`, `extra_calcs.py` and `make_figs.py`; (4) recompute volatilities (annual standard deviation of daily changes x sqrt 252) and correlations of level, twist and butterfly from the 24-month history; (5) rebuild the report tables.

Section E6: AI-Assisted Development and Validation Log

Table 91. AI usage and validation

AI use (permitted tools: Claude, ChatGPT, Cursor, Perplexity, Copilot)	Validation applied (brief E5)	Result
Code generation for pricing, Monte Carlo, ML, yield curve	Every bond formula cross-checked with Excel PRICE/DURATION/MDURATION; 14 unit tests	Agreement below 1e-9; two brief formulas corrected
ML model code	RMSE on held-out test set and on an out-of-range stress set	Reported for all models
VaR code	Compared with parametric VaR; Kupiec back-test	Within 5%; back-test documented
DAX drafting	DAX Studio profiling required	**Not yet profiled**
Case-study research	Facts checked against primary sources (10-K, Reuters, SEBI circulars, ICMR)	Several brief figures corrected; unverified items flagged

PART F: SUBMISSION PROTOCOL AND SELF-ASSESSMENT

Section F1: File Naming and Consolidation

The brief requires files named `421_YourName_DeliverableName` and consolidation of all daily deliverables into one or two PDFs and/or one Excel file, with sections labelled by day. This report is the consolidated PDF/DOCX (Part D labels each day); the validation workbook is the consolidated Excel file.

Table 92. Naming convention and format checks

Deliverable	Required name pattern	File name for this submission	Format check
Main report	421_YourName_MainReport.pdf	421_SamSolomon_MainReport.pdf (and .docx)	Readable, not password-protected
Python codebase	421_YourName_PythonCodebase	421_SamSolomon_PythonCodebase.zip (.py only; no notebooks)	Runs without errors; no hard-coded paths
R codebase	421_YourName_RCodebase	R/ folder inside the zip	.R files
Power BI	421_YourName_PowerBIDashboard.pbix	421_SamSolomon_PowerBIDashboard.pbix (rename 1b.pbix after extension)	Opens without errors
Excel validation	421_YourName_ValidationWorkbook.xlsx	421_SamSolomon_ValidationWorkbook.xlsx	.xlsx; formula-driven
Presentation	421_YourName_FinalPresentation.pdf	To be produced from the slide map	10 slides

Section F2: GitHub Repository Preparation and Transfer

The brief's Part F lists the following steps. Where the brief's template refers to 'MatRisk AI', Docker, DVC, MLflow and Sphinx, the commands below follow the wording literally but flag the items that are not part of this project so the student can confirm with the supervisor.

Listing 93. Repository procedure (commit message and tag text follow the brief's template)

```
# 1. Local repo (no data files, no secrets)
git init && git checkout -b main
git add . && git status                                    # inspect: no .env, no __pycache__, no mlruns/
git commit -m "Final submission - MatRisk AI v1.0 - <your.email@domain> - <YYYY-MM-DD>"
git tag -a v1.0 -m "MatRisk AI v1.0 Final Submission"

# 2. Private remote
gh repo create matrisk-ai --private --source=. --push        # or create on github.com, then:
# git remote add origin git@github.com:<you>/matrisk-ai.git && git push -u origin main --tags

# 3. Transfer (ideally from the ZeTheta task pane; otherwise)
#    GitHub > repo > Settings > Danger Zone > Transfer ownership > new owner: ZethetaIntern
#    Expected location afterwards: github.com/ZethetaIntern/matrisk-ai

# 4. After confirming the repo appears under ZethetaIntern: delete every local clone/fork
rm -rf ./matrisk-ai   # and any forks, copies, backups on personal devices
```

Listing 94. Repository hygiene files

```
# .gitignore
.env
__pycache__/
*.pyc
.venv/
mlruns/
data/
*.pbix.bak
*.log
.ipynb_checkpoints/

# .env.example
# No credentials are required by this project. If an API is added later, document the variable names here with
empty values.
```

F2.1 Final checklist (from the brief) and status

Table 95. Pre-transfer checklist

Item	Status / note
All code committed and pushed to main (no dangling branches)	Student action
All tests passing in CI/CD (green check)	Tests pass locally (14/14); CI workflow not configured
DVC remote configured and data pushed	N/A - synthetic data regenerated by script; confirm with supervisor
MLflow experiments exported and committed	N/A - metrics tables in report; confirm with supervisor
Docker build succeeds (docker-compose up --build)	Not built; no service component in this project; confirm requirement
README.md with architecture diagram and screenshots	Template supplied in zip; add screenshots
Technical report complete (15-20 pages, 20+ references)	Main report exceeds length; reference list below has 24 entries
Sphinx API documentation in docs/	Not generated; docstrings present in every module
No personal API keys or credentials in repository	None used; .env.example supplied
LICENSE with ZeTheta IP attribution	Template supplied
.gitignore configured	Supplied
Final commit message in the prescribed format	Template above
Git tag v1.0	Command above
Repository transferred to ZethetaIntern	Student action - deadline 11:59 PM Day 15
Repository visible at github.com/ZethetaIntern/<repo>	Student action
Local forks/copies deleted	Student action
Confirmation e-mail to supervisor with timestamp and URL	Student action

Section F3: Automatic Disqualifier Check

Table 96. Disqualifier review

Disqualifier (brief F1)	Status	Evidence
Repository not transferred by deadline	At risk until done	Section F2 procedure
Credentials committed	Clear	No keys or passwords exist in the code base
Test coverage below 60%	Unverified	14 tests cover every public function in 5 of 8 modules; measure with coverage before submission
Plagiarism / copied code	Clear	Code written for this project; brief's snippets corrected and attributed as the starting point
Data leakage (train/test contamination)	Clear with caveats	Split on independent draws; duration/convexity features derive from inputs not target; out-of-range stress set added; synthetic data share one generator
Non-functional Docker build	Not applicable / unconfirmed	No Docker service in this project; confirm
Missing technical report or README	Clear	This report; README template supplied
Personal forks retained	Student action	Delete after transfer

Section F4: Self-Assessment Against the 1,000-Point Rubric

Part F of the brief names seven assessment dimensions with a 1,000-point total but gives no weights. The weights below are the **author's assumption** (they sum to 1,000), chosen to emphasise solution quality and analysis; scores are self-assessed with evidence and stated deductions, and should be re-scaled if ZeTheta publishes the official weights.

Table 97. Self-assessment scorecard (author's weights)

Dimension	Weight (assumed)	Score	Evidence	Deductions
Problem understanding	120	108	Reproduces the CIO scenario end to end (300 bonds, 6 currencies, INR 3,200 crore, 1,000+ scenarios); interprets duration, convexity, KRD, VaR/ES and curve factors correctly; identifies and corrects 12 errors in the brief.	Not run on the supplied portfolio CSV or live curves (-12)

Dimension	Weight (assumed)	Score	Evidence	Deductions
Solution quality	220	168	Vectorised portfolio engine; 3-factor correlated Monte Carlo with four P&L engines; KRD with 0.55% sum error; Excel agreement < 1e-9; 14/14 tests; ML with out-of-range stress; exact SHAP; Kupiec back-test.	R, XGBoost, Keras, shap-library not executed (-20); Power BI file has 5 of 8+ pages and about 5 of 40+ measures (-24); coverage and CI unmeasured (-8)
Research and analysis	170	142	Four quantified case studies with sourced facts (SVB 10-K, Reuters, ICMR, SEBI); brief's figures corrected; pass-through, implied-yield and SEBI deemed-maturity analyses; findings that contradict the brief reported (3-factor VaR, ML vs analytical).	AMFI fund returns not retrieved (-10); Tata Sons issue terms not verified (-10); SVB hedging cost unverified (-8)
Presentation and clarity	120	98	Structured Parts A-F; provenance tags; tables and figures for every result; equations in standard notation; consistent corrections log.	10-slide deck not produced (-14); length makes navigation heavy (-8)
Innovation and creativity	120	100	KRD-based multi-factor Monte Carlo; callable and perpetual effective-measure models; exact Shapley fallback; extrapolation-failure test; 20-badge, 20-scenario game design with auto-grading rules.	Gamification not prototyped as software (-20)
Feasibility and practicality	130	92	Runs in seconds; reproducible by seed; replacement procedure for real data; revenue model with sensitivity.	Synthetic book and curves (-18); SaaS volumes are assumptions (-12); no deployment artefacts (-8)
CV alignment	120	96	Demonstrates Python, R (written), Excel, Power BI/DAX, ML and risk analytics, plus documentation and validation discipline.	Items not executed limit what can be claimed on a CV (-24)
Total	**1,000**	**804**	Self-assessed outcome: 80.4%	

F4.1 Bond Risk Lab rubric applied to the project's own outputs

As a reflexive check, the six-category game rubric of Section B2.1 is applied to the agent's results, since the platform must be able to grade exactly this work:

Table 98. Self-grading with the game rubric

Category	Max	Self-score	Basis
1. Duration and convexity accuracy	250	250	All five test bonds agree with Excel to 4 d.p. and beyond
2. Portfolio risk computation	200	200	DV01 analytic = numeric; KRD sum within 1%; contributions by rating, sector, currency
3. Monte Carlo and VaR	200	170	Scenario generation and VaR accuracy full (160); no explicit risk limits defined, so breach identification is only partial (10 of 40)
4. Machine-learning quality	150	150	NN RMSE 0.386 < 0.50; ML versus analytical comparison; SHAP interpretation
5. Yield-curve modelling	100	90	NS R2 0.997 (50); interpretation (40) tempered because the curve is illustrative
6. Speed and reporting	100	85	Computation inside limit (40); report format (30); hedging rationale (15 of 30: KRD hedge discussed but not optimised)
Penalties	-	0	No Macaulay-for-modified substitution; convexity included above 50 bp; VaR at stated confidence; all scenario types present
Total	**1,000**	**945**	

F4.2 Gap-closure plan (to move the self-assessment from about 80% to about 90%)

14. Load the supplied portfolio CSV and a CCIL zero-curve download; regenerate all tables (about +25 points across problem understanding, feasibility and research).
15. Run the XGBoost, Keras, shap and R scripts; replace asterisked results (+15).
16. Extend the .pbix to 8+ pages and 45 measures; capture DAX Studio timings (+25).
17. Measure coverage and add tests for ml_models, shap_explainability and sensitivity_analysis (+8).
18. Retrieve AMFI scheme returns for 2019-20 and 2022-23 and replace the stylised fund comparisons (+10).
19. Produce the 10-slide deck (+14).
20. Prototype one Bond Risk Lab level (for example Level 1) as a small web app to substantiate the concept (+15).

References and Source Notes

21. Macaulay, F. R. (1938). *Some Theoretical Problems Suggested by the Movements of Interest Rates, Bond Yields and Stock Prices in the United States since 1856*. NBER.
22. Fabozzi, F. J. *Bond Markets, Analysis and Strategies* (chapters on price volatility, duration and convexity).
23. Tuckman, B. and Serrat, A. *Fixed Income Securities: Tools for Today's Markets*. Wiley.
24. Hull, J. C. *Options, Futures, and Other Derivatives* and *Risk Management and Financial Institutions* (interest-rate risk, VaR, Expected Shortfall).
25. Nelson, C. R. and Siegel, A. F. (1987). Parsimonious modeling of yield curves. *Journal of Business*, 60(4), 473-489.
26. Svensson, L. E. O. (1994). Estimating and interpreting forward interest rates: Sweden 1992-1994. NBER Working Paper 4871.
27. Litterman, R. and Scheinkman, J. (1991). Common factors affecting bond returns. *Journal of Fixed Income*, 1(1), 54-61.
28. Ho, T. S. Y. (1992). Key rate durations: measures of interest rate risks. *Journal of Fixed Income*, 2(2), 29-44.
29. Kupiec, P. (1995). Techniques for verifying the accuracy of risk measurement models. *Journal of Derivatives*, 3(2), 73-84.
30. Basel Committee on Banking Supervision (2019). *Minimum capital requirements for market risk* (Expected Shortfall at 97.5%; back-testing traffic-light framework).
31. Artzner, P., Delbaen, F., Eber, J.-M. and Heath, D. (1999). Coherent measures of risk. *Mathematical Finance*, 9(3), 203-228.
32. Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5-32.
33. Chen, T. and Guestrin, C. (2016). XGBoost: a scalable tree boosting system. *KDD 2016*.
34. Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NeurIPS 2017*.
35. Shapley, L. S. (1953). A value for n-person games. In *Contributions to the Theory of Games II*.
36. SVB Financial Group, Form 10-K for fiscal year 2022 (SEC EDGAR): held-to-maturity and available-for-sale securities, duration and equity disclosures.
37. Board of Governors of the Federal Reserve System (April 2023). *Review of the Federal Reserve's Supervision and Regulation of Silicon Valley Bank* (Barr review).
38. Reuters (31 March 2023). *India bond yields rise for third straight fiscal on aggressive RBI rate hikes* (10-year yield change in FY2022-23).
39. Reuters (2 March 2023). *India bonds: yields marginally rise tracking U.S. peers* (cross-check of benchmark yield levels).
40. ICMR case catalogue, *Interest Rate Changes and Duration Management* (10-year G-sec yield 7.591% in February 2019 and 6.013% in May 2020).
41. Reserve Bank of India, Monetary Policy Committee statements, 2019-20 and 2022-23 (repo rate path).
42. SEBI circulars of 10 March 2021 and 22 March 2021 on investment and valuation norms for perpetual (AT1 and Tier 2) bonds; trade-press summaries consulted: Bloomberg Quint, Business Standard, Value Research, Money9.
43. Yes Bank reconstruction scheme, March 2020: write-down of AT1 bonds of about INR 8,415 crore (trade and financial press).
44. ZeTheta Algorithms Pvt. Ltd. (2026). *Data Analyst - Convexity Sensitivity AI Agent* (Project 1B brief).

Numerical facts in Part C were checked against the sources above where available online during preparation; items not re-verified are marked in the text. Treat the case-study facts as research notes, not as a substitute for the primary documents.

Appendix: Reproducibility and Archive Contents

Table 99. Code archive layout

Path	Contents
src/bond_pricing.py … var_backtest.py	Eight modules of Section A2
src/test_core.py	14 unit tests
src/run_core.py, extra_calcs.py, build_excel.py, make_figs.py	Pipeline that produces every table, figure and the Excel workbook
src/ml_xgb_keras.py	Production XGBoost/Keras/shap script (untested)
R/*.R	R scripts (untested)
README.md, requirements.txt, .gitignore, .env.example, LICENSE	Repository files